Unit 02: Memory

UVIC

CSC 115: Fundamentals of Programming II

Anthony Estey



Learning Objectives

- Explain the difference between stack and heap memory
- Describe when stack frames are automatically allocated and removed
- Explain why an uninitialized variable can contain unpredictable leftover data
- Declare a pointer and use the address-of operator (&) to store a variable's address
- Explain what a pointer's value actually represents
- Explain what a null pointer is and why it is used
- Use the dereference operator (*) to read and write a value through a pointer
- Distinguish between the declaration and expression uses of * in pointer syntax
- Explain the relationship between & and * as inverse operations
- Trace a program involving pointers across function calls using memory diagrams
- Explain how dangling pointers occur and what the dangers of dereferencing one are
- Declare and use a reference, and explain how it differs from a pointer
- Justify the choice between using a pointer or a reference for a given scenario
- Use a reference in a range-based for loop to read or modify elements of an array
- Explain why passing an array to a function does not copy its contents, while passing a struct by value does.

Learning Objectives

- Apply the three parameter-passing mechanisms: by value, by reference, and by pointer and explain when to use each
- Explain which of them allow a function to modify the caller's variable
- Apply const to variables, parameters, and references to prevent unintended modification
- Explain the benefit of passing by const reference for larger data types
- Explain why dynamic memory allocation is necessary when a data size is not known until runtime
- Explain why heap memory might be used instead of allocating memory on the stack
- Allocate and free memory on the heap using new and delete
- Allocate and free dynamically-sized arrays using new[] and delete[], and explain why they must be paired correctly
- Define a memory leak and explain how one occurs
- Trace a program to identify leaked memory and explain why it became unreachable
- Explain why manual memory management places responsibility entirely on the programmer, with no compiler enforcement

01: The Memory Model

Anthony Estey
CSC 115: Fundamentals of Programming: II

Programming and Memory

- So far, we've written variable after variable without ever asking *where* they actually live.
- In Python, we never had to, because memory management happens invisibly. It’s more explicit in C++, and something we always need to be mindful of as developers

Memory: Two Regions

- There are two main regions where variables are stored:
- **Stack**: where local variables and function calls live.
  - A frame is allocated on the stack when a function is called, and removed from the stack when it returns
  - This is automatic, and how we traced variable scope up to this point (i.e. the variables just go away when the function is finished)
- **Heap**: a separate region for memory that needs to outlive a single function call.
  - We will manually allocate and deallocate this memory (more next week)

Stack Frames

- A stack frame holds all data needed for the function to run: parameters, local variables, return address[†], etc.
- They are allocated on the stack when a function is called and removed when it returns
  - This process is fully automatic (no manual work required)
- Frames stack on top of each other
  - the most recently called function's frame is always on top.
- When a frame is removed, control returns to whatever frame is now on top
  - there's never any ambiguity about where execution resumes.

[†]Return addresses are covered in a course that follows this course

Stack Frames Example

```
void fn2() {
  std::cout << "in fn2" << std::endl;
}

void fn3() {
  std::cout << "in fn3" << std::endl;
}

void fn1() {
  std::cout << "in fn1" << std::endl;
  fn2();
  fn3();
}

int main() {
  fn1();
  return 0;
}
```

Stack Frames Example

```
void fn2() {
  std::cout << "in fn2" << std::endl;
}

void fn3() {
  std::cout << "in fn3" << std::endl;
}

void fn1() {
  std::cout << "in fn1" << std::endl;
  fn2();
  fn3();
}

int main() {
  fn1();
  return 0;
}
```



Stack Frames Example

```
void fn2() {
  std::cout << "in fn2" << std::endl;
}

void fn3() {
  std::cout << "in fn3" << std::endl;
}

void fn1() {
  std::cout << "in fn1" << std::endl;
  fn2();
  fn3();
}

int main() {
  fn1();
  return 0;
}
```



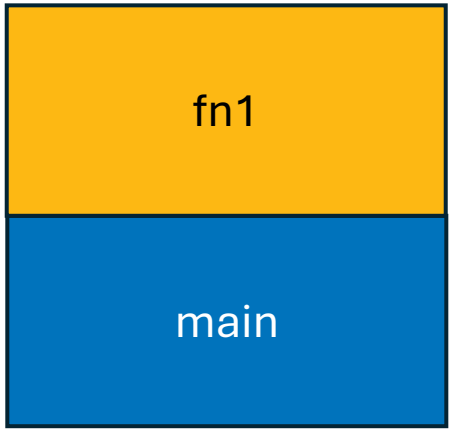
Stack Frames Example

```
void fn2() {
  std::cout << "in fn2" << std::endl;
}

void fn3() {
  std::cout << "in fn3" << std::endl;
}

void fn1() {
  std::cout << "in fn1" << std::endl;
  fn2();
  fn3();
}

int main() {
  fn1();
  return 0;
}
```



Stack Frames Example

```
void fn2() {
  std::cout << "in fn2" << std::endl;
}

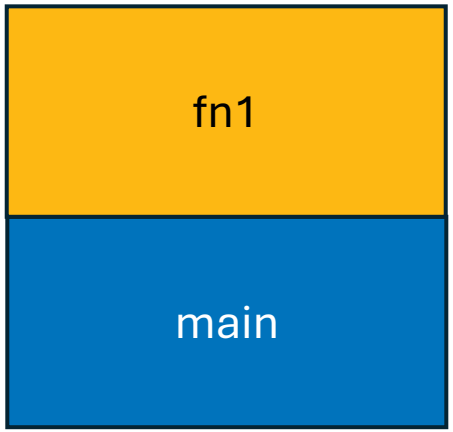
void fn3() {
  std::cout << "in fn3" << std::endl;
}

void fn1() {
  std::cout << "in fn1" << std::endl;
  fn2();
  fn3();
}

int main() {
  fn1();
  return 0;
}
```

Output:

```
in fn1
```



Stack Frames Example

```
void fn2() {
  std::cout << "in fn2" << std::endl;
}

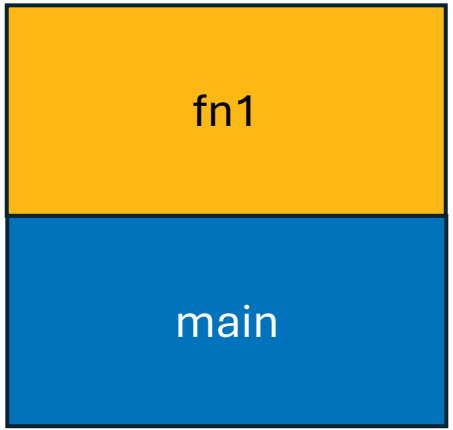
void fn3() {
  std::cout << "in fn3" << std::endl;
}

void fn1() {
  std::cout << "in fn1" << std::endl;
  fn2();
  fn3();
}

int main() {
  fn1();
  return 0;
}
```

Output:

```
in fn1
```



Stack Frames Example

```
void fn2() {
   std::cout << "in fn2" << std::endl;
}

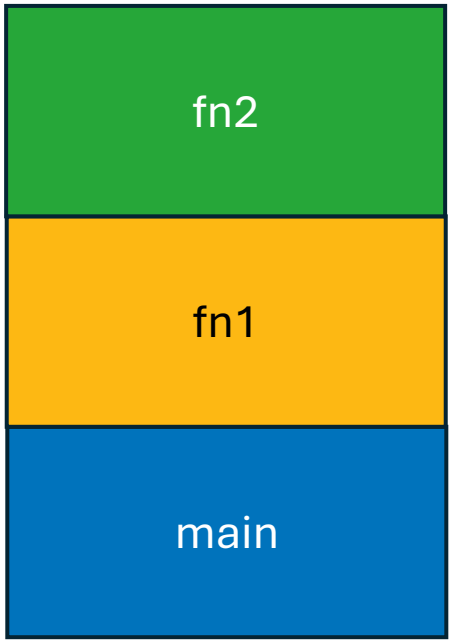
void fn3() {
   std::cout << "in fn3" << std::endl;
}

void fn1() {
   std::cout << "in fn1" << std::endl;
   fn2();
   fn3();
}

int main() {
   fn1();
   return 0;
}
```

Output:

```
in fn1
```



Stack Frames Example

```
void fn2() {
  std::cout << "in fn2" << std::endl;
}

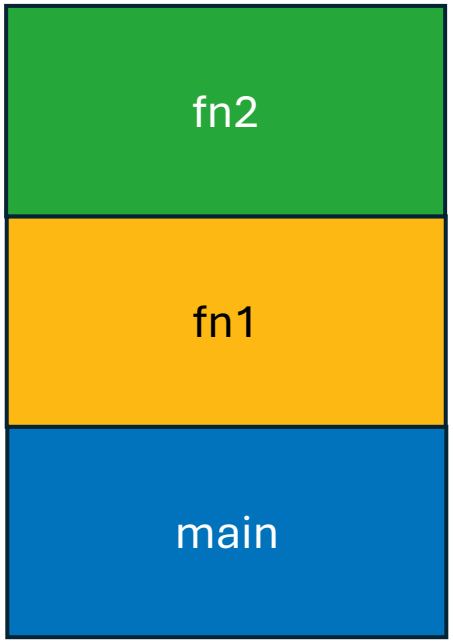
void fn3() {
  std::cout << "in fn3" << std::endl;
}

void fn1() {
  std::cout << "in fn1" << std::endl;
  fn2();
  fn3();
}

int main() {
  fn1();
  return 0;
}
```

Output:

```
in fn1
in fn2
```



Stack Frames Example

```
void fn2() {
   std::cout << "in fn2" << std::endl;
}

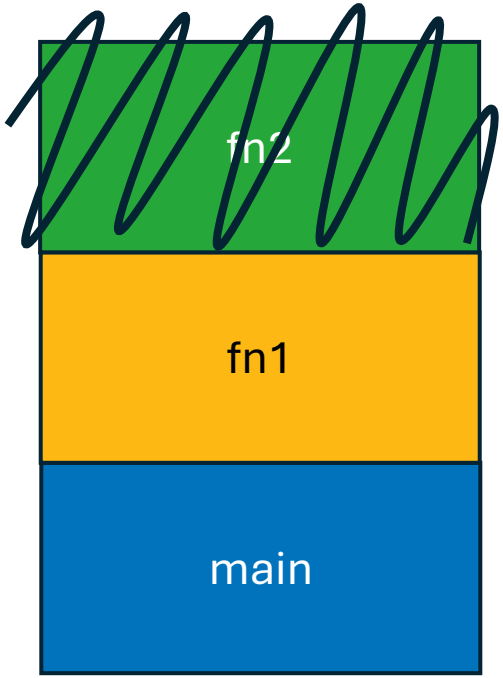
void fn3() {
   std::cout << "in fn3" << std::endl;
}

void fn1() {
   std::cout << "in fn1" << std::endl;
   fn2();
   fn3();
}

int main() {
   fn1();
   return 0;
}
```

Output:

```
in fn1
in fn2
```



Stack Frames Example

```
void fn2() {
  std::cout << "in fn2" << std::endl;
}

void fn3() {
  std::cout << "in fn3" << std::endl;
}

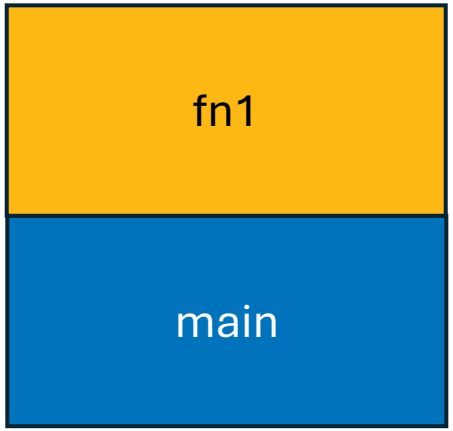
void fn1() {
  std::cout << "in fn1" << std::endl;
  fn2();
  fn3();
}

int main() {
  fn1();
  return 0;
}
```

Output:

```
in fn1
in fn2
```

Note: fn1 is at the top, meaning that is where the execution continues from



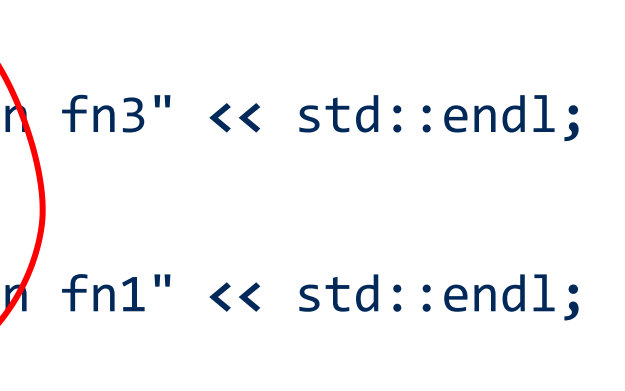
Stack Frames Example

```
void fn2() {
  std::cout << "in fn2" << std::endl;
}

void fn3() {
  std::cout << "in fn3" << std::endl;
}

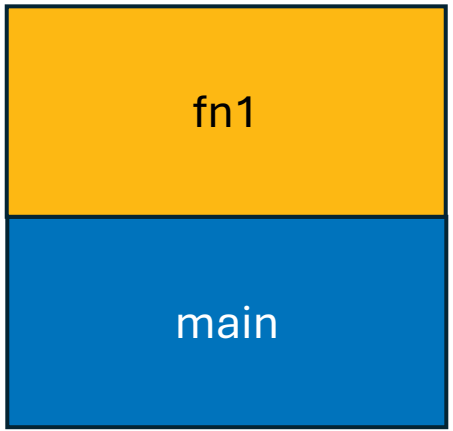
void fn1() {
  std::cout << "in fn1" << std::endl;
  fn2();
  fn3();
}

int main() {
  fn1();
  return 0;
}
```



Output:

```
in fn1
in fn2
```



Stack Frames Example

```
void fn2() {
  std::cout << "in fn2" << std::endl;
}

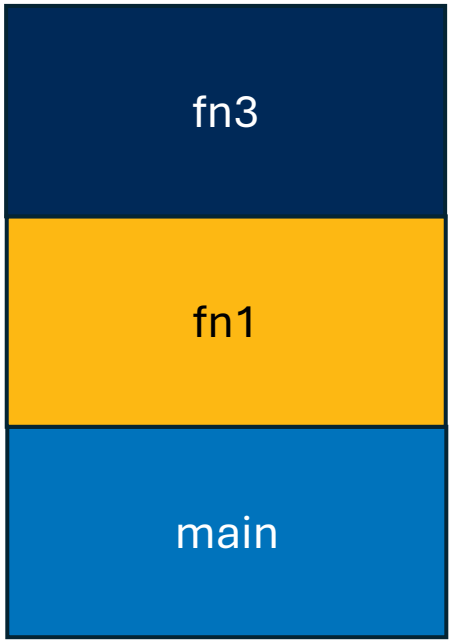
void fn3() {
  std::cout << "in fn3" << std::endl;
}

void fn1() {
  std::cout << "in fn1" << std::endl;
  fn2();
  fn3();
}

int main() {
  fn1();
  return 0;
}
```

Output:

```
in fn1
in fn2
```



Stack Frames Example

```
void fn2() {
  std::cout << "in fn2" << std::endl;
}

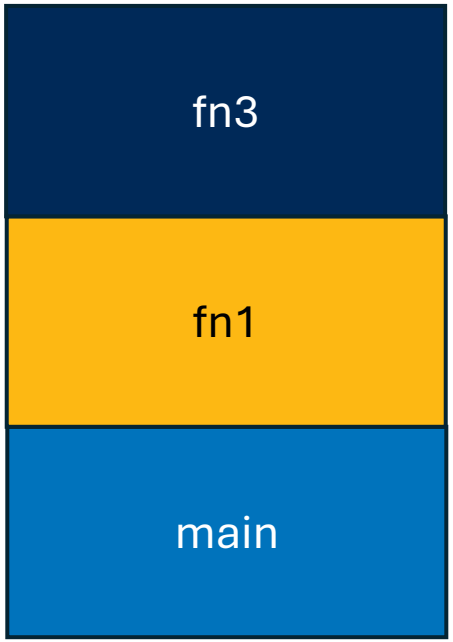
void fn3() {
  std::cout << "in fn3" << std::endl;
}

void fn1() {
  std::cout << "in fn1" << std::endl;
  fn2();
  fn3();
}

int main() {
  fn1();
  return 0;
}
```

Output:

```
in fn1
in fn2
in fn3
```



Stack Frames Example

```
void fn2() {
  std::cout << "in fn2" << std::endl;
}

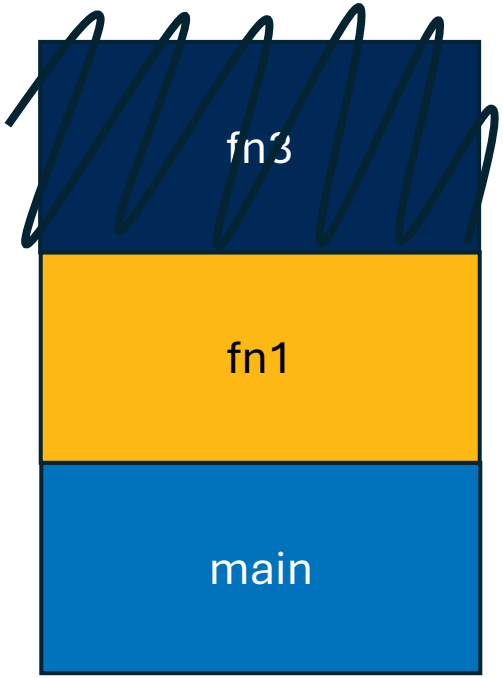
void fn3() {
  std::cout << "in fn3" << std::endl;
}

void fn1() {
  std::cout << "in fn1" << std::endl;
  fn2();
  fn3();
}

int main() {
  fn1();
  return 0;
}
```

Output:

```
in fn1
in fn2
in fn3
```



Stack Frames Example

```
void fn2() {
  std::cout << "in fn2" << std::endl;
}

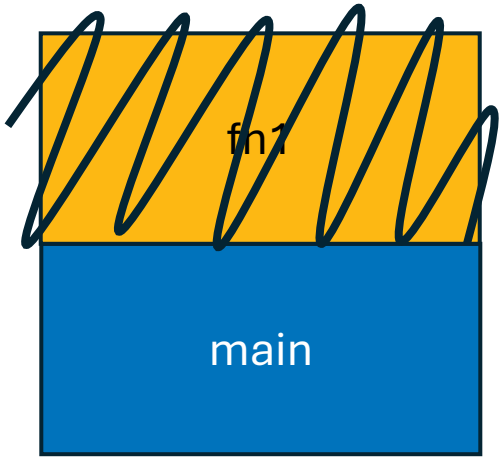
void fn3() {
  std::cout << "in fn3" << std::endl;
}

void fn1() {
  std::cout << "in fn1" << std::endl;
  fn2();
  fn3();
}

int main() {
  fn1();
  return 0;
}
```

Output:

```
in fn1
in fn2
in fn3
```



Stack Frames Example

```
void fn2() {
  std::cout << "in fn2" << std::endl;
}

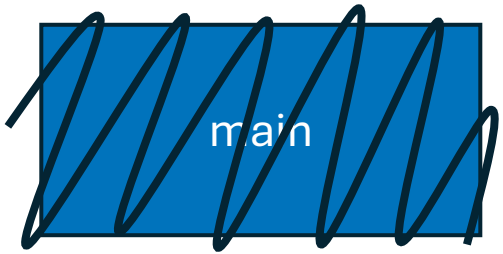
void fn3() {
  std::cout << "in fn3" << std::endl;
}

void fn1() {
  std::cout << "in fn1" << std::endl;
  fn2();
  fn3();
}

int main() {
  fn1();
  return 0;
}
```

Output:

```
in fn1
in fn2
in fn3
```



Stack Frames and Memory

- In these animations, when we finish a function I erased the stack frame. When this happens, all variables declared in that function are erased too (sort of how I erase the whole function's memory diagram in earlier tracing examples)
  - But is the memory wiped clean?
- No, it is actually just marked as available for reuse
  - Whatever is physically in that memory location is left there until we need to use that memory for something else later and overwrite it
- This is a very subtle but important thing to remember
  - Because most of the time it appears like it actually is wiped clean

Another Stack Frame Example

```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```

Another Stack Frame Example

```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```

Memory:

	Address	Value
	0x9000	
	0x9004	
	0x9008	

Another Stack Frame Example

```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```

Memory:

	Address	Value
	0x9000	
	0x9004	
main:	0x9008	

Another Stack Frame Example

```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```

Memory:

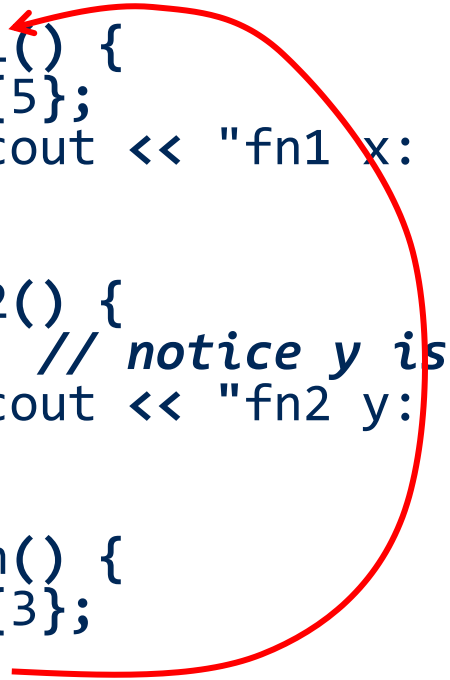
	Address	Value
	0x9000	
	0x9004	
main: x	0x9008	3

Another Stack Frame Example

```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```



Memory:

	Address	Value
	0x9000	
	0x9004	
main: x	0x9008	3

Another Stack Frame Example

```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```

Memory:

		Address	Value
		0x9000	
fn1:		0x9004	
main:	x	0x9008	3

Another Stack Frame Example

```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```

Memory:

		Address	Value
		0x9000	
fn1:	x	0x9004	5
main:	x	0x9008	3

Another Stack Frame Example

```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```

Output:

```
fn1 x: 5
```

Memory:

		Address	Value
		0x9000	
fn1:	x	0x9004	5
main:	x	0x9008	3

Another Stack Frame Example

```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```

Output:

```
fn1 x: 5
```

Memory:

		Address	Value
		0x9000	
fn1:	x	0x9004	5
main:	x	0x9008	3

Another Stack Frame Example

```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```

Although the x variable declared in fn1 goes out of scope, the value 5 at memory address 0x9004 is left in that memory location.

Output:

```
fn1 x: 5
```

Memory:

	Address	Value
	0x9000	
	0x9004	5
main: x	0x9008	3

Another Stack Frame Example

```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```

Output:

```
fn1 x: 5
```

Memory:

	Address	Value
	0x9000	
	0x9004	5
main: x	0x9008	3

Another Stack Frame Example

```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```



Output:

```
fn1 x: 5
```

Memory:

	Address	Value
	0x9000	
	0x9004	5
main: x	0x9008	3

Another Stack Frame Example

```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```

Output:

```
fn1 x: 5
```

Memory:

	Address	Value
	0x9000	
fn2:	0x9004	5
main: x	0x9008	3

Another Stack Frame Example

```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```

The y in fn2 is allocated to the same address as x from fn1. Since y is not initialized, it inherits the value that was left in that address.

Output:

```
fn1 x: 5
```

Memory:

		Address	Value
		0x9000	
fn2:	y	0x9004	5
main:	x	0x9008	3

Another Stack Frame Example

```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```

Output:

```
fn1 x: 5
fn2 y: 5
```

Memory:

		Address	Value
		0x9000	
fn2:	y	0x9004	5
main:	x	0x9008	3

Another Stack Frame Example

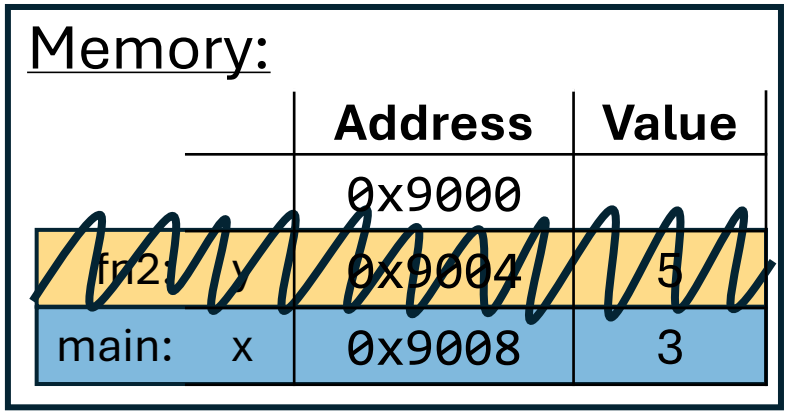
```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```

Output:

```
fn1 x: 5
fn2 y: 5
```



Another Stack Frame Example

```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```

Output:

```
fn1 x: 5
fn2 y: 5
```

Memory:

	Address	Value
	0x9000	
	0x9004	5
main: x	0x9008	3

Another Stack Frame Example

```
void fn1() {
  int x{5};
  std::cout << "fn1 x: " << x << std::endl;
}

void fn2() {
  int y; // notice y is not initialized
  std::cout << "fn2 y: " << y << std::endl;
}

int main() {
  int x{3};
  fn1();
  fn2();
  return 0;
}
```

Key Takeaway: Stack memory is handled automatically so it's not something we often need to think about. That being said, an uninitialized variable can silently contain leftover data from a previous frame, so we must *always* initialize our variables.

02: Pointers

Anthony Estey
CSC 115: Fundamentals of Programming: II

Introduction to Pointers

- We've seen that we store variables in memory addresses
  - this allocation happens automatically when we declare a variable…
  - but we have not yet interacted with an address directly
- For certain tasks, we will need to work with addresses
  - similar to how we use variables to store other values in our programs, we can store address values in a variable as well
- That's what a **pointer** is: a variable whose value is a memory address
  - We will use pointers in slightly different ways than regular variables

Pointers Motivation Analogy

- Scenario: You heard about a great quote in a book at my house, and you really want to read it!
- Possible Solutions:
  - I could make a copy of everything in my house and give that to you.
  - I could give you directions to my house and you can go there and find it yourself.
- Tradeoffs:
  - Copying everything in my house is a lot of work, and not efficient with resources.
  - Giving you access to my house means you can do whatever you want in there.
- Using pointers has this same tradeoff, we can give access to a location, allowing the contents of that location to be shared, but that also means the contents can be changed!

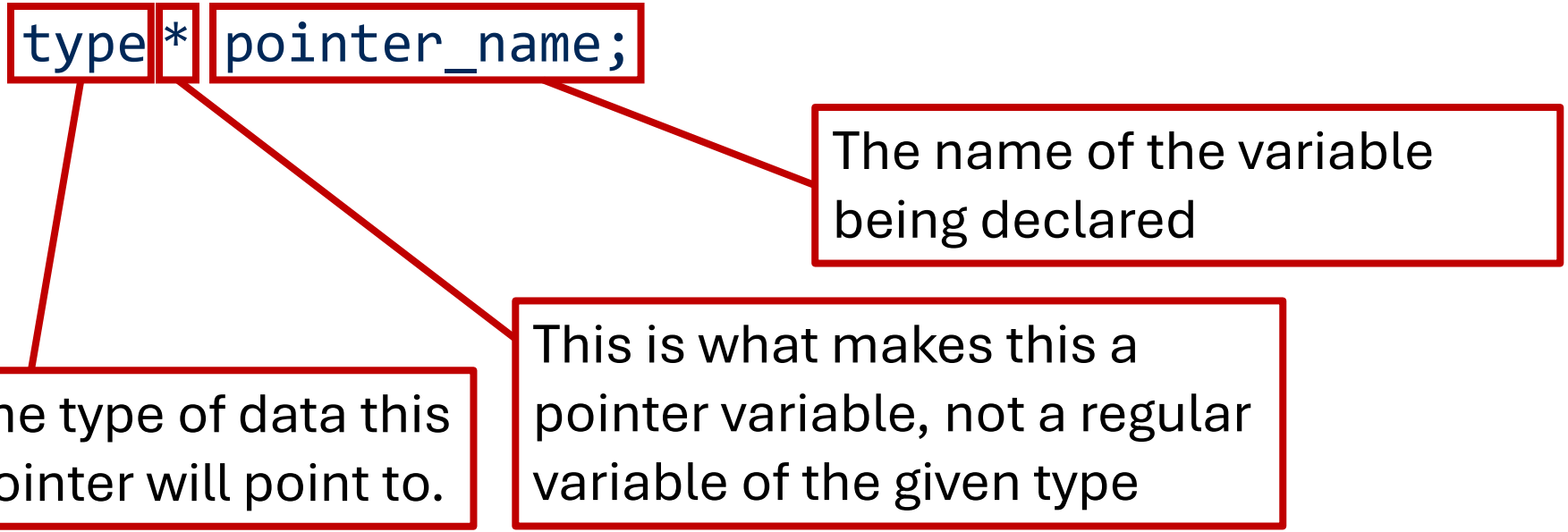
Pointers Motivation with Code

- If we consider the analogy with code, we could assume:
  - We want to pass some information to a function (from me to you)
    - With regular variables, we have seen that we can copy a value to a function when we pass it as a parameter
  - The problem:
    - What if we need to pass a LOT of data (millions or billions of bytes)?
  - Options:
    - Copy that massive amount of data and pass the massive copy
    - Give a pointer to where the data is stored, so you can access it yourself

Key Takeaway: It comes down to creating an extra copy and sharing it, or sharing the location of where the value is stored

Pointers Syntax

- The syntax for a pointer is very similar to regular variables:
- General Syntax:



Pointers Syntax

- The syntax for a pointer is very similar to regular variables:
- General Syntax:

```
type * pointer_name;
```

- General Syntax:

```
int * int_ptr;
double* dbl_ptr;
```

`int_ptr` will point to an int value

`dbl_ptr` will point to an double value

Problem: These pointers aren't initialized. This creates the same danger as an uninitialized variable!

Pointers Syntax

- We typically want a value in memory to point to before using a pointer:
- General Syntax:

```
type var_name{value};
type* pointer_name{address of var_name};
```

These types must be the same

Remaining question: How do we access the address of a variable?

New syntax related to pointers

- Since pointers store addresses, we need to be able to access the memory address where a value is stored:
  - The & is the **address-of** operator, used to get the address of a variable.
- During variable **declaration**, the * indicates a pointer variable:
  - `int* ptr;` *//Declares a new pointer named ptr to points to an* `int` *value*
- We need `ptr` to point to an integer. To properly initialize it during its declaration, we use the address-of operator (**&**):
  - `int x{5};` *// Declares an integer variable, x, that stores the value 5*
  - `int* ptr{&x};` *// Declares a new pointer, ptr, that stores the address of x*

Pointers Takeaways

- **Pointer** variables store the **memory address** of another variable
- Instead of holding a direct value, they **point** to a location in memory where that value is stored.
  - When we don't want a pointer pointing anywhere, we can set it to `nullptr`; this is called a **null pointer**.
- **Benefit**: Pointers help us save memory and manage large amounts of data without using unnecessary space.
  - Instead of copying a large amount of data, pointers allow us to point to the original data. This is like sharing a reference to a book in the library instead of making a copy of the entire book.
- **Usage:** A pointer is a variable whose value is a memory address. The address-of operator (&) is how we obtain an existing variable's address to store in a pointer.

03: Dereferencing

Anthony Estey
CSC 115: Fundamentals of Programming: II

Pointers

- We now know that we can create pointer variables to hold the address of different values for us
  - This is really useful when working with large data types or collections of values (like an array)

- We haven't yet seen how to access the value at the memory location where the pointer is pointing to

Dereferencing a Pointer

- Assume we have the following setup, where we have an integer, x, and a pointer, `int_ptr`, pointing to x's memory address:

```
int x{5};
int* int_ptr{&x};
```

- Our `int_ptr` is now storing the address of x. Remember, the value `int_ptr` is storing is an address, not the 5.
- To access the *value* at that address, we use the **dereference operator** (*):

```
std::cout << int_ptr << std::endl;  // outputs x's address
std::cout << *int_ptr << std::endl; // outputs 5
```

Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```

Memory:

	Address	Value
	0x1000	
	0x1004	
	0x1008	
	0x100c	

Output:

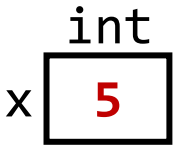
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	5
	0x1004	
	0x1008	
	0x100c	

Output:

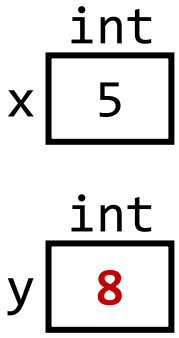
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	5
y	0x1004	8
	0x1008	
	0x100c	

Output:

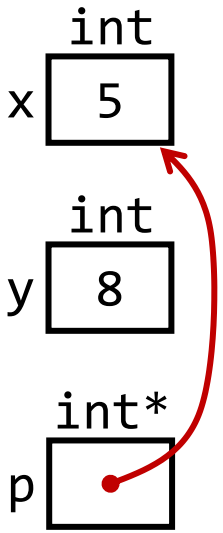
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	5
y	0x1004	8
p	0x1008	0x1000
	0x100c	

Output:

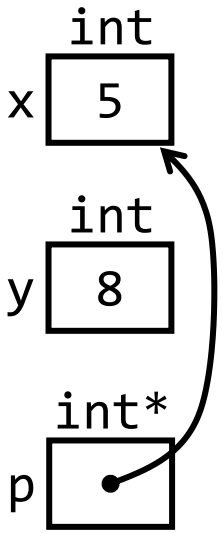
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	5
y	0x1004	8
p	0x1008	0x1000
	0x100c	

Output:

5

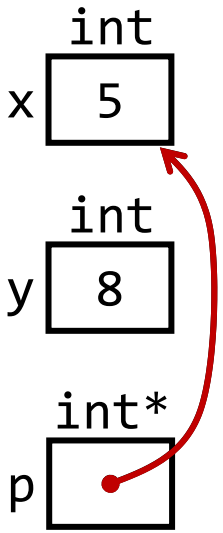
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	5
y	0x1004	8
p	0x1008	0x1000
	0x100c	

Output:

```
5
0x1000
```

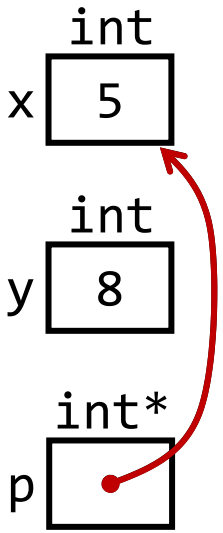
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	5
y	0x1004	8
p	0x1008	0x1000
	0x100c	

Output:

```
5
0x1000
5
```

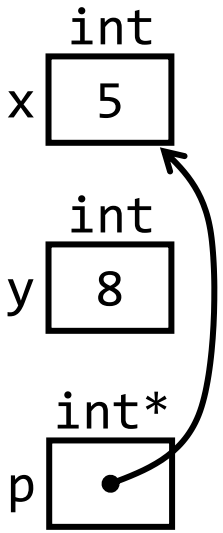
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	5
y	0x1004	8
p	0x1008	0x1000
	0x100c	

Output:

```
5
0x1000
5
```

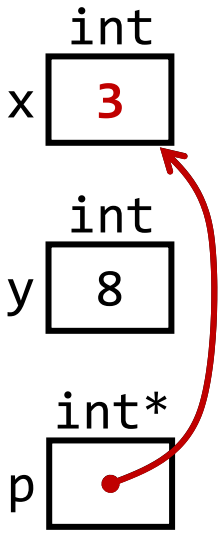
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	5
y	0x1004	8
p	0x1008	0x1000
	0x100c	

Output:

```
5
0x1000
5
```

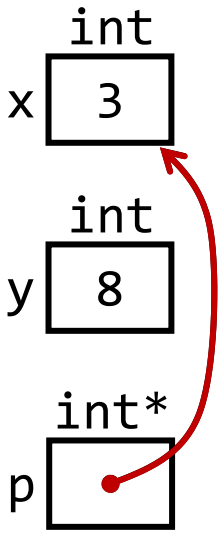
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	3
y	0x1004	8
p	0x1008	0x1000
	0x100c	

Output:

```
5
0x1000
5
```

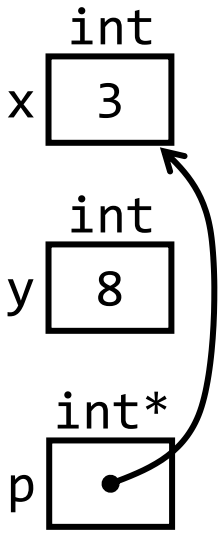
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	3
y	0x1004	8
p	0x1008	0x1000
	0x100c	

Output:

```
5
0x1000
5
```

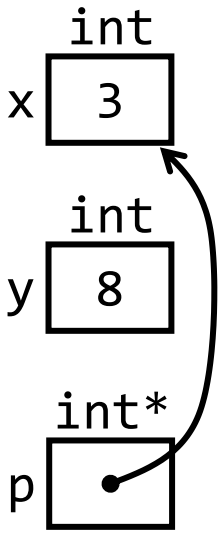
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	3
y	0x1004	8
p	0x1008	0x1004
	0x100c	

Output:

```
5
0x1000
5
```

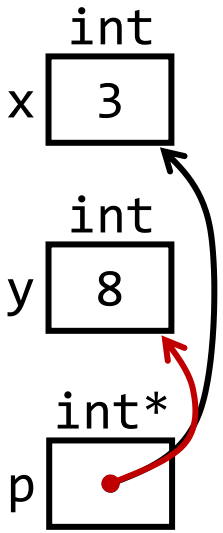
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	3
y	0x1004	8
p	0x1008	0x1004
	0x100c	

Output:

```
5
0x1000
5
```

Pointers & Dereferencing Example

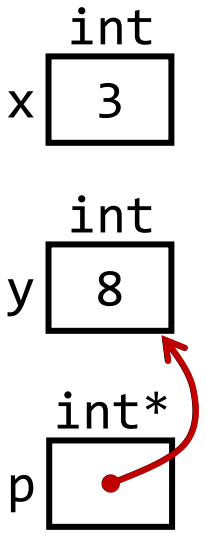
```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```

10



Memory:

	Address	Value
x	0x1000	3
y	0x1004	8
p	0x1008	0x1004
	0x100c	

Output:

```
5
0x1000
5
```

Pointers & Dereferencing Example

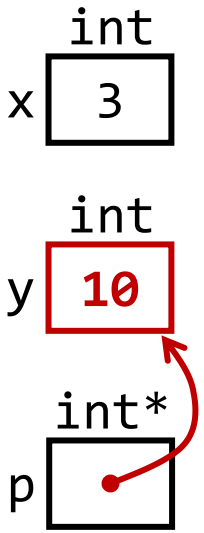
```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```

10



Memory:

	Address	Value
x	0x1000	3
y	0x1004	8
p	0x1008	0x1004
	0x100c	

Output:

```
5
0x1000
5
```

Pointers & Dereferencing Example

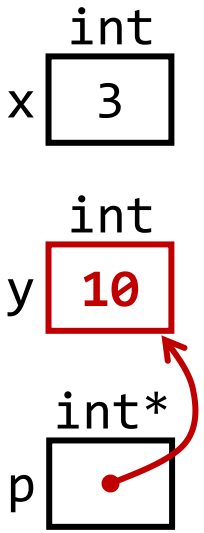
```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```

10



Memory:

	Address	Value
x	0x1000	3
y	0x1004	10
p	0x1008	0x1004
	0x100c	

Output:

```
5
0x1000
5
```

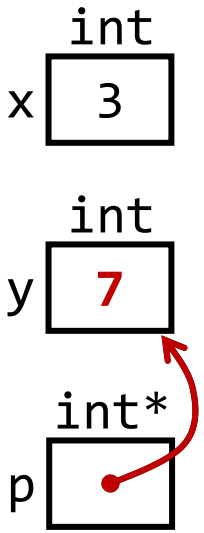
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	3
y	0x1004	10
p	0x1008	0x1004
	0x100c	

Output:

```
5
0x1000
5
```

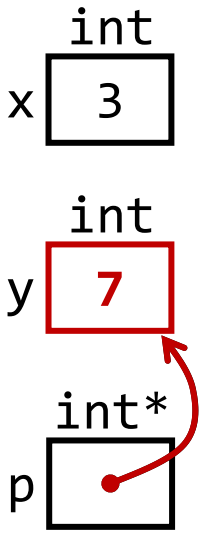
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	3
y	0x1004	7
p	0x1008	0x1004
	0x100c	

Output:

```
5
0x1000
5
```

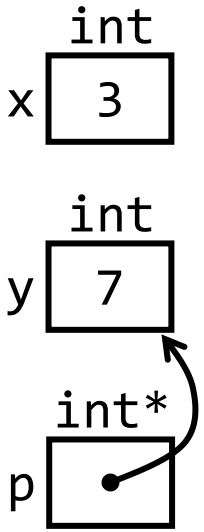
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	3
y	0x1004	7
p	0x1008	0x1004
	0x100c	

Output:

```
5
0x1000
5
```

3

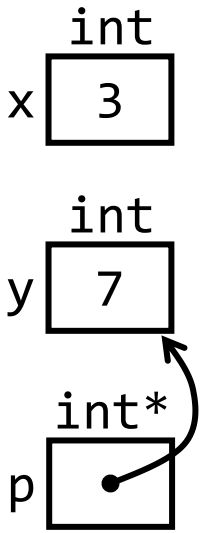
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	3
y	0x1004	7
p	0x1008	0x1004
	0x100c	

Output:

```
5                3
0x1000           7
5
```

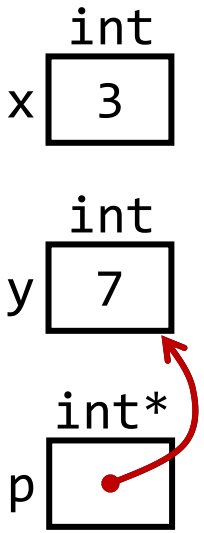
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	3
y	0x1004	7
p	0x1008	0x1004
	0x100c	

Output:

```
5
0x1000
5
3
7
0x1004
```

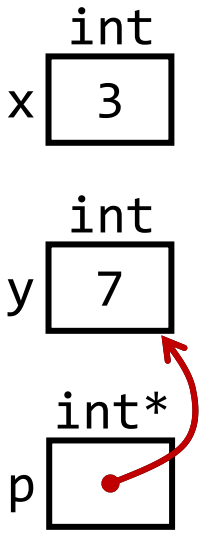
Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```



Memory:

	Address	Value
x	0x1000	3
y	0x1004	7
p	0x1008	0x1004
	0x100c	

Output:

```
5
0x1000
5
3
7
0x1004
7
```

Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```

The * character is used in two different ways in this program. First, it is used to **declare** the pointer variable p, and later, it is used to **dereference** it.

Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```

Whenever we want to assign a new **value** to the memory location a pointer is pointing to, we use the **dereference** operator on the left side of the assignment statement.

Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```

Similarly, we **dereference** a pointer whenever we want to **access** the value the pointer is pointing to.

Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```

Whenever we want to **change where the pointer points to**, we assign it to point to a **different address** using the **address-of** operator (&).

Pointers & Dereferencing Example

```
int x{5};
int y{8};
int* p{&x};

std::cout << x << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;

*p = 3;
p = &y;
y = *p + 2;
*p -= x;

std::cout << x << std::endl;
std::cout << y << std::endl;
std::cout << p << std::endl;
std::cout << *p << std::endl;
```

* and **&** can be thought of as inverses of each other:

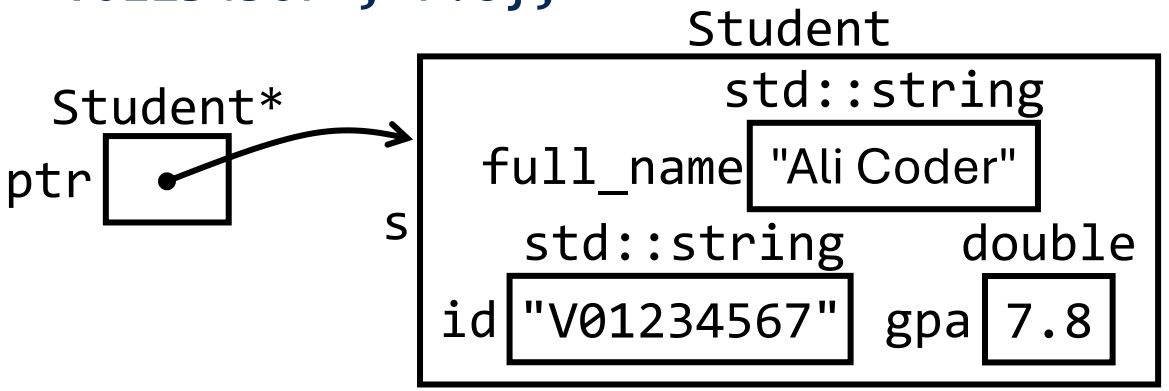
- To get the value at an address: *
- To get the address of a variable: **&**

Dereferencing a Struct

- When working with a struct, there is a shorthand notation for accessing its members: the **arrow operator** (**->**)
  - This is equivalent to a dereference followed by using dot notation:

```
Student s{"Ali Coder", "V01234567", 7.8};
Student* ptr{&s};

s.full_name;
(*ptr).full_name;
ptr->full_name;
```



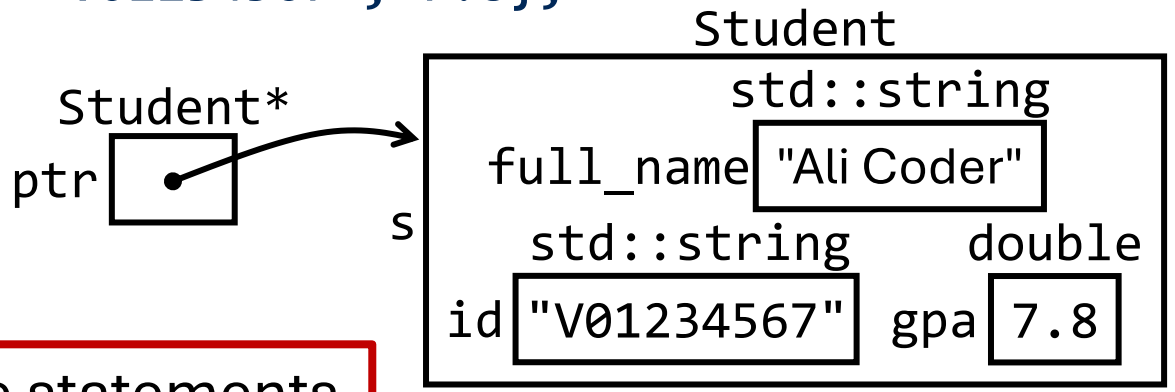
Dereferencing a Struct

- When working with a struct, there is a shorthand notation for accessing its members: the **arrow operator** (**->**)
  - This is equivalent to a dereference followed by using dot notation:

```
Student s{"Ali Coder", "V01234567", 7.8};
Student* ptr{&s};
```

```
s.full_name;
(*ptr).full_name;
ptr->full_name;
```

All three of these statements access the "Ali Coder" string.

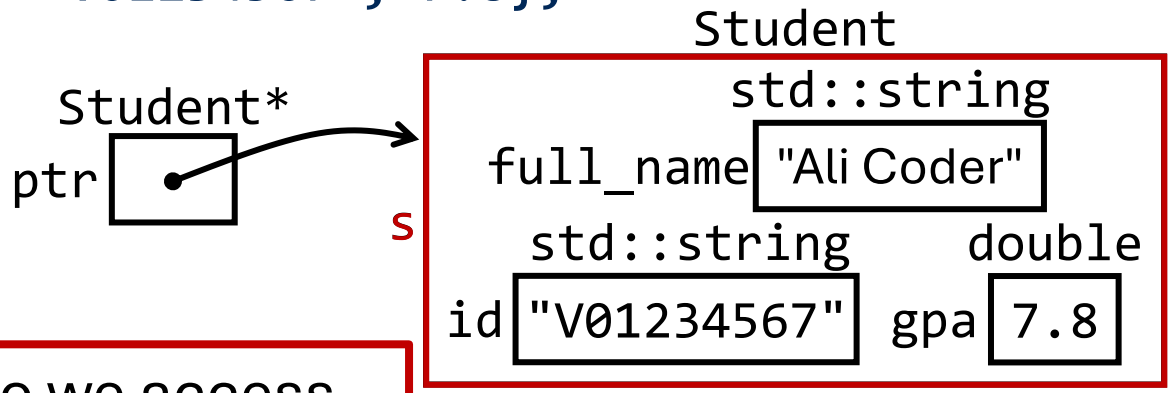


Dereferencing a Struct

- When working with a struct, there is a shorthand notation for accessing its members: the **arrow operator** (->)
  - This is equivalent to a dereference followed by using dot notation:

```
Student s{"Ali Coder", "V01234567", 7.8};
Student* ptr{&s};

s.full_name;
(*ptr).full_name;
ptr->full_name;
```



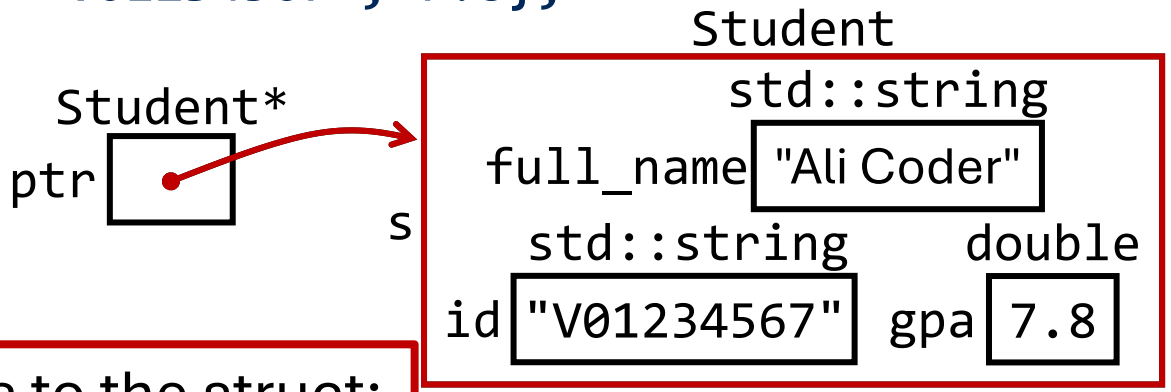
s is the struct variable, so we access its members directly with dot notation

Dereferencing a Struct

- When working with a struct, there is a shorthand notation for accessing its members: the **arrow operator** (->)
  - This is equivalent to a dereference followed by using dot notation:

```
Student s{"Ali Coder", "V01234567", 7.8};
Student* ptr{&s};

s.full_name;
(*ptr).full_name;
ptr->full_name;
```



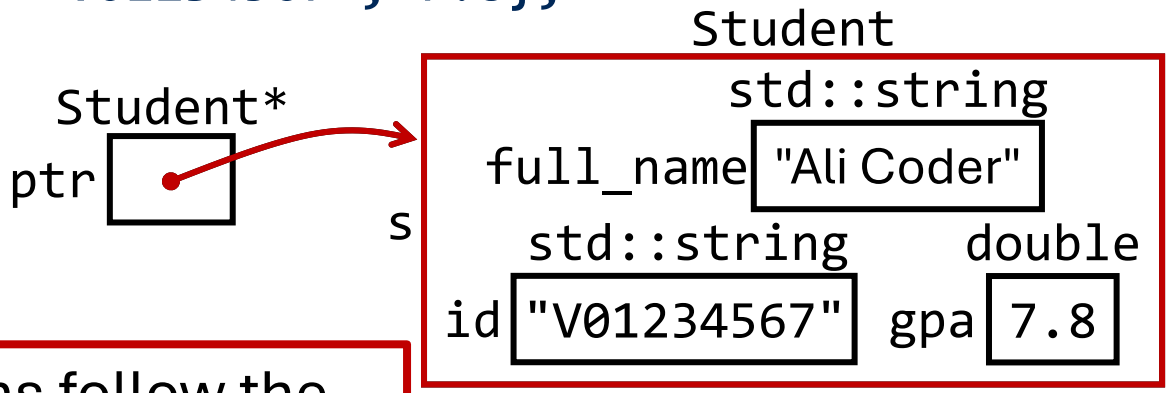
Dereferencing `ptr` gets us to the struct; dot notation then accesses the member

Dereferencing a Struct

- When working with a struct, there is a shorthand notation for accessing its members: the **arrow operator** (->)
  - This is equivalent to a dereference followed by using dot notation:

```
Student s{"Ali Coder", "V01234567", 7.8};
Student* ptr{&s};

s.full_name;
(*ptr).full_name;
ptr->full_name;
```



The arrow notation means follow the pointer and then get the data member

04: Tracing with Functions and Pointers

Anthony Estey
CSC 115: Fundamentals of Programming: II

Functions and Pointers

- We've learned to follow a pointer both ways:
  - get an address with **&**
  - get the value at an address with ***
- But what if the address a pointer holds stops being valid? Let's trace a case where that happens.

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}

void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}

int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}

void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}

int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```

Memory:

	Address	Value
	0x9000	
	0x9004	
	0x9008	

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}

void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}

int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```

Memory:

	Address	Value
	0x9000	
	0x9004	
main:	0x9008	

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}

void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}

int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```

Memory:

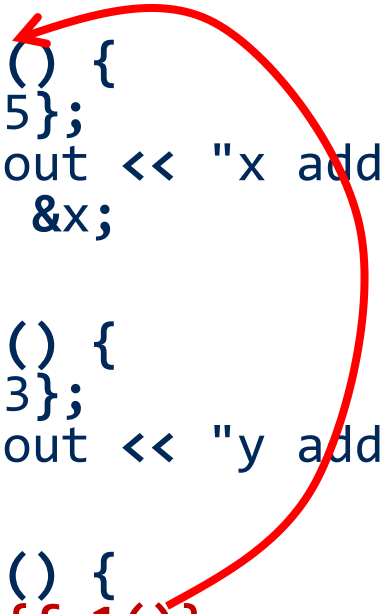
		Address	Value
		0x9000	
		0x9004	
main:	a	0x9008	

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}

void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}

int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```



Memory:

		Address	Value
		0x9000	
		0x9004	
main:	a	0x9008	

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}

void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}

int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```

Memory:

		Address	Value
		0x9000	
fn1:		0x9004	
main:	a	0x9008	

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}

void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}

int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```

Memory:

		Address	Value
		0x9000	
fn1:	x	0x9004	5
main:	a	0x9008	

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}

void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}

int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```

Output:

```
x addr: 0x9004
```

Memory:

		Address	Value
		0x9000	
fn1:	x	0x9004	5
main:	a	0x9008	

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}

void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}

int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```

0x9004

Output:

```
x addr: 0x9004
```

Memory:

		Address	Value
		0x9000	
fn1:	x	0x9004	5
main:	a	0x9008	0x9004

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}

void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}

int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```

Output:

```
x addr: 0x9004
5
```

Memory:

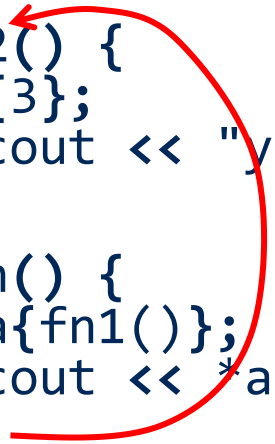
		Address	Value
		0x9000	
		0x9004	5
main:	a	0x9008	0x9004

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}

void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}

int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```



Output:

```
x addr: 0x9004
5
```

Memory:

		Address	Value
		0x9000	
		0x9004	5
main:	a	0x9008	0x9004

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}
void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}
int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```

Output:

```
x addr: 0x9004
5
```

Memory:

		Address	Value
		0x9000	
fn2:		0x9004	
main:	a	0x9008	0x9004

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}

void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}

int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```

Output:

```
x addr: 0x9004
5
```

Memory:

		Address	Value
		0x9000	
fn2:	y	0x9004	3
main:	a	0x9008	0x9004

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}

void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}

int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```

Output:

```
x addr: 0x9004
5
y addr: 0x9004
```

Memory:

		Address	Value
		0x9000	
fn2:	y	0x9004	3
main:	a	0x9008	0x9004

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}

void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}

int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```

Output:

```
x addr: 0x9004
5
y addr: 0x9004
```

Memory:

		Address	Value
		0x9000	
fn2:	y	0x9004	3
main:	a	0x9008	0x9004

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}
void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}
int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```

Output:

```
x addr: 0x9004
5
y addr: 0x9004
3
```

Memory:

		Address	Value
		0x9000	
		0x9004	3
main:	a	0x9008	0x9004

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}

void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}

int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```

Output:

```
x addr: 0x9004
5
y addr: 0x9004
3
```

Memory:

	Address	Value
	0x9000	
	0x9004	3
main: a	0x9008	0x9004

Functions and Pointers Example

```
int* fn1() {
  int x{5};
  std::cout << "x addr: " << &x << std::endl;
  return &x;
}

void fn2() {
  int y{3};
  std::cout << "y addr: " << &y << std::endl;
}

int main() {
  int* a{fn1()};
  std::cout << *a << std::endl;
  fn2();
  std::cout << *a << std::endl;
  return 0;
}
```

Output:

```
x addr: 0x9004
5
y addr: 0x9004
3
```

Key Takeaway: A pointer can continue to point to an address after the object that lived there is gone. Dereferencing it may not crash the program, but instead could return a plausible-looking, incorrect value, because the memory got reused by something else.

05: References

Anthony Estey
CSC 115: Fundamentals of Programming: II

References

- Pointers are powerful, and we will use them when we pass arrays and other objects to functions
- We have also seen there are some things we need to be aware of when using them:
  - they can be uninitialized (they aren't pointing anywhere)
  - they can dangle (they are pointing to something that is gone)
  - Sometimes it can be difficult to remember how to use * and &
- What if we could give a function direct access to a variable, without that syntax overhead or that danger?
- That's what a **reference** is.

References Syntax

- General syntax:

```
type a_variable{initial_value};
type& reference_name{a_variable};
```

When learning about pointers we saw that the * character is used in two different ways. It is first used to **declare** a pointer variable, and later used to **dereference** that variable.

A similar thing is happening here. The **&** here doesn't mean *"give me an address"*, because when **&** is used in a variable declaration it instead means: *"this is a reference"*.

References Syntax

- General syntax:

```
type a_variable{initial_value};
type& reference_name{a_variable};
```

- Specific example:

```
int x{5};
int& ref{x};
```

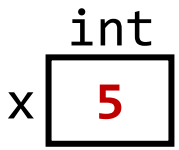
References Syntax

- General syntax:

```
type a_variable{initial_value};
type& reference_name{a_variable};
```

- Specific example:

```
int x{5};
int& ref{x};
```



Memory:

	Address	Value
x	0x1000	5
	0x1004	

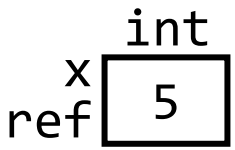
References Syntax

- General syntax:

```
type a_variable{initial_value};
type& reference_name{a_variable};
```

- Specific example:

```
int x{5};
int& ref{x};
```



Memory:

	Address	Value
ref x	0x1000	5
	0x1004	

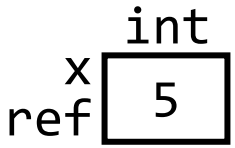
References Syntax

- General syntax:

```
type a_variable{initial_value};
type& reference_name{a_variable};
```

- Specific example:

```
int x{5};
int& ref{x};
```



Memory:

	Address	Value
ref x	0x1000	5
	0x1004	

Reference variables must be initialized to a pre-existing variable at declaration

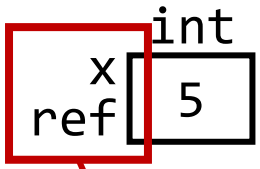
References Syntax

- General syntax:

```
type a_variable{initial_value};
type& reference_name{a_variable};
```

- Specific example:

```
int x{5};
int& ref{x};
```



Memory:

	Address	Value
ref x	0x1000	5
	0x1004	

Reference variables are not allocated a new memory location, they are a second label on an existing one.

References Example

- The following example illustrates why we might use references when looping through an array:

```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```

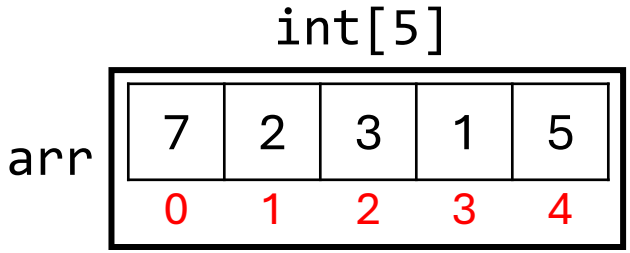
References Example

- The following example illustrates why we might use references when looping through an array:

```
int arr[]{7, 2, 3, 1, 5};

for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}

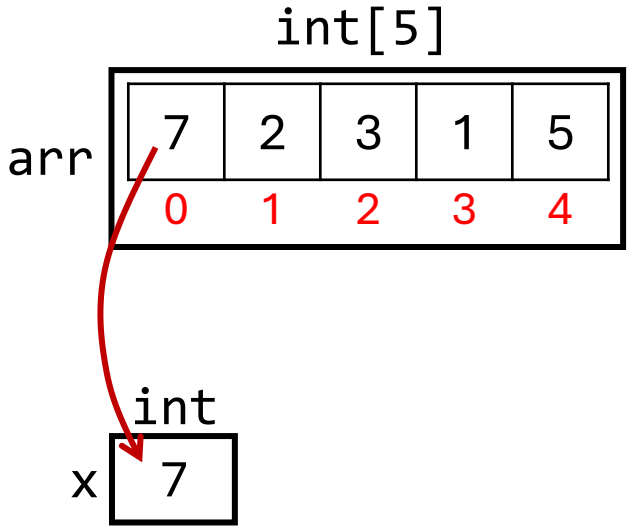
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```



References Example

- The following example illustrates why we might use references when looping through an array:

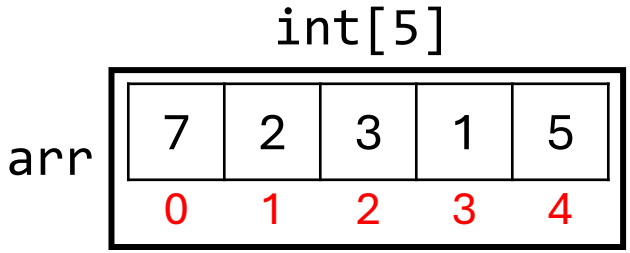
```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```

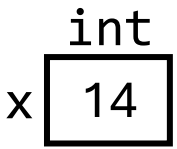


References Example

- The following example illustrates why we might use references when looping through an array:

```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```

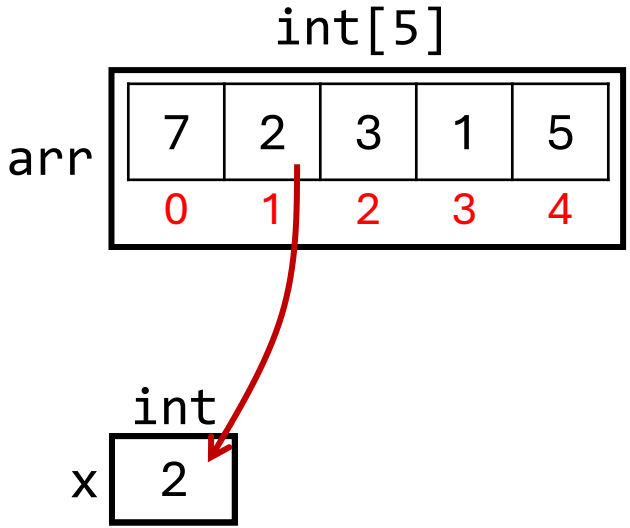




References Example

- The following example illustrates why we might use references when looping through an array:

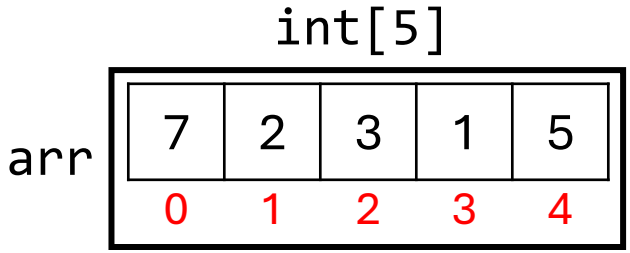
```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```

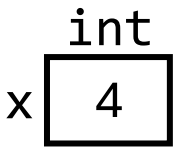


References Example

- The following example illustrates why we might use references when looping through an array:

```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```

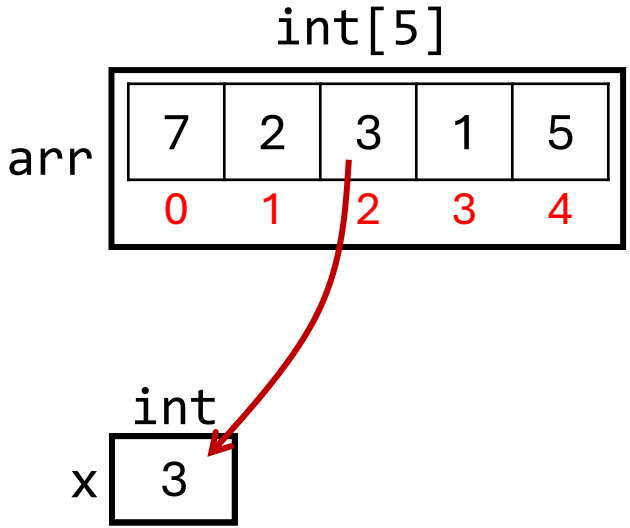




References Example

- The following example illustrates why we might use references when looping through an array:

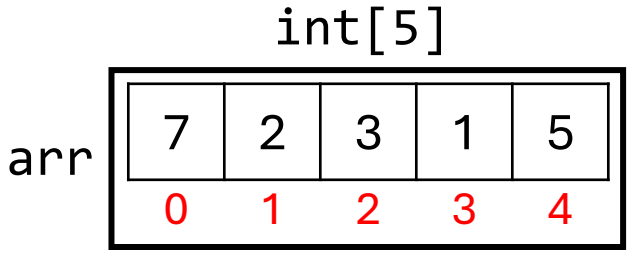
```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```

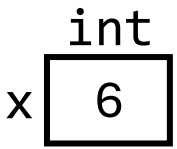


References Example

- The following example illustrates why we might use references when looping through an array:

```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```

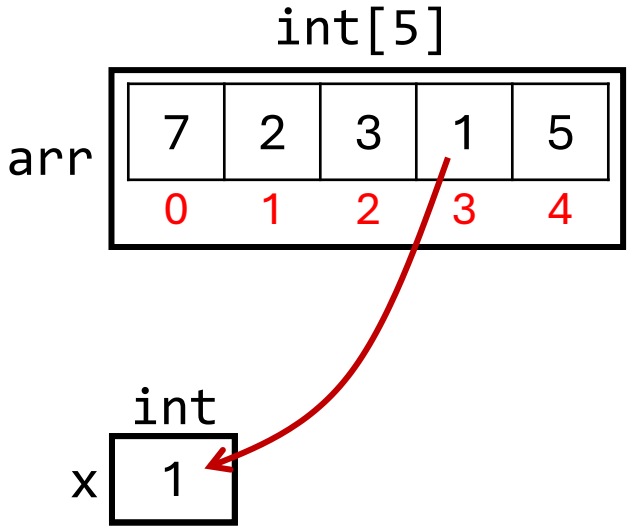




References Example

- The following example illustrates why we might use references when looping through an array:

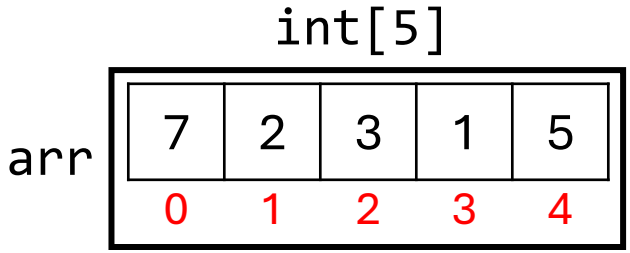
```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```

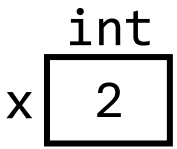


References Example

- The following example illustrates why we might use references when looping through an array:

```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```

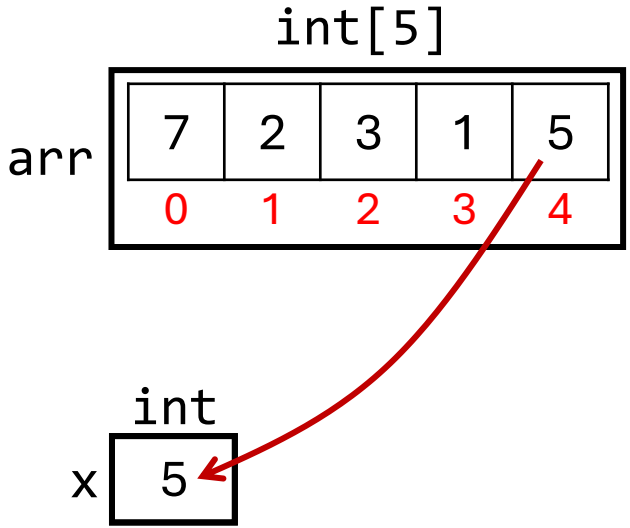




References Example

- The following example illustrates why we might use references when looping through an array:

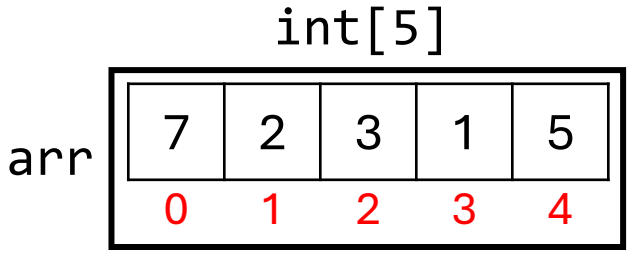
```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```

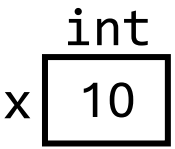


References Example

- The following example illustrates why we might use references when looping through an array:

```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```

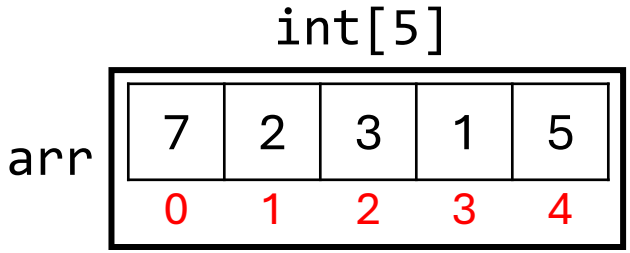




References Example

- The following example illustrates why we might use references when looping through an array:

```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```

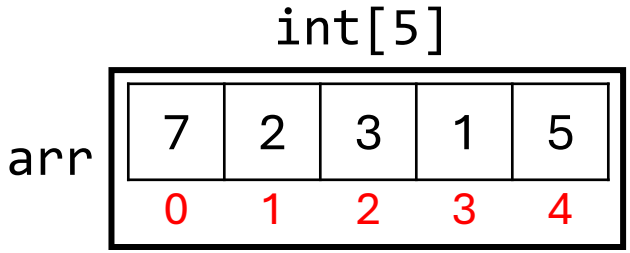


Note: When the first loop finishes, the array contents have not been modified at all

References Example

- The following example illustrates why we might use references when looping through an array:

```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```

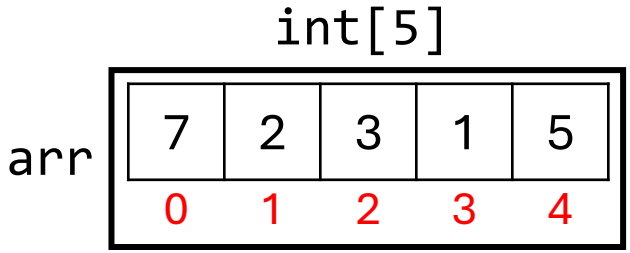


Reference variables do not get their own box, they are another label on a pre-existing allocation.

References Example

- The following example illustrates why we might use references when looping through an array:

```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```

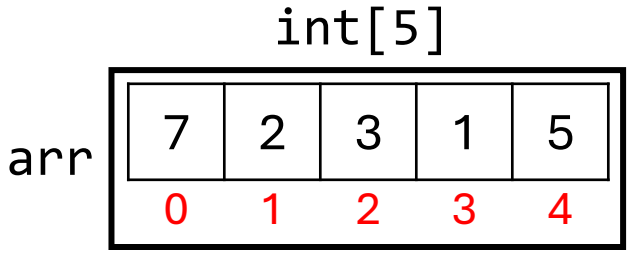


Since `int&` x is declared in a loop, a new x reference appears for each element in the array and is then erased for each iteration.

References Example

- The following example illustrates why we might use references when looping through an array:

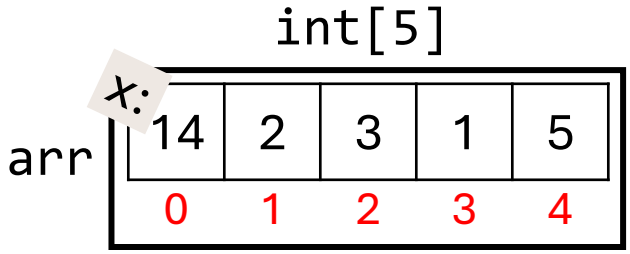
```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```



References Example

- The following example illustrates why we might use references when looping through an array:

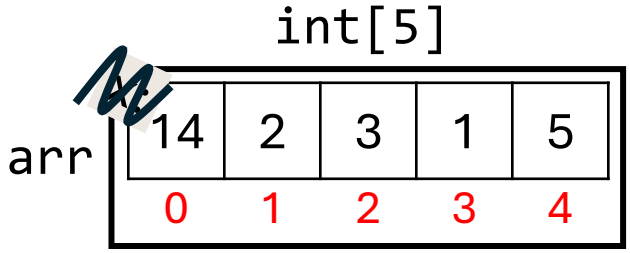
```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```



References Example

- The following example illustrates why we might use references when looping through an array:

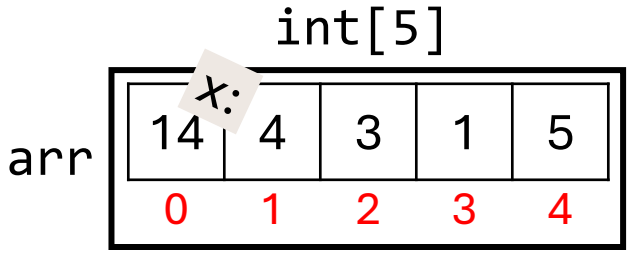
```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```



References Example

- The following example illustrates why we might use references when looping through an array:

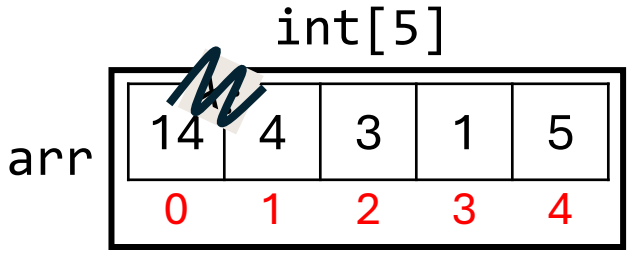
```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```



References Example

- The following example illustrates why we might use references when looping through an array:

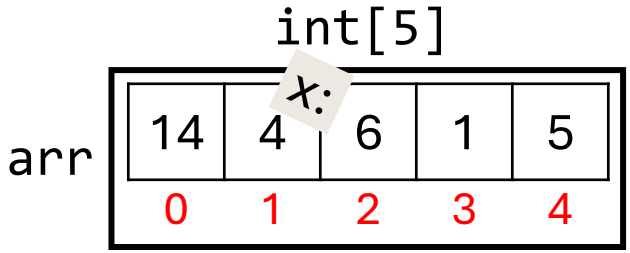
```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```



References Example

- The following example illustrates why we might use references when looping through an array:

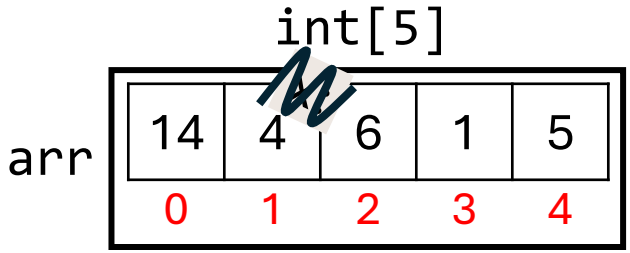
```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```



References Example

- The following example illustrates why we might use references when looping through an array:

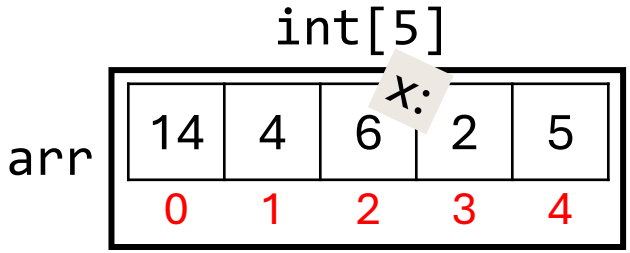
```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```



References Example

- The following example illustrates why we might use references when looping through an array:

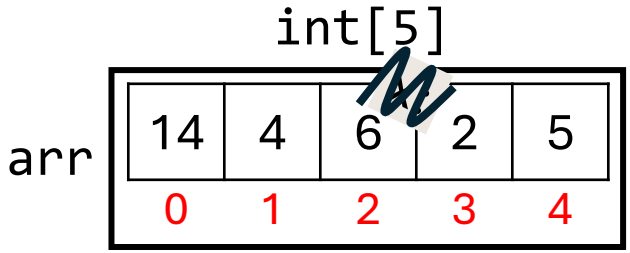
```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```



References Example

- The following example illustrates why we might use references when looping through an array:

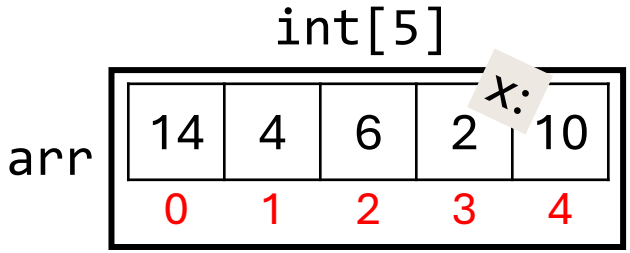
```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```



References Example

- The following example illustrates why we might use references when looping through an array:

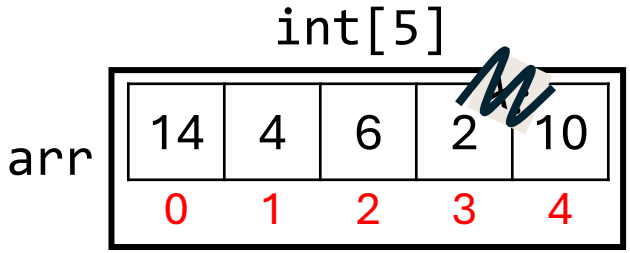
```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```



References Example

- The following example illustrates why we might use references when looping through an array:

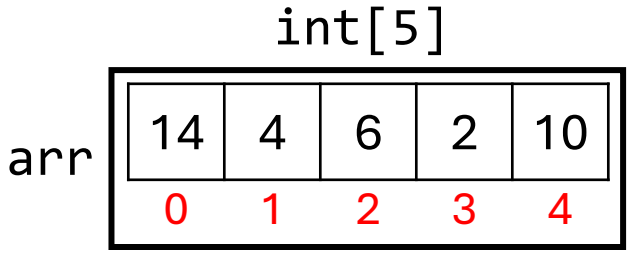
```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```



References Example

- The following example illustrates why we might use references when looping through an array:

```
int arr[]{7, 2, 3, 1, 5};
for (int x : arr) {
    x *= 2; // no effect (x is a copy)
}
for (int& x : arr) {
    x *= 2; // x modifies arr
}
```



Since x referenced each element in the array, when we doubled x's value we actually modified each of the values inside the array.

Pointer vs. Reference

	Pointer	Reference
Does it need to be initialized when declared?	No (but it should be, as it could point anywhere)	Yes, required to compile
Can it be `null` (point to nothing)?	Yes (`nullptr`)	No
Can it be redirected to point to a different variable later?	Yes (reassign it to a different memory address)	No, it references the variable it was initialized with†
How is the value accessed?	With the dereference operator (*)	It is used directly as a new label to an existing variable

†In the for-each loop, a new reference is declared each iteration

06: Pass by Value vs. Pass by Pointer vs. Pass by Reference

Anthony Estey
CSC 115: Fundamentals of Programming: II

Functions, Pointers, and References

- So far in this course, every time we've made a function call, we have made a copy of the value and passed that copy to the function
- What if we don't want a copy? What if we want the function to reach back and modify our original variable?
- We now have two tools that allow us to do this!

Pass by Value

- What we've seen before:

```
void increment(int n) {
  n++;
}

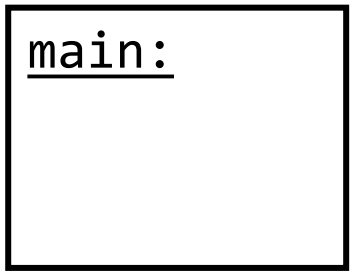
int main() {
  int x{5};
  increment(x);
  std::cout << x << std::endl;
}
```

Pass by Value

- What we've seen before:

```
void increment(int n) {
  n++;
}

int main() {
  int x{5};
  increment(x);
  std::cout << x << std::endl;
}
```

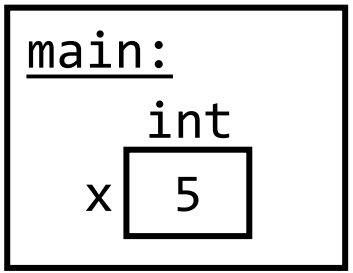


Pass by Value

- What we've seen before:

```
void increment(int n) {
  n++;
}

int main() {
  int x{5};
  increment(x);
  std::cout << x << std::endl;
}
```

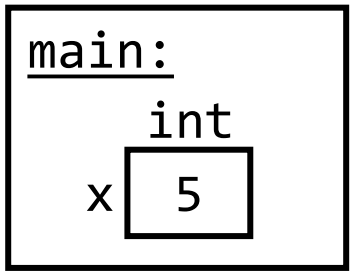


Pass by Value

- What we've seen before:

```
void increment(int n) {
  n++;
}

int main() {
  int x{5};
  increment(x);
  std::cout << x << std::endl;
}
```

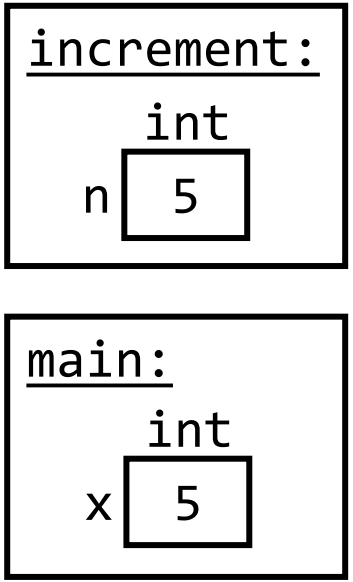


Pass by Value

- What we've seen before:

```
void increment(int n) {
  n++;
}

int main() {
  int x{5};
  increment(x);
  std::cout << x << std::endl;
}
```

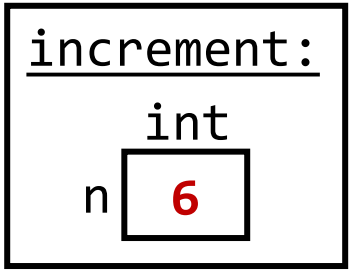


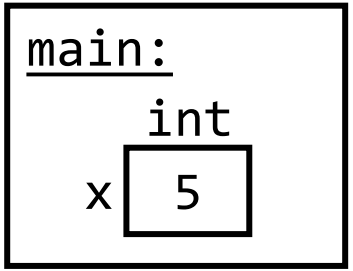
Pass by Value

- What we've seen before:

```
void increment(int n) {
  n++;
}

int main() {
  int x{5};
  increment(x);
  std::cout << x << std::endl;
}
```



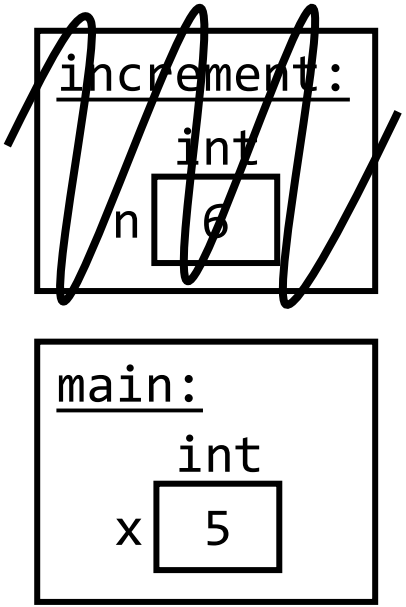


Pass by Value

- What we've seen before:

```
void increment(int n) {
  n++;
}

int main() {
  int x{5};
  increment(x);
  std::cout << x << std::endl;
}
```



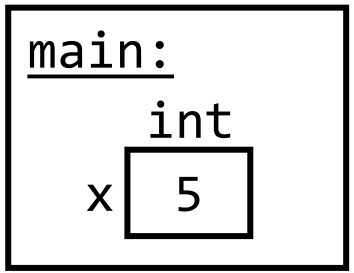
Pass by Value

- What we've seen before:

```
void increment(int n) {
  n++;
}

int main() {
  int x{5};
  increment(x);
  std::cout << x << std::endl;
}
```

The value incremented in the `increment` function was an independent copy of `x` from `main`



Output: 5

Pass by Reference

- A different way to do parameter passing:

```
void increment(int& n) {
  n++;
}

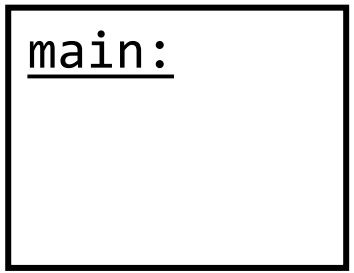
int main() {
  int x{5};
  increment(x);
  std::cout << x << std::endl;
}
```

Pass by Reference

- A different way to do parameter passing:

```
void increment(int& n) {
  n++;
}

int main() {
  int x{5};
  increment(x);
  std::cout << x << std::endl;
}
```

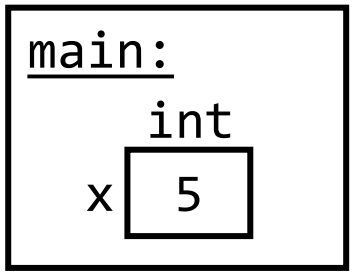


Pass by Reference

- A different way to do parameter passing:

```
void increment(int& n) {
  n++;
}

int main() {
  int x{5};
  increment(x);
  std::cout << x << std::endl;
}
```

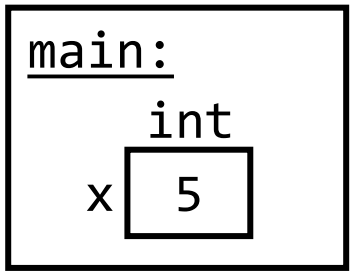


Pass by Reference

- A different way to do parameter passing:

```
void increment(int& n) {
  n++;
}

int main() {
  int x{5};
  increment(x);
  std::cout << x << std::endl;
}
```

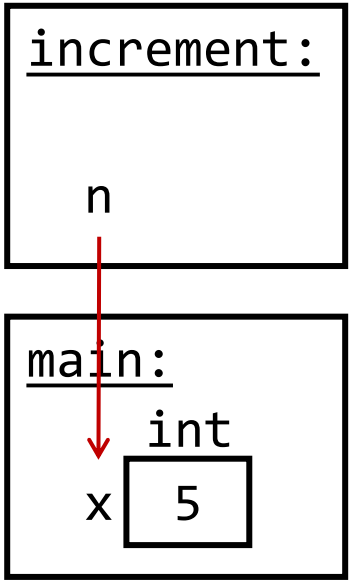


Pass by Reference

- A different way to do parameter passing:

```
void increment(int& n) {
  n++;
}

int main() {
  int x{5};
  increment(x);
  std::cout << x << std::endl;
}
```

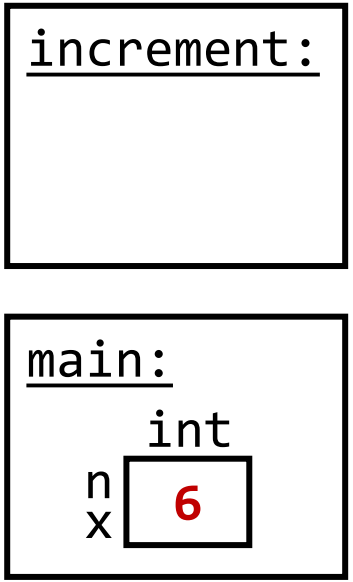


Pass by Reference

- A different way to do parameter passing:

```
void increment(int& n) {
  n++;
}

int main() {
  int x{5};
  increment(x);
  std::cout << x << std::endl;
}
```

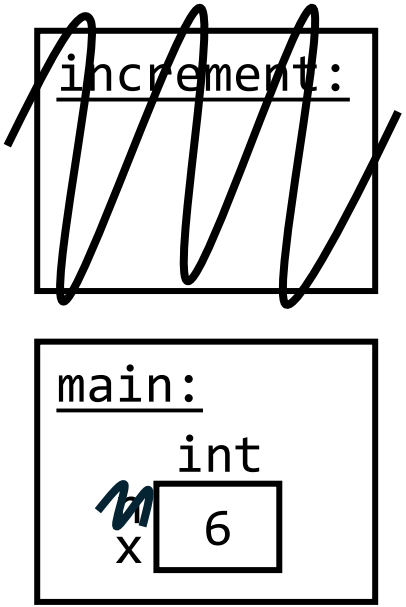


Pass by Reference

- A different way to do parameter passing:

```
void increment(int& n) {
  n++;
}

int main() {
  int x{5};
  increment(x);
  std::cout << x << std::endl;
}
```



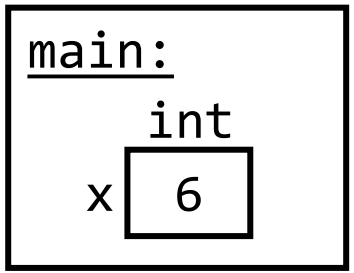
Pass by Reference

- A different way to do parameter passing:

```
void increment(int& n) {
  n++;
}

int main() {
  int x{5};
  increment(x);
  std::cout << x << std::endl;
}
```

Since n is a reference to the x in main (just a new label on it), n++ results in the x value in main being incremented.



Output: 6

Pass by Pointer

- Another method with some similarities to references:

```
void increment(int* n) {
  (*n)++;
}

int main() {
  int x{5};
  increment(&x);
  std::cout << x << std::endl;
}
```

Pass by Pointer

- Another method with some similarities to references:

```
void increment(int* n) {
  (*n)++;
}

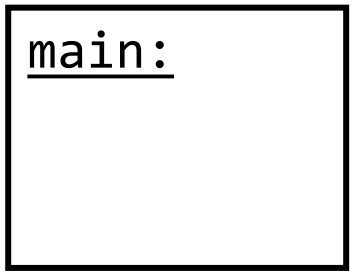
int main() {
  int x{5};
  increment(&x);
  std::cout << x << std::endl;
}
```

Pass by Pointer

- Another method with some similarities to references:

```
void increment(int* n) {
  (*n)++;
}

int main() {
  int x{5};
  increment(&x);
  std::cout << x << std::endl;
}
```

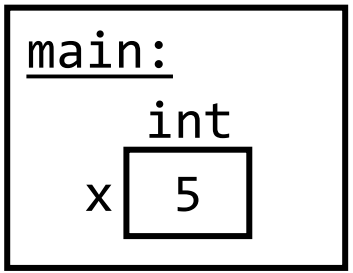


Pass by Pointer

- Another method with some similarities to references:

```
void increment(int* n) {
  (*n)++;
}

int main() {
  int x{5};
  increment(&x);
  std::cout << x << std::endl;
}
```

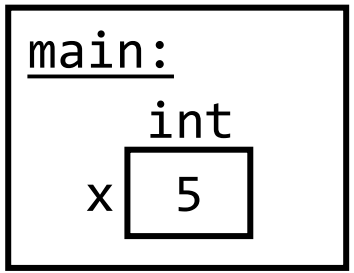


Pass by Pointer

- Another method with some similarities to references:

```
void increment(int* n) {
  (*n)++;
}

int main() {
  int x{5};
  increment(&x);
  std::cout << x << std::endl;
}
```

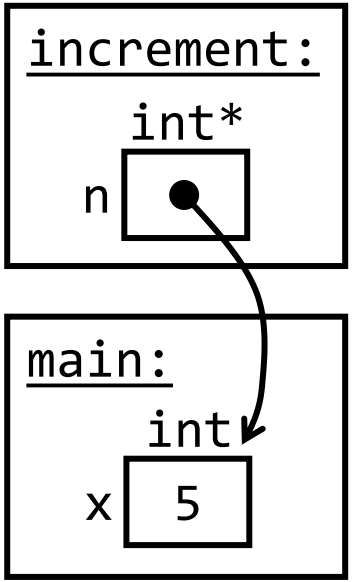


Pass by Pointer

- Another method with some similarities to references:

```
void increment(int* n) {
  (*n)++;
}

int main() {
  int x{5};
  increment(&x);
  std::cout << x << std::endl;
}
```

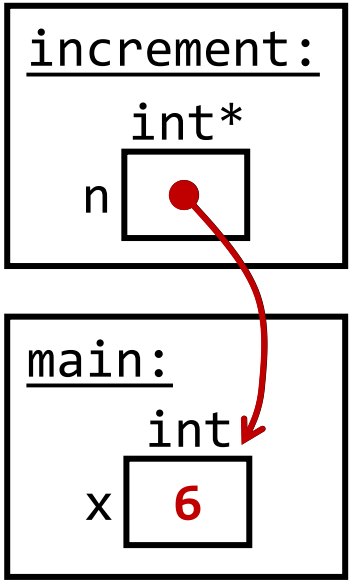


Pass by Pointer

- Another method with some similarities to references:

```
void increment(int* n) {
  (*n)++;
}

int main() {
  int x{5};
  increment(&x);
  std::cout << x << std::endl;
}
```

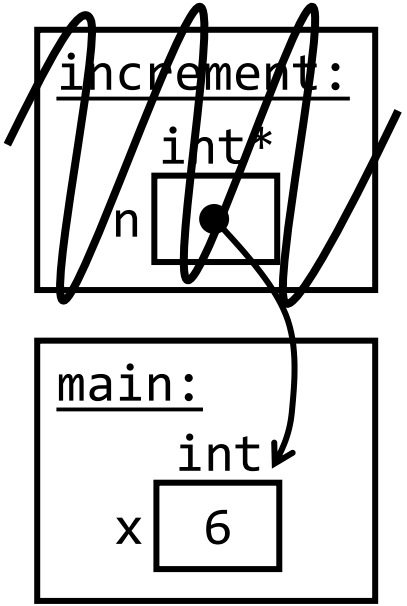


Pass by Pointer

- Another method with some similarities to references:

```
void increment(int* n) {
  (*n)++;
}

int main() {
  int x{5};
  increment(&x);
  std::cout << x << std::endl;
}
```



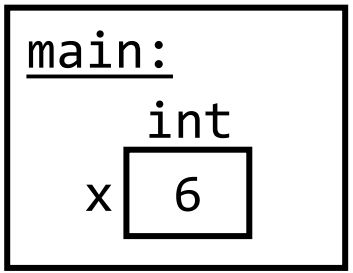
Pass by Pointer

- Another method with some similarities to references:

```
void increment(int* n) {
  (*n)++;
}

int main() {
  int x{5};
  increment(&x);
  std::cout << x << std::endl;
}
```

Here, n is a pointer, so it *is* allocated space. By incrementing the value at the address it points to, we change x



Output: 6

Why is this important?

- Passing an int to a function is a small, simple task, so it may be difficult to see why these three examples are important
- Why does any of this actually matter?
- Remember the house analogy when I introduced pointers?
  - Copying is fine for something small, but what if it isn't a single `int`?
  - What if it's an array with a billion elements, or a struct with dozens of data members? When we pass larger objects by value, every byte is copied, which can be resource intensive and time-consuming
  - Often, passing a pointer to a large object allows us to complete the same task at a fraction of the cost to memory and speed

Why is this important?

- Sometimes C++ already does this for us, even if we are unaware of what is actually happening:

```
void array_fn(int arr[]) {
  // do something with arr
}

void some_fn() {
  int arr[]{7,2,3,1,6};
  array_fn(arr);
}
```

Why is this important?

- Sometimes C++ already does this for us, even if we are unaware of what is actually happening:

```
void array_fn(int arr[]) {
  // do something with arr
}

void some_fn() {
  int arr[]{7,2,3,1,6};
  array_fn(arr);
}
```

some_fn:

Memory:

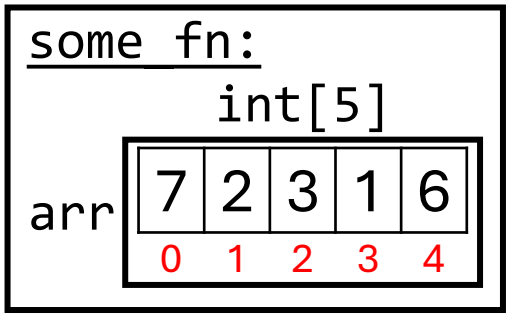
	Address	Value
	0x9000	
	0x9004	
	0x9008	
	0x900c	
	0x9010	
	0x9014	
	0x9018	

Why is this important?

- Sometimes C++ already does this for us, even if we are unaware of what is actually happening:

```
void array_fn(int arr[]) {
  // do something with arr
}

void some_fn() {
  int arr[]{7,2,3,1,6};
  array_fn(arr);
}
```



Memory:

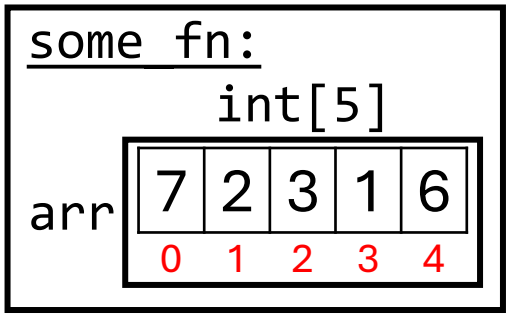
	Address	Value
	0x9000	
	0x9004	
arr	0x9008	7
	0x900c	2
	0x9010	3
	0x9014	1
	0x9018	6

Why is this important?

- Sometimes C++ already does this for us, even if we are unaware of what is actually happening:

```
void array_fn(int arr[]) {
  // do something with arr
}

void some_fn() {
  int arr[]{7,2,3,1,6};
  array_fn(arr);
}
```



Memory:

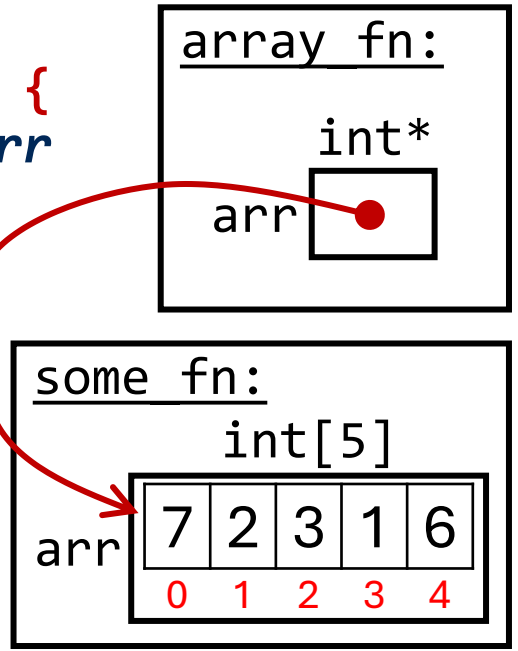
	Address	Value
	0x9000	
	0x9004	
arr	0x9008	7
	0x900c	2
	0x9010	3
	0x9014	1
	0x9018	6

Why is this important?

- Sometimes C++ already does this for us, even if we are unaware of what is actually happening:

```
void array_fn(int arr[]) {
  // do something with arr
}

void some_fn() {
  int arr[]{7,2,3,1,6};
  array_fn(arr);
}
```



Memory:

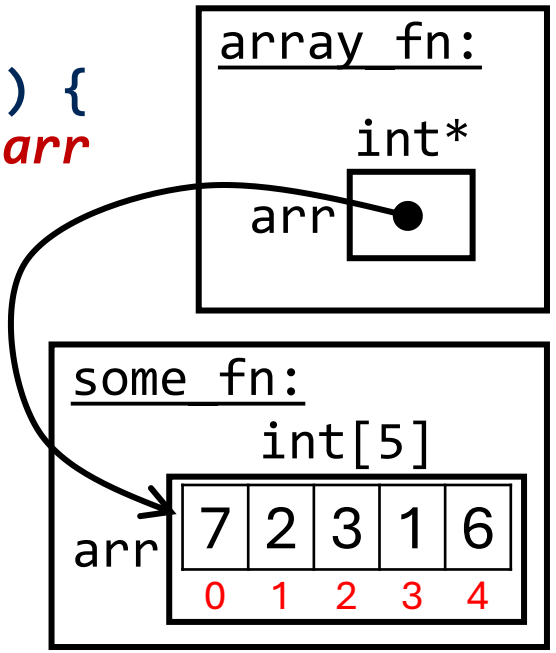
	Address	Value
arr	0x9000	0x9008
	0x9004	
arr	0x9008	7
	0x900c	2
	0x9010	3
	0x9014	1
	0x9018	6

Why is this important?

- Sometimes C++ already does this for us, even if we are unaware of what is actually happening:

```
void array_fn(int arr[]) {
  // do something with arr
}
```

Whatever we do to **arr** in **array_fn** changes the original array, since it is pointer to that data, **not a copy**.



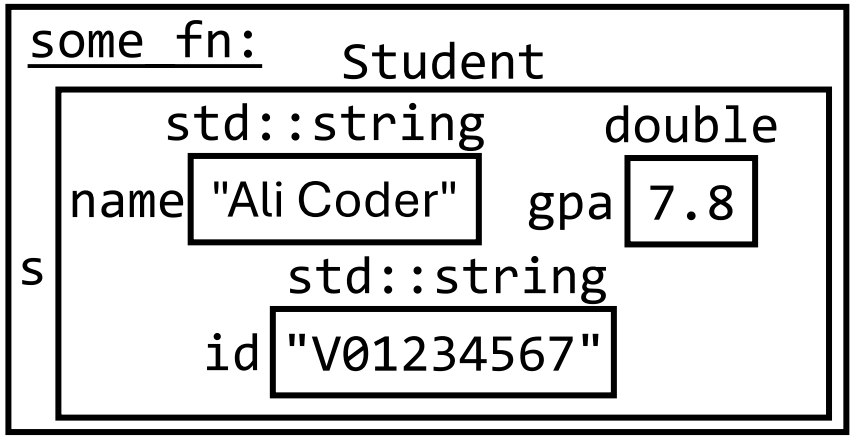
Memory:

	Address	Value
arr	0x9000	0x9008
	0x9004	
arr	0x9008	7
	0x900c	2
	0x9010	3
	0x9014	1
	0x9018	6

Not everything is done ‘for’ us

- We have multiple ways to pass structs and other objects.
- Here is an example working with our Student struct:

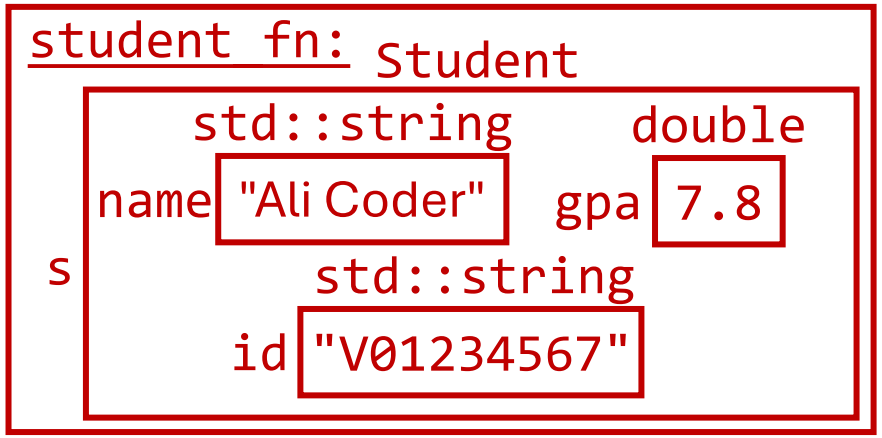
```
void student_fn(Student s); // copies all data
void student_fn(Student& s); // no copy, just a reference
void student_fn(Student* s); // points to original, no copy
```

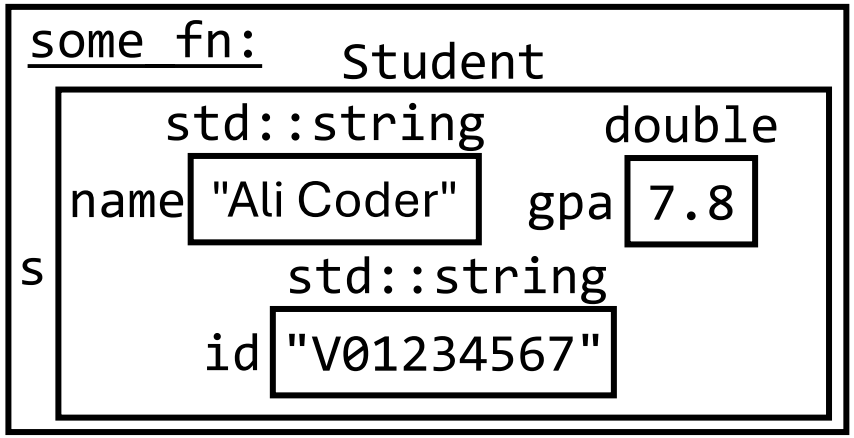


Not everything is done ‘for’ us

- We have multiple ways to pass structs and other objects.
- Here is an example working with our Student struct:

```
void student_fn(Student s); // copies all data
void student_fn(Student& s); // no copy, just a reference
void student_fn(Student* s); // points to original, no copy
```



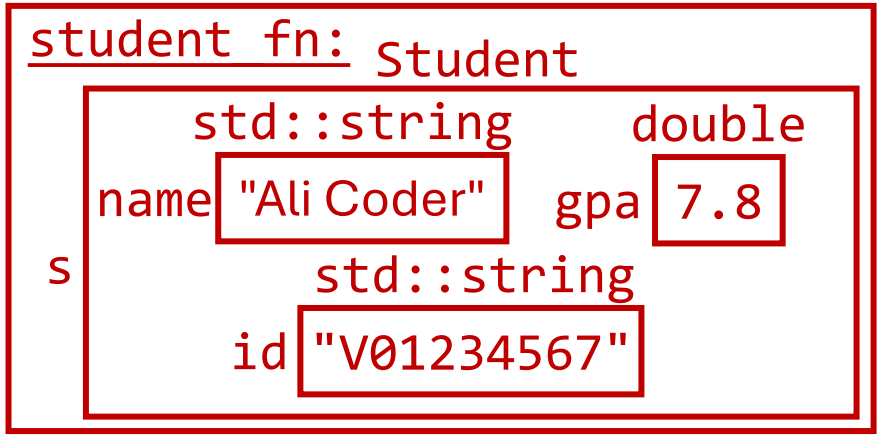


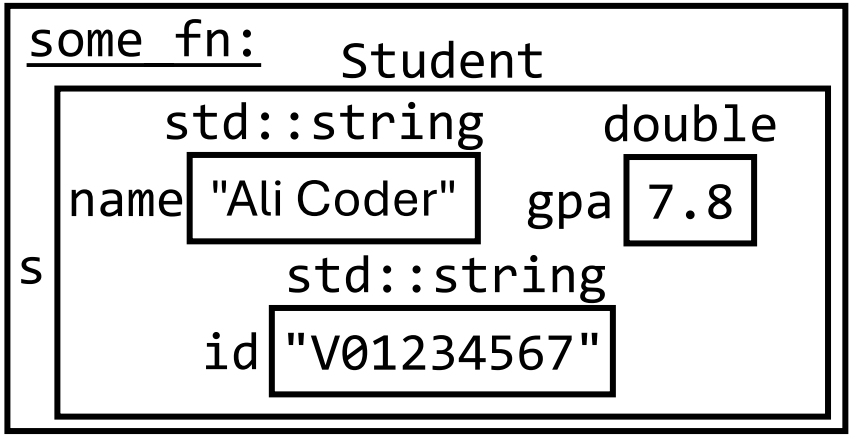
Not everything is done ‘for’ us

- We have multiple ways to pass structs and other objects.
- Here is an example working with our Student struct:

```
void student_fn(Student s); // copies all data
```

The **s** in `student_fn` is a copy, so any changes to it in the function have no effect on the original.

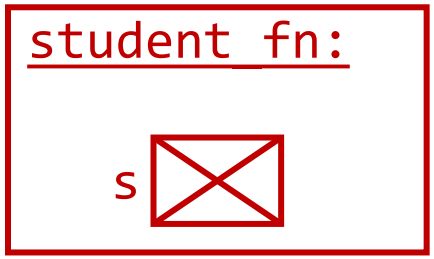


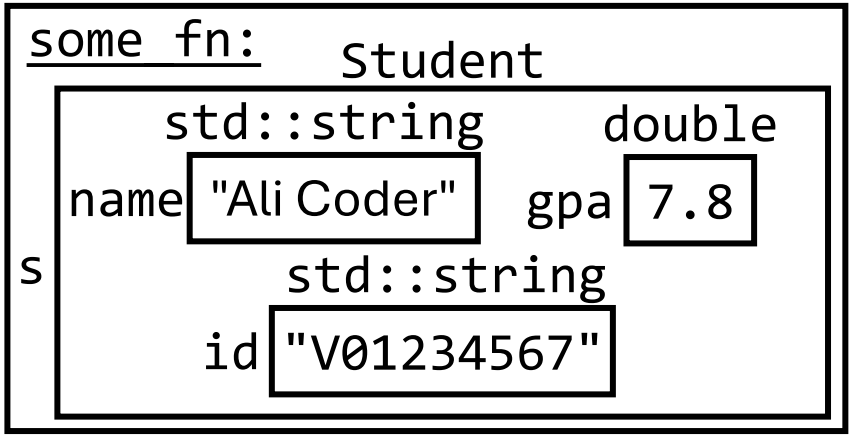


Not everything is done ‘for’ us

- We have multiple ways to pass structs and other objects.
- Here is an example working with our Student struct:

```
void student_fn(Student s); // copies all data
void student_fn(Student& s); // no copy, just a reference
void student_fn(Student* s); // points to original, no copy
```

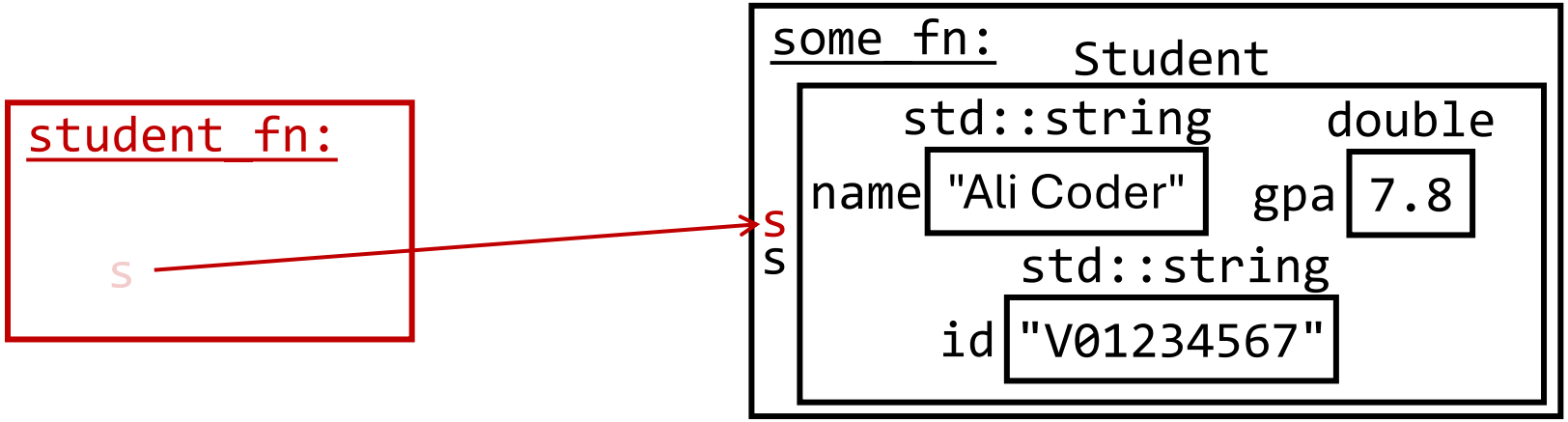




Not everything is done ‘for’ us

- We have multiple ways to pass structs and other objects.
- Here is an example working with our Student struct:

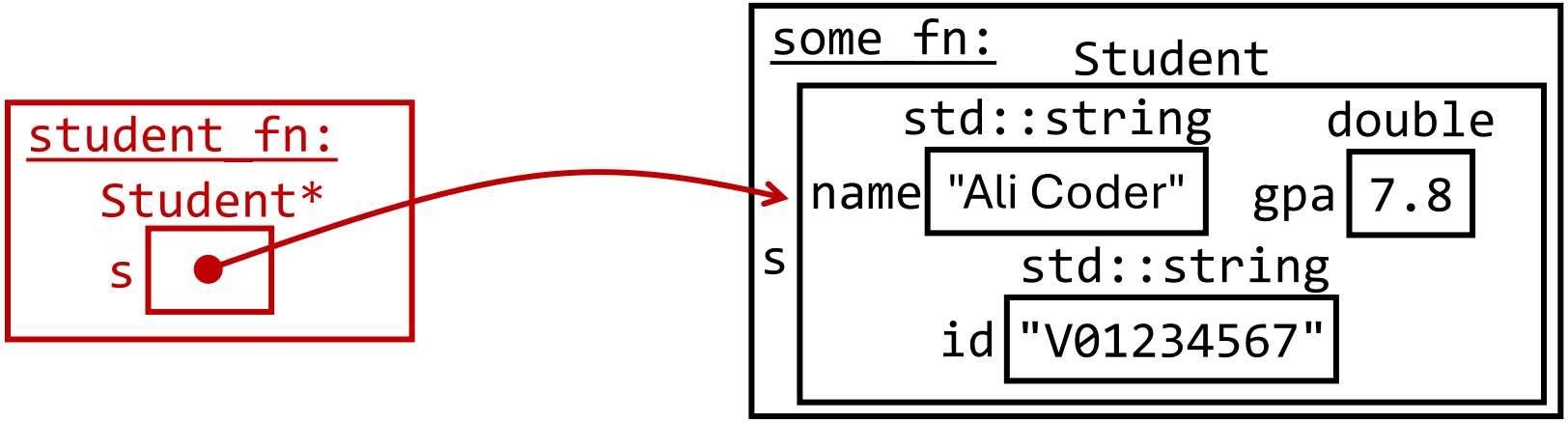
```
void student_fn(Student s); // copies all data
void student_fn(Student& s); // no copy, just a reference
void student_fn(Student* s); // points to original, no copy
```



Not everything is done ‘for’ us

- We have multiple ways to pass structs and other objects.
- Here is an example working with our Student struct:

```
void student_fn(Student s); // copies all data
void student_fn(Student& s); // no copy, just a reference
void student_fn(Student* s); // points to original, no copy
```

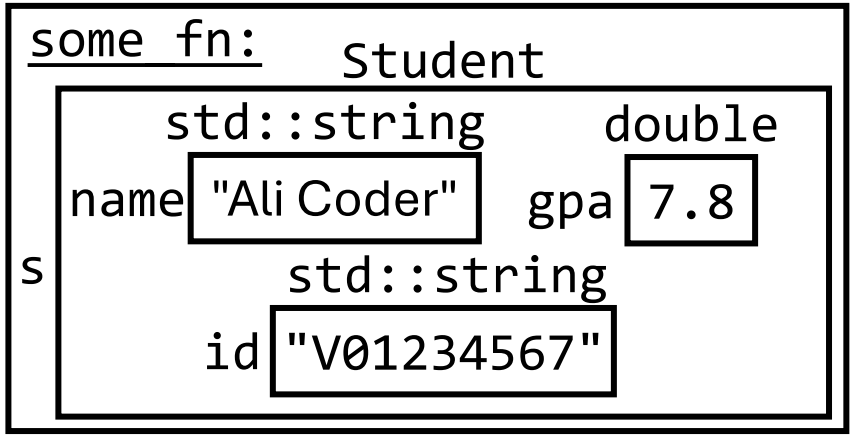


Not everything is done ‘for’ us

- We have multiple ways to pass structs and other objects.
- Here is an example working with our Student struct:

```
void student_fn(Student s); // copies all data
void student_fn(Student& s); // no copy, just a reference
void student_fn(Student* s); // points to original, no copy
```

No copy is made when passing by reference or by pointer. This means any changes made to s in student_fn will modify the original version. Be careful!

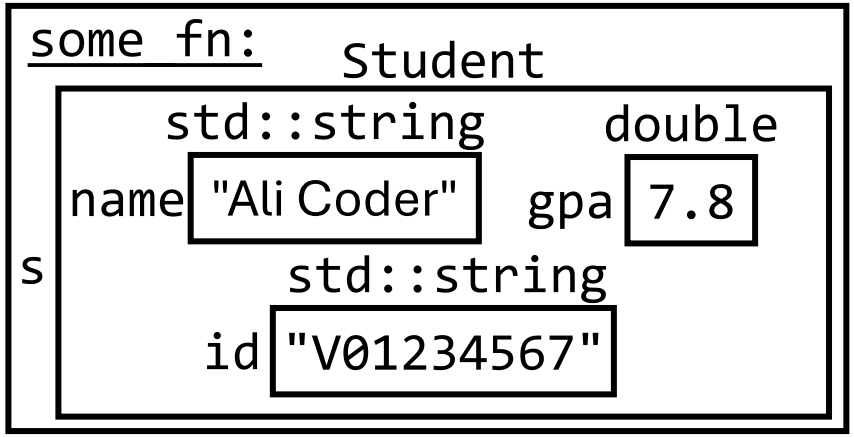


Not everything is done ‘for’ us

- We have multiple ways to pass structs and other objects.
- Here is an example working with our Student struct:

```
void student_fn(Student s); // copies all data
void student_fn(Student& s); // no copy, just a reference
void student_fn(Student* s); // points to original, no copy
```

The difference: a pointer variable is allocated space in memory to point at the original. There is no box for a reference variable, it’s another label for the original.



Value vs. Reference vs. Pointer

	Value	Reference	Pointer
Is a copy of the passed variable created?	Yes	No	No (the address of the variable is copied)
How is a variable, x, passed to a function, fn?	`fn(x);`	`fn(x);`	`fn(&x);`
What does the parameter look like in the function signature?	`type name`	`type& name`	`type* name`
Can the calling function's variable be modified?	No	Yes	Yes
How is the calling function's variable accessed?	Not applicable	`name`	`*name`
Can the parameter represent 'nothing'?	Not applicable	No	Yes (not pointing anywhere: `nullptr`)

07: const

Anthony Estey
CSC 115: Fundamentals of Programming: II

Motivation for **const**

- A reference solves the copying problem, but it hands the function full permission to modify our data too
  - Even if all we wanted was to avoid copying over a bunch of data
- What if we could get the improvement in memory and runtime efficiency without giving up that control?
- Once initialized, the value of a **const** variable can never be changed, and any attempt to reassign it is a compile error

Syntax for const variables

- General syntax:

```
const type variable_name{permanent_value};
```

- Specific examples:

```
const double pi{3.14159};
```

```
pi = 3.14;
```



Compile error!

Safer pass by reference

- Given our Student struct, this allows us to pass a student by reference (more efficient) without worrying that the original will be modified in any way.
  - For example, we would expect a `print_student` function to output information about a student, not modify it.

```
void print_student(Student& s) {
    s.gpa = 0.0;
}
```

This is perfectly legal, but not what we want (and maybe a hard bug to find).

```
void print_student(const Student& s) {
    s.gpa = 0.0;
}
```

Causes a compile error – identified immediately!

Correct use of **const**

- **General rule:** Mark anything that shouldn't change as **const**
  - This is not just as a safety net, but as documentation.
  - A **`const&`** parameter tells every future reader, without a comment, *"this function only reads, it never writes."*
- To summarize: passing with **`const&`** gives us the efficiency of a reference and the safety of passing by value. This is the default choice for most parameters in real C++ code.
- We've now seen how stack memory works, but at the start of this unit I also talked about another region, the heap:
  - Next, we explore how the heap is used for memory that needs to outlive a function call.

08: Dynamic Memory

Anthony Estey
CSC 115: Fundamentals of Programming: II

Motivation

- Every variable so far has lived on the stack:
  - Parameters and local variables (those declared within a function) are allocated and removed for us automatically in the stack frame
- There are times when we need to use certain data long after the function that created it has completed
- There are also times when we don't know the size of something until the program is running:
  - Reading data from a file to put into an array
  - Creating a container based on a user input value
- These are situations when we use **heap** memory

Dynamic Memory Syntax

- General syntax:

```
type* pointer_name{new type{value}};
delete pointer_name;
```

The **new** keyword allocates memory based on the size of the type being created. It returns a pointer to that address on the heap.

The **delete** keyword frees up the heap memory at the location pointed to by the pointer variable, so it can be reused. This is similar to what happens automatically when a stack frame is removed when a function returns. The difference: heap memory is only freed when we explicitly use delete.

Dynamic Memory Syntax

- Specific example:

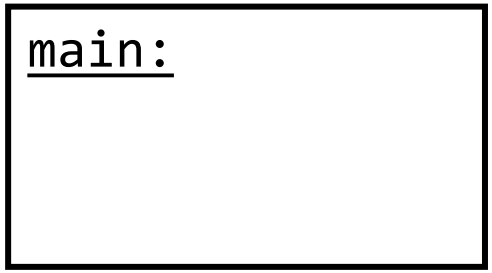
```
int* create_int() {
  int* ptr{new int{5}};
  std::cout << *ptr << std::endl;
  return ptr;
}

int main() {
  int* main_ptr{create_int()};
  std::cout << *main_ptr << std::endl;
  delete main_ptr;
  return 0;
}
```

Dynamic Memory Syntax

- Specific example:

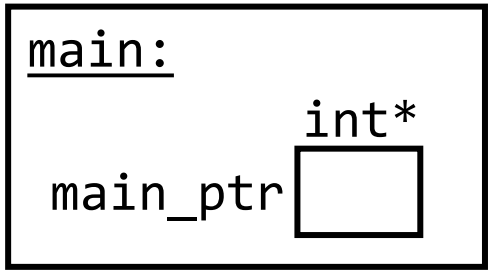
```
int* create_int() {
  int* ptr{new int{5}};
  std::cout << *ptr << std::endl;
  return ptr;
}
int main() {
  int* main_ptr{create_int()};
  std::cout << *main_ptr << std::endl;
  delete main_ptr;
  return 0;
}
```



Dynamic Memory Syntax

- Specific example:

```
int* create_int() {
  int* ptr{new int{5}};
  std::cout << *ptr << std::endl;
  return ptr;
}
int main() {
  int* main_ptr{create_int()};
  std::cout << *main_ptr << std::endl;
  delete main_ptr;
  return 0;
}
```

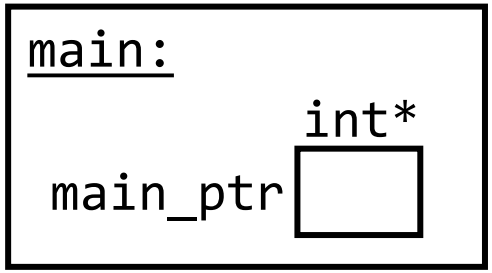


Dynamic Memory Syntax

- Specific example:

```
int* create_int() {
  int* ptr{new int{5}};
  std::cout << *ptr << std::endl;
  return ptr;
}

int main() {
  int* main_ptr{create_int()};
  std::cout << *main_ptr << std::endl;
  delete main_ptr;
  return 0;
}
```

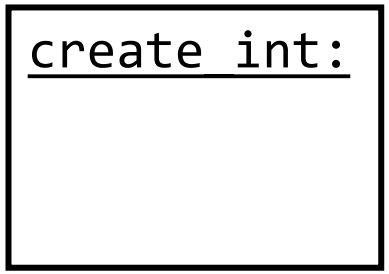


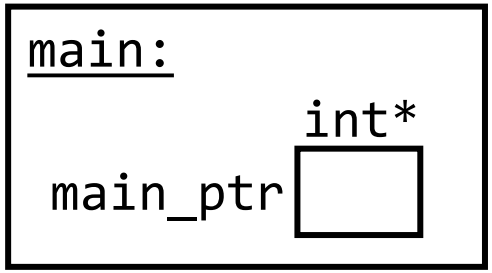
Dynamic Memory Syntax

- Specific example:

```
int* create_int() {
  int* ptr{new int{5}};
  std::cout << *ptr << std::endl;
  return ptr;
}

int main() {
  int* main_ptr{create_int()};
  std::cout << *main_ptr << std::endl;
  delete main_ptr;
  return 0;
}
```



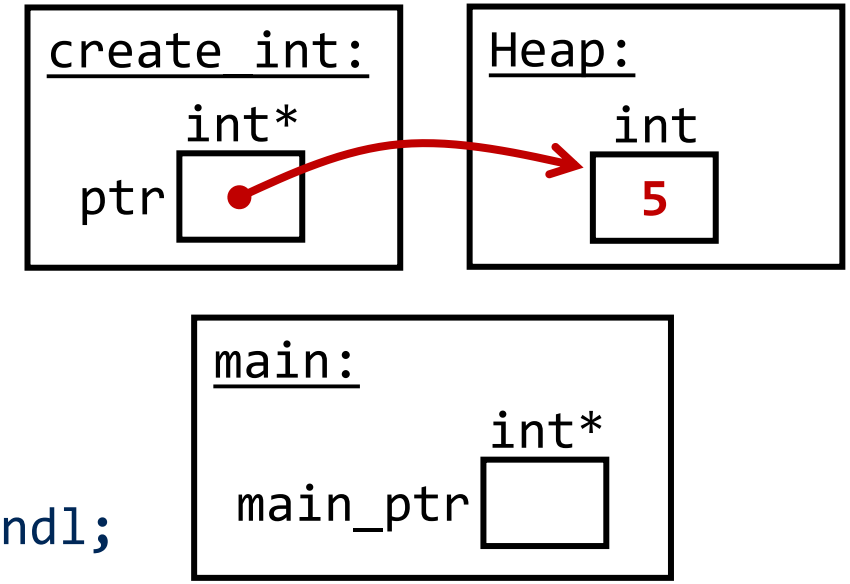


Dynamic Memory Syntax

- Specific example:

```
int* create_int() {
  int* ptr{new int{5}};
  std::cout << *ptr << std::endl;
  return ptr;
}

int main() {
  int* main_ptr{create_int()};
  std::cout << *main_ptr << std::endl;
  delete main_ptr;
  return 0;
}
```

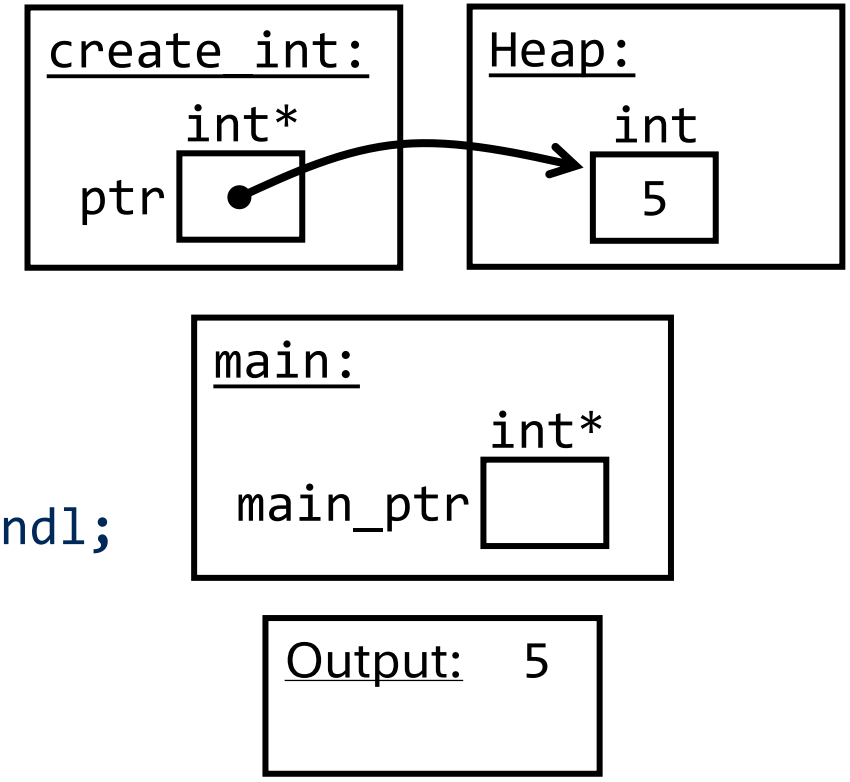


Dynamic Memory Syntax

- Specific example:

```
int* create_int() {
  int* ptr{new int{5}};
  std::cout << *ptr << std::endl;
  return ptr;
}

int main() {
  int* main_ptr{create_int()};
  std::cout << *main_ptr << std::endl;
  delete main_ptr;
  return 0;
}
```

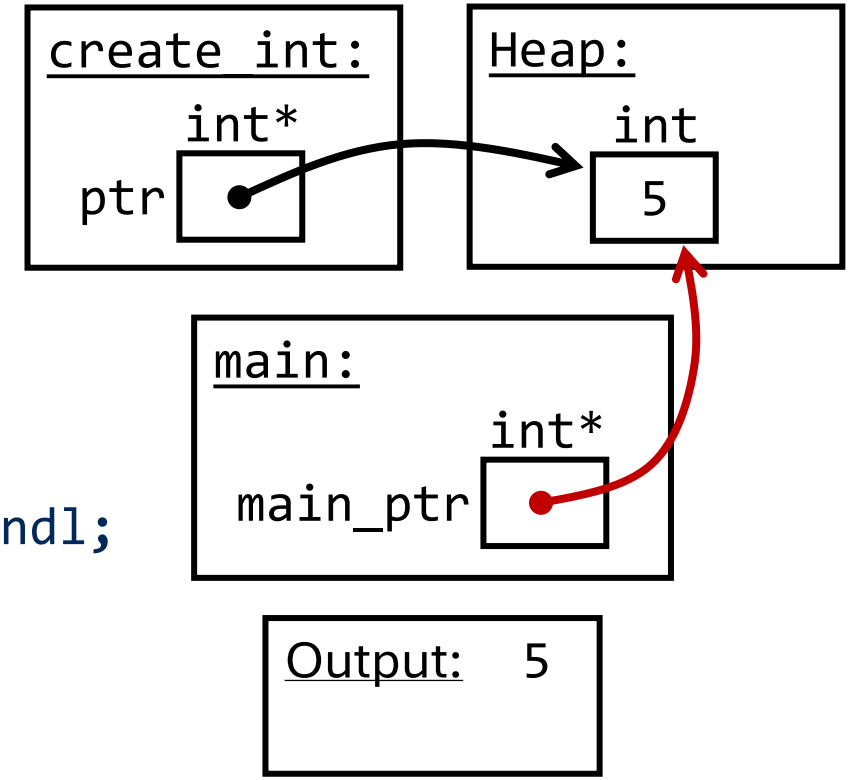


Dynamic Memory Syntax

- Specific example:

```
int* create_int() {
  int* ptr{new int{5}};
  std::cout << *ptr << std::endl;
  return ptr;
}
int main() {
  int* main_ptr{create_int()};
  std::cout << *main_ptr << std::endl;
  delete main_ptr;
  return 0;
}
```

What is returned?
The address of the 5

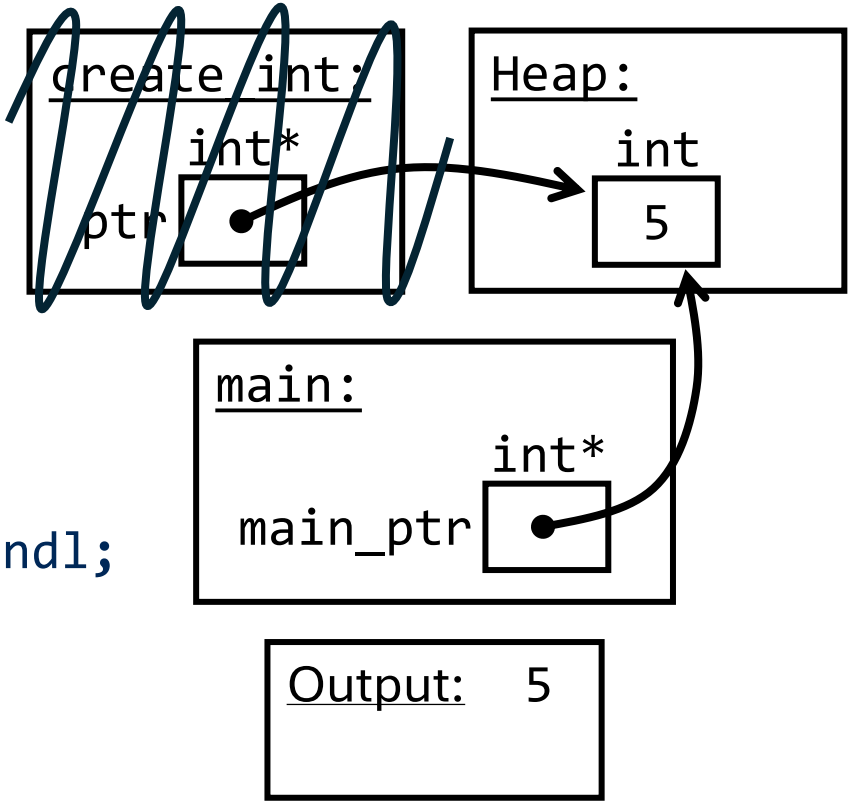


Dynamic Memory Syntax

- Specific example:

```
int* create_int() {
  int* ptr{new int{5}};
  std::cout << *ptr << std::endl;
  return ptr;
}

int main() {
  int* main_ptr{create_int()};
  std::cout << *main_ptr << std::endl;
  delete main_ptr;
  return 0;
}
```



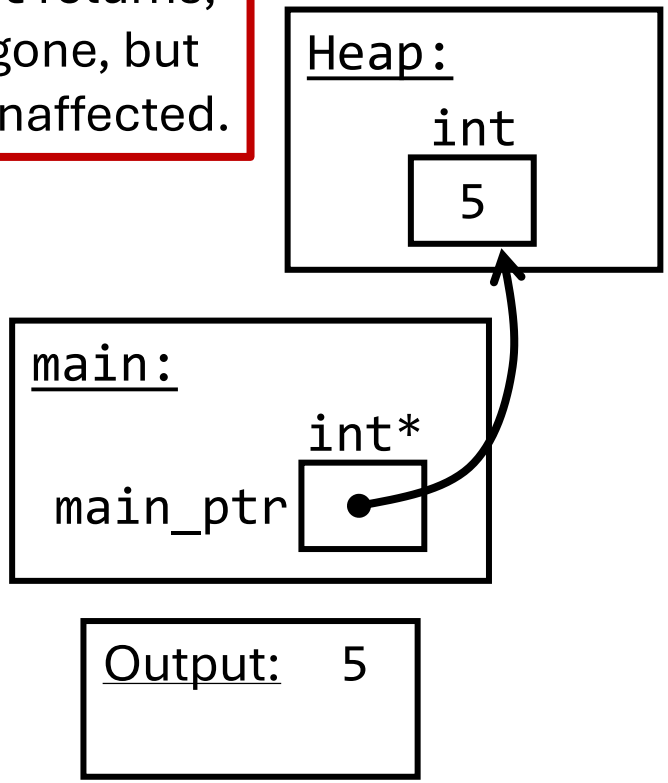
Dynamic Memory Syntax

- Specific example:

```
int* create_int() {
  int* ptr{new int{5}};
  std::cout << *ptr << std::endl;
  return ptr;
}

int main() {
  int* main_ptr{create_int()};
  std::cout << *main_ptr << std::endl;
  delete main_ptr;
  return 0;
}
```

When `create_int` returns, its stack frame is gone, but heap memory is unaffected.

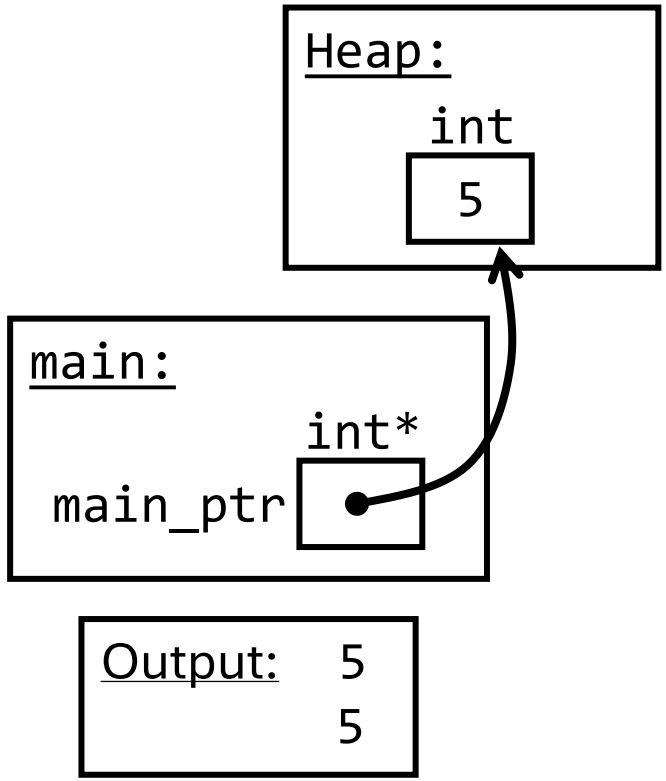


Dynamic Memory Syntax

- Specific example:

```
int* create_int() {
  int* ptr{new int{5}};
  std::cout << *ptr << std::endl;
  return ptr;
}

int main() {
  int* main_ptr{create_int()};
  std::cout << *main_ptr << std::endl;
  delete main_ptr;
  return 0;
}
```

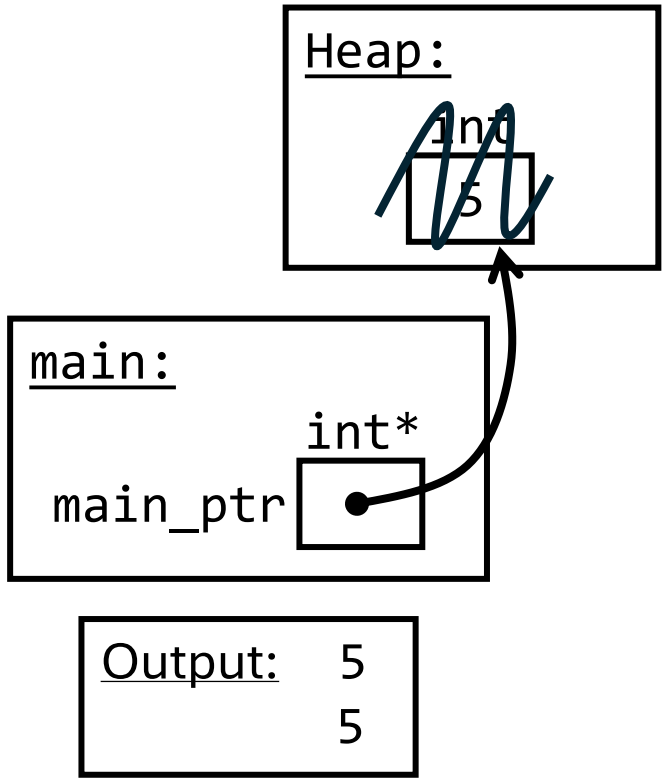


Dynamic Memory Syntax

- Specific example:

```
int* create_int() {
  int* ptr{new int{5}};
  std::cout << *ptr << std::endl;
  return ptr;
}

int main() {
  int* main_ptr{create_int()};
  std::cout << *main_ptr << std::endl;
  delete main_ptr;
  return 0;
}
```



Key Takeaway: Heap memory is only freed with delete

Dynamic Allocation with Arrays

- In many programs, we read from a data repository during the execution of the program
  - The repository could be a file, a webpage, or something else
- Many programs are also interactive, and their execution is driven by user input
- Next, we will see an example of this to demonstrate the usefulness of dynamic allocation in a scenario like this (while skipping the user input and file reading)

Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```

Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```

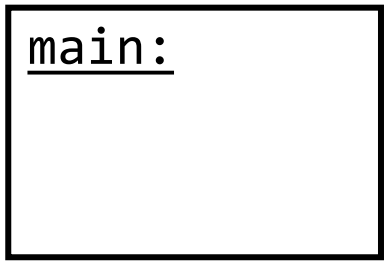
This example focuses on dynamic allocation is, so I removed the code where the user enters the number of scores to read, and also the code that reads the scores from the array <both formatted like this>

Notice the [ ]: when space is allocated for an array on the heap with `new[ ]`, it is freed with `delete[ ]`.

Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

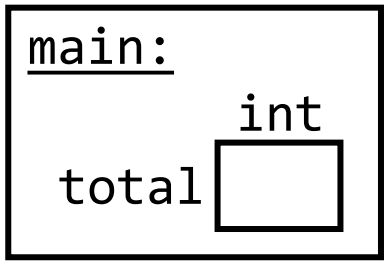
int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```



Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

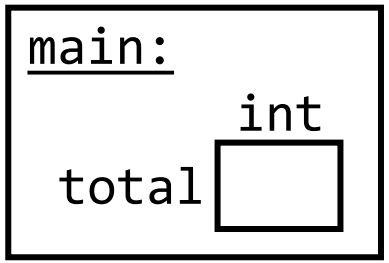
int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```



Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

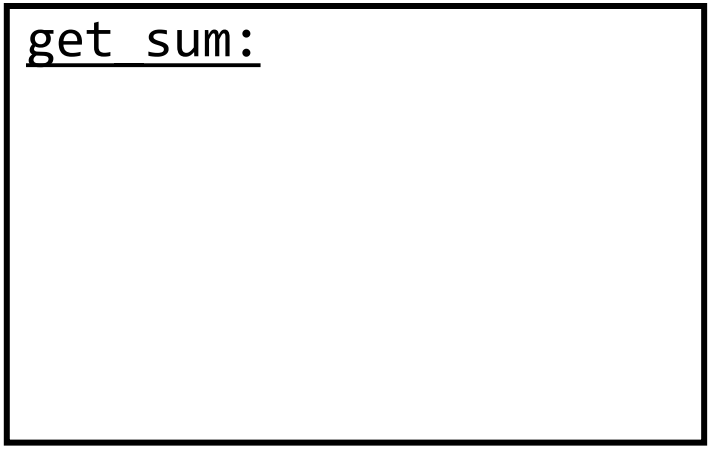
int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```

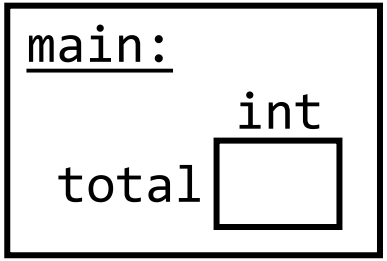


Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```

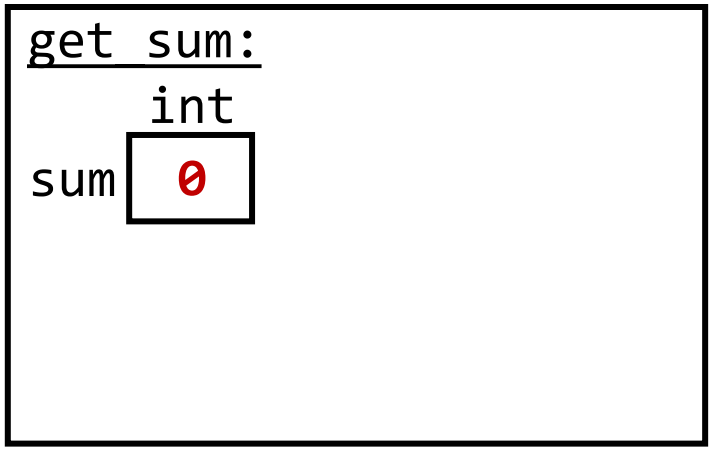


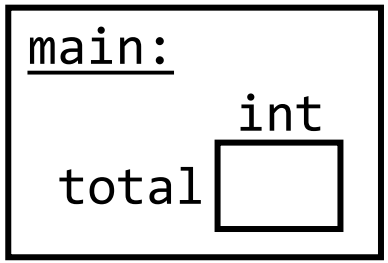


Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```

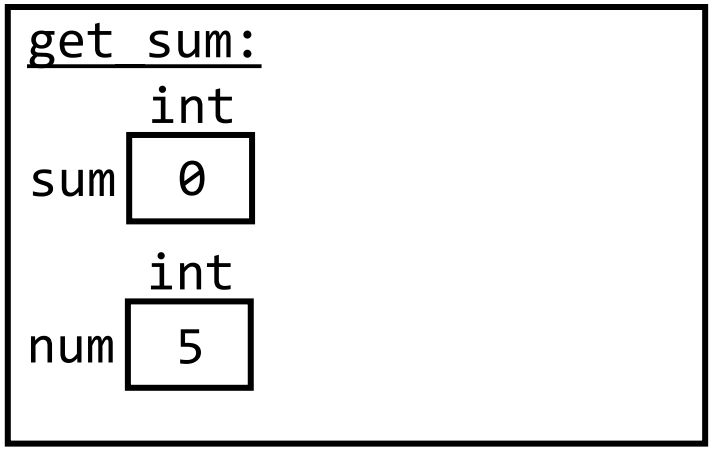


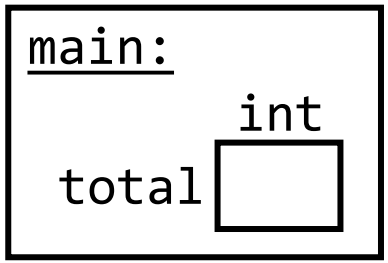


Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```

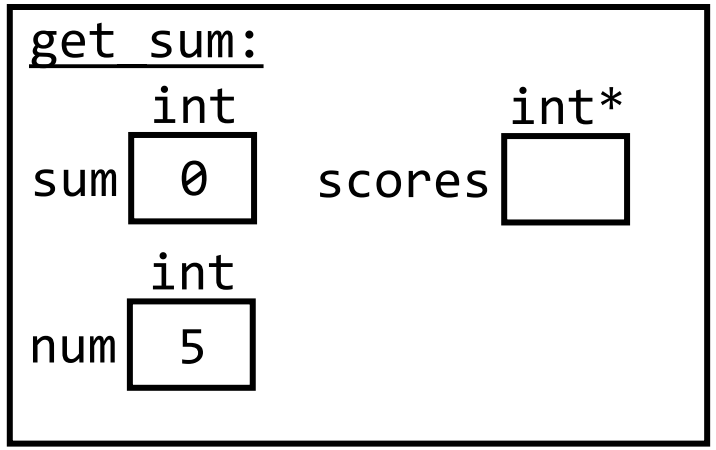


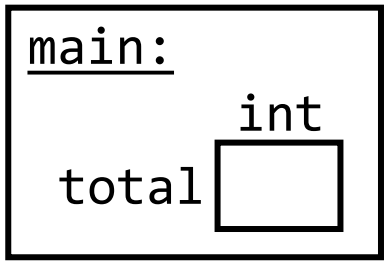


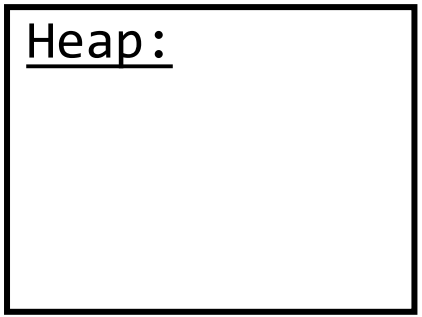
Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```



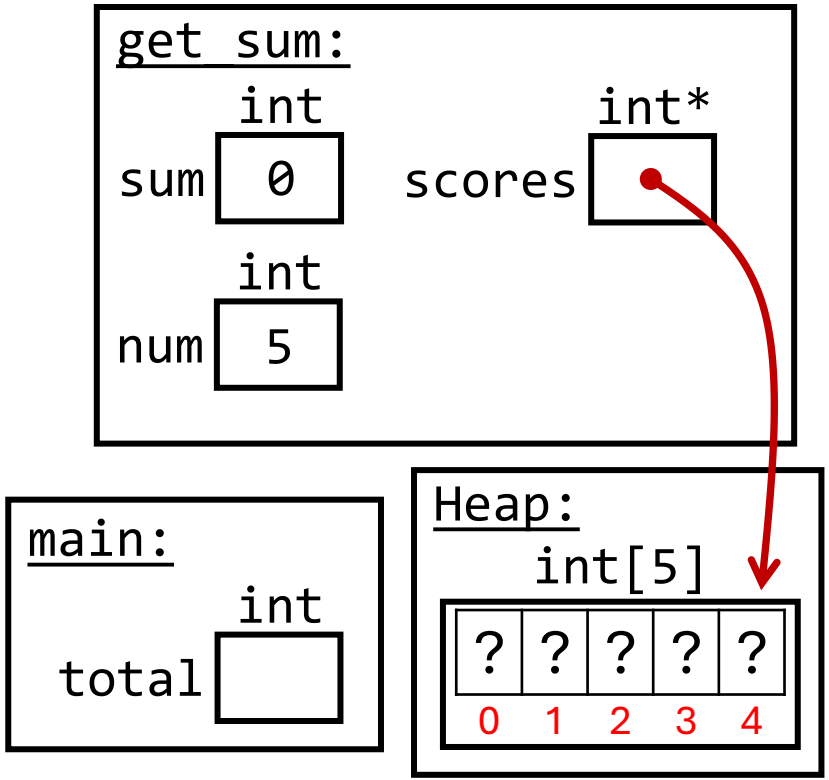




Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

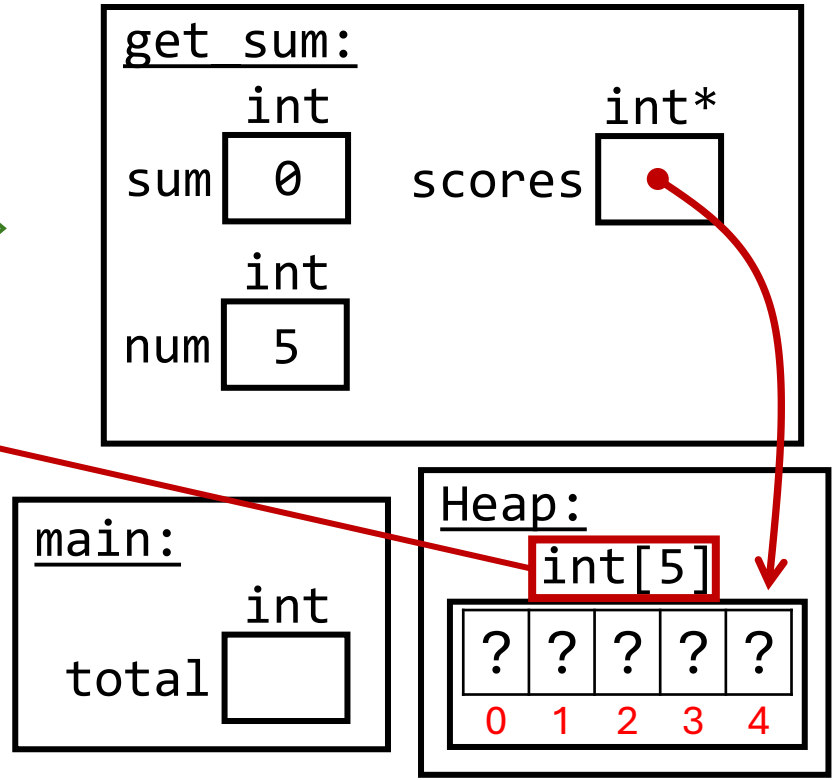
int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```



Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
```

The size of the array, 5, isn't tracked by C++. Remember: `new int[num]` returns an `int*`, and nothing about the pointer or the heap memory itself remembers how many elements are allocated for the array. Visually, I write the 5 here for clarity, as we can see how many integer elements are allocated together for the scores array.

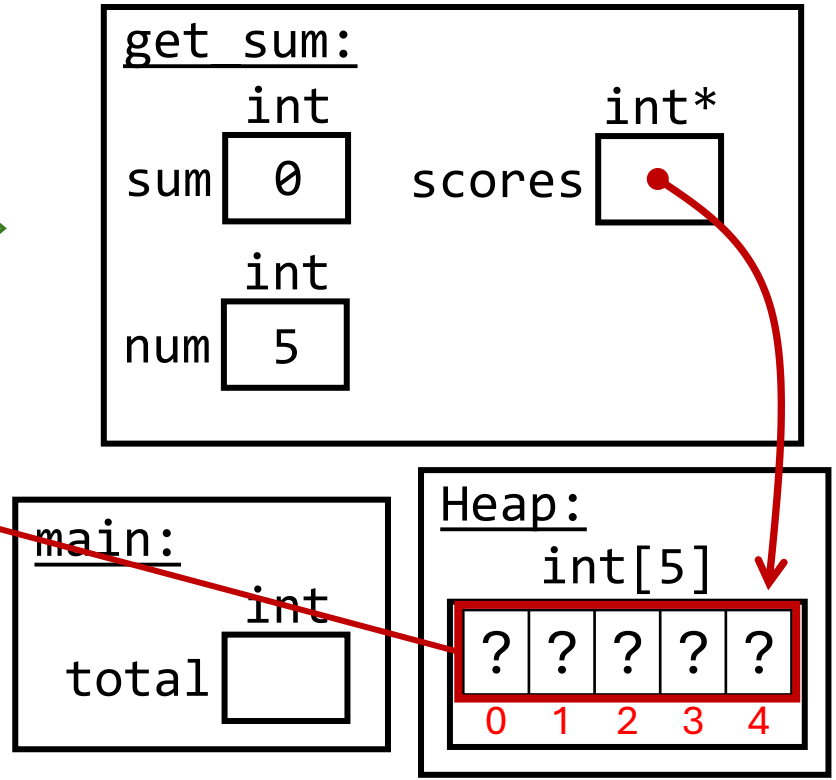


Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
                  [i]
```

With new `int[num]`, we allocate space for 5 `int`s on the heap. The **?**'s represent the fact that the array is full of **garbage values** (whatever was left there in the past). Like memory on the stack, we need to make sure every element gets initialized before we use it.

```
}
```

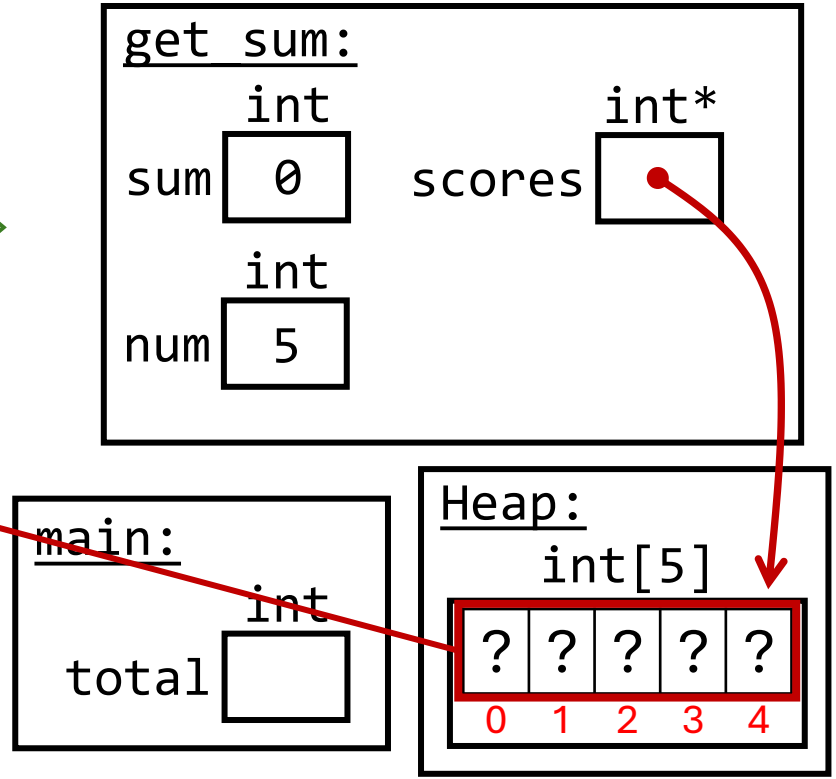


Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
                  [i]
```

In fact, this matters even more now! Since the array's size isn't tracked, nothing checks whether we actually filled in all 5 slots. For example, if we ended up reading less than 5 numbers from the file, some of these **?**'s silently stay uninitialized, and the program would have no way to know.

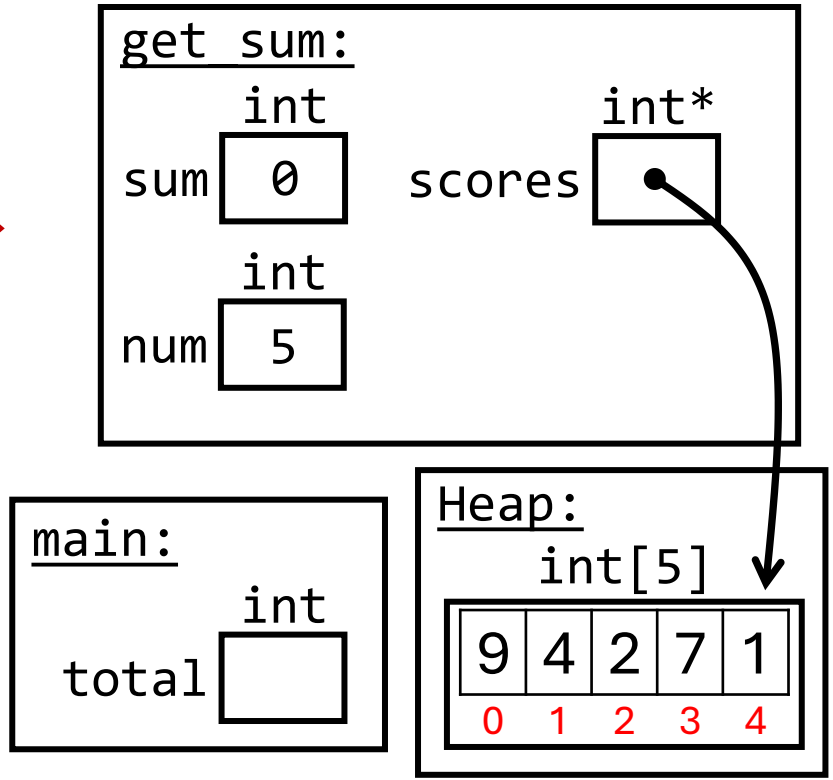
```
}
```



Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

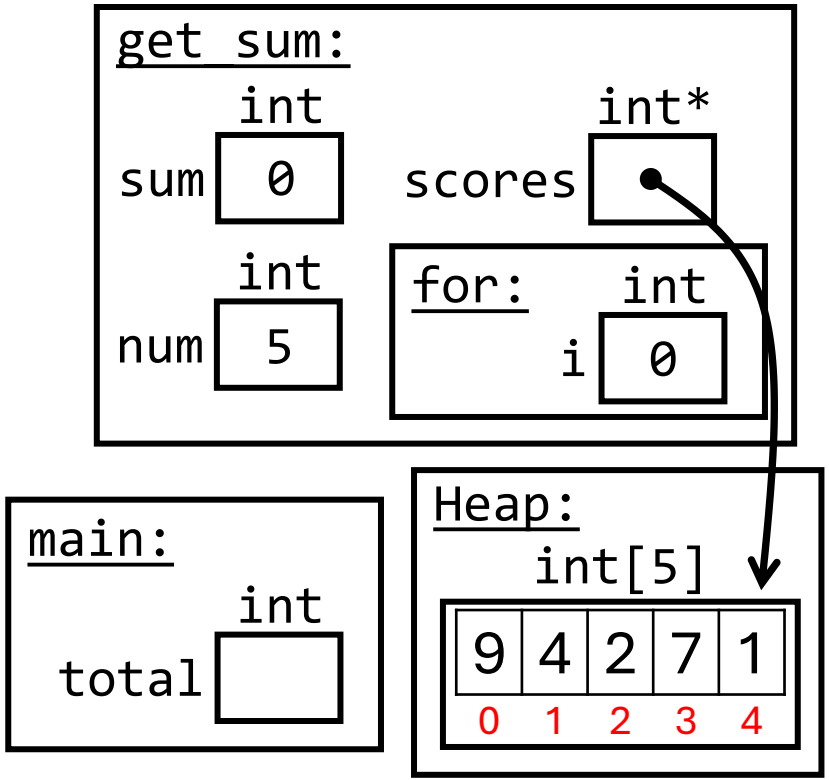
int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```



Dynamic Array Allocation Example

```
int get_sum() {
   int sum{0};
   int num{<read from user>};
   int* scores{new int[num]};
   <read values into array from file>
   for (int i{0}; i < num; i++) {
   ↳sum += scores[i];
   }
   delete[] scores;
   return sum;
}

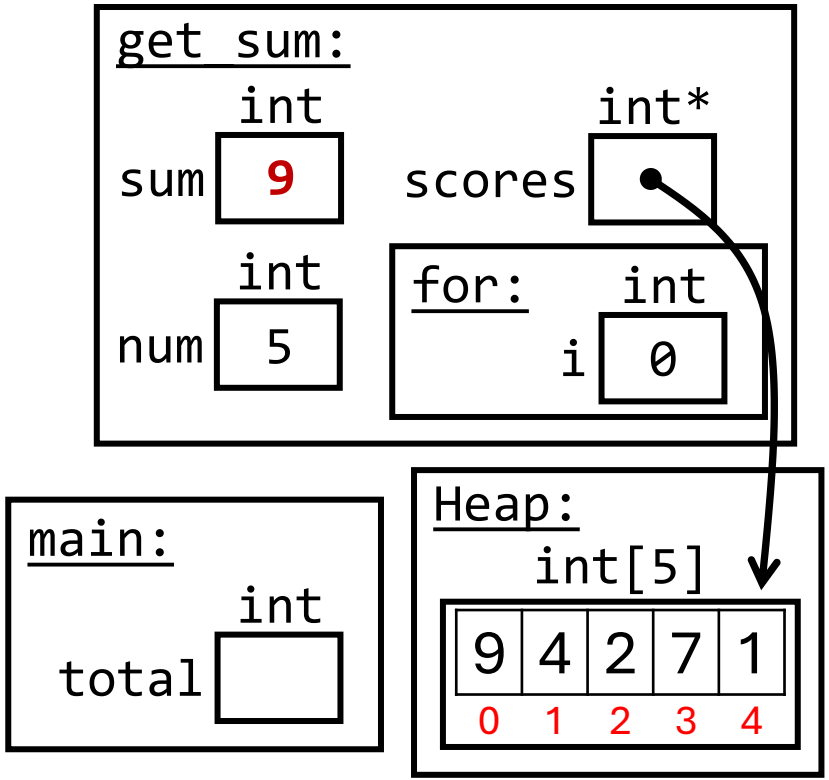
int main() {
   int total{get_sum()};
   std::cout << total << std::endl;
   return 0;
}
```



Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

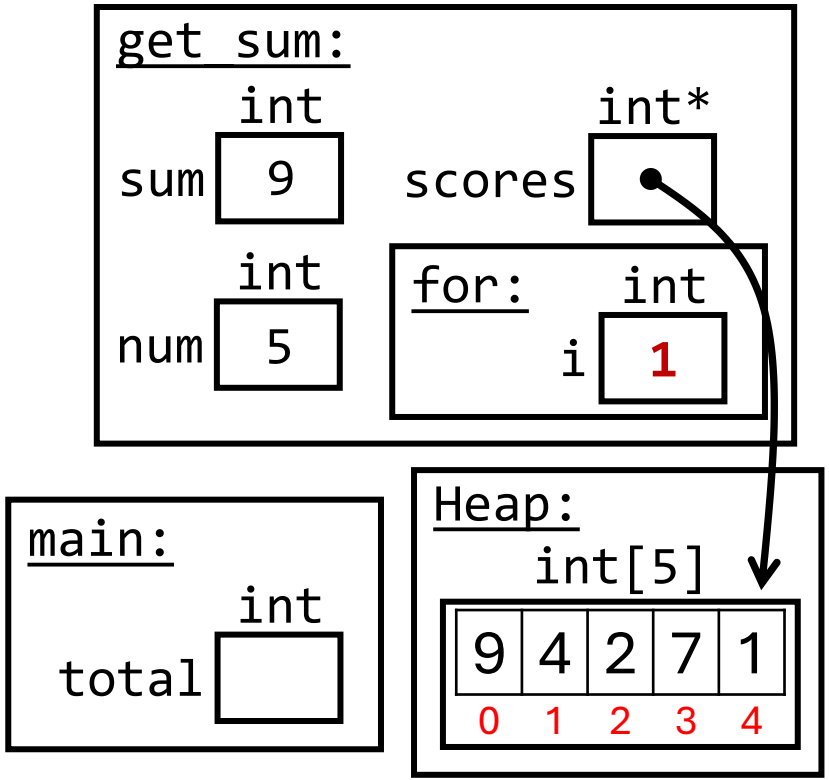
int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```



Dynamic Array Allocation Example

```
int get_sum() {
   int sum{0};
   int num{<read from user>};
   int* scores{new int[num]};
   <read values into array from file>
   for (int i{0}; i < num; i++) {
   ↳sum += scores[i];
   }
   delete[] scores;
   return sum;
}

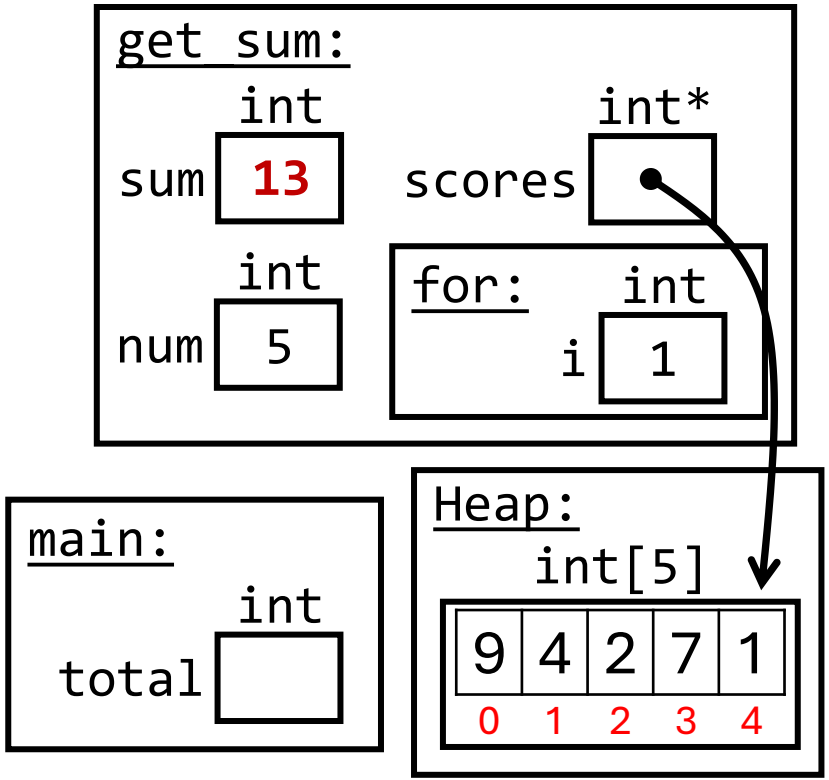
int main() {
   int total{get_sum()};
   std::cout << total << std::endl;
   return 0;
}
```



Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

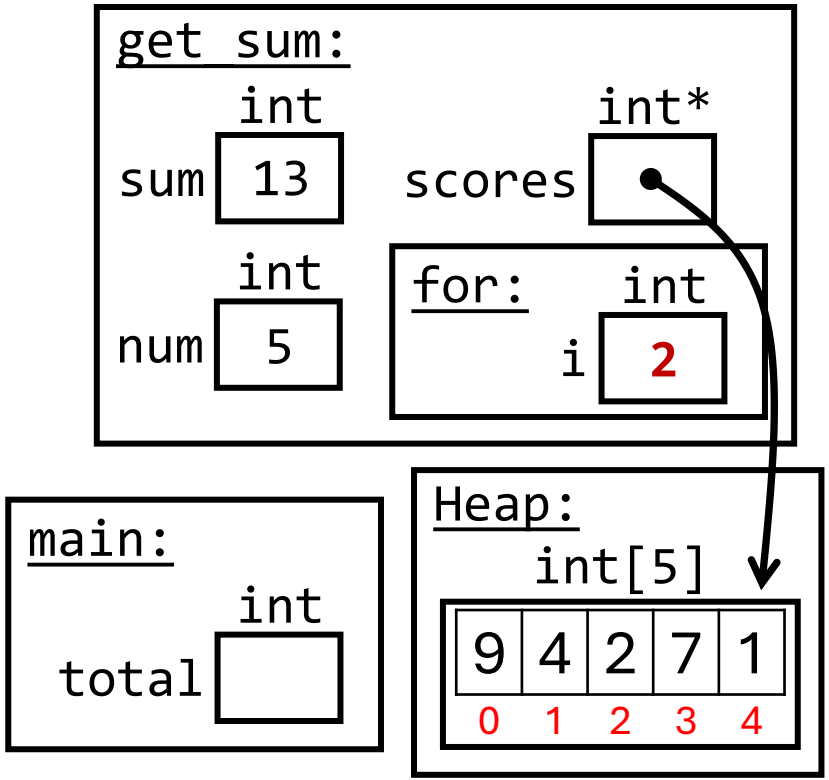
int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```



Dynamic Array Allocation Example

```
int get_sum() {
   int sum{0};
   int num{<read from user>};
   int* scores{new int[num]};
   <read values into array from file>
   for (int i{0}; i < num; i++) {
   ↳sum += scores[i];
   }
   delete[] scores;
   return sum;
}

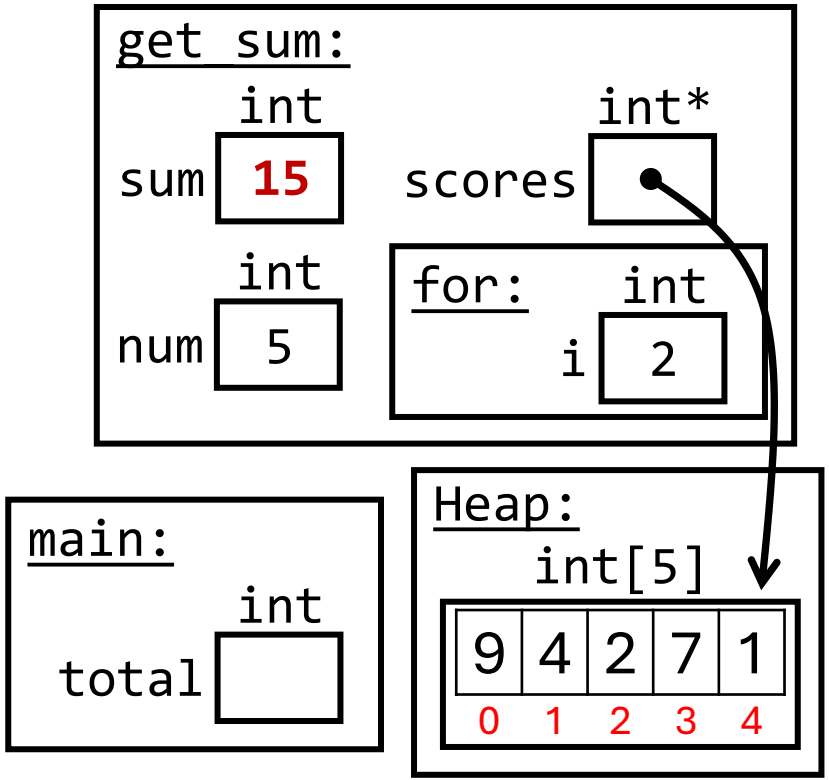
int main() {
   int total{get_sum()};
   std::cout << total << std::endl;
   return 0;
}
```



Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

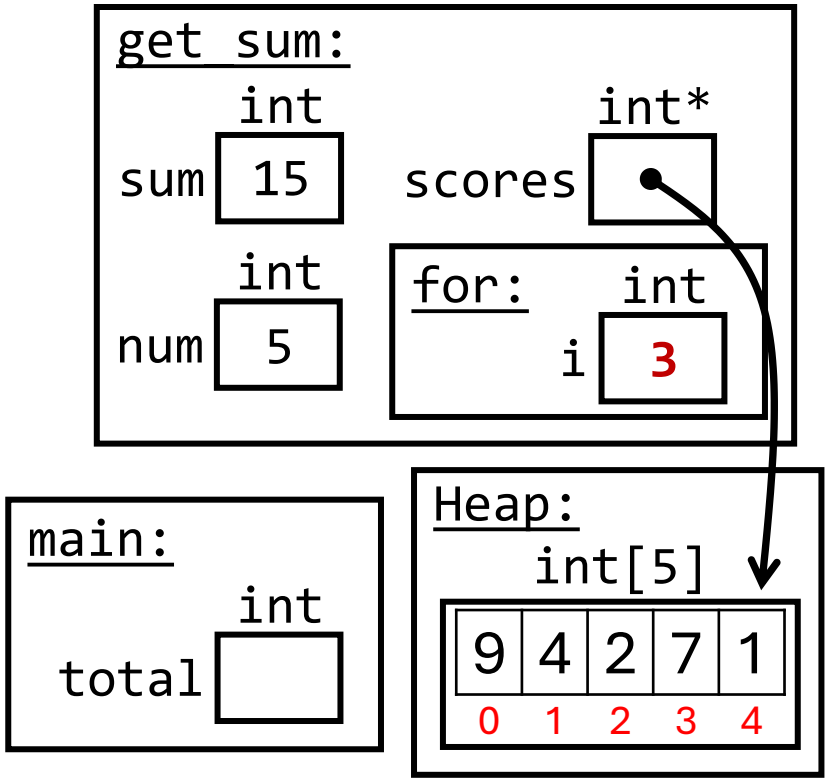
int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```



Dynamic Array Allocation Example

```
int get_sum() {
   int sum{0};
   int num{<read from user>};
   int* scores{new int[num]};
   <read values into array from file>
   for (int i{0}; i < num; i++) {
   ↳sum += scores[i];
   }
   delete[] scores;
   return sum;
}

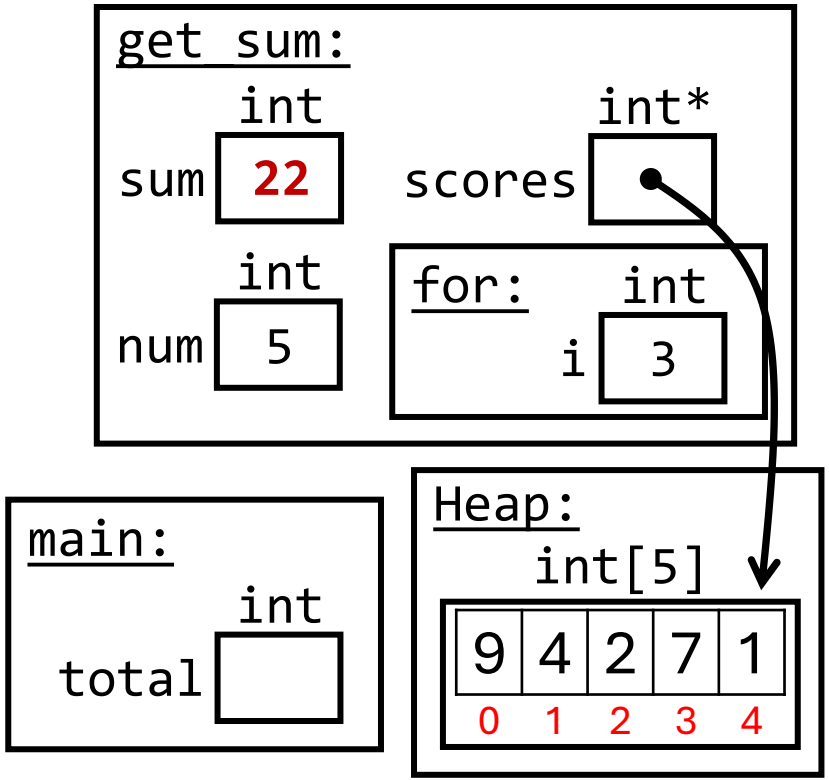
int main() {
   int total{get_sum()};
   std::cout << total << std::endl;
   return 0;
}
```



Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

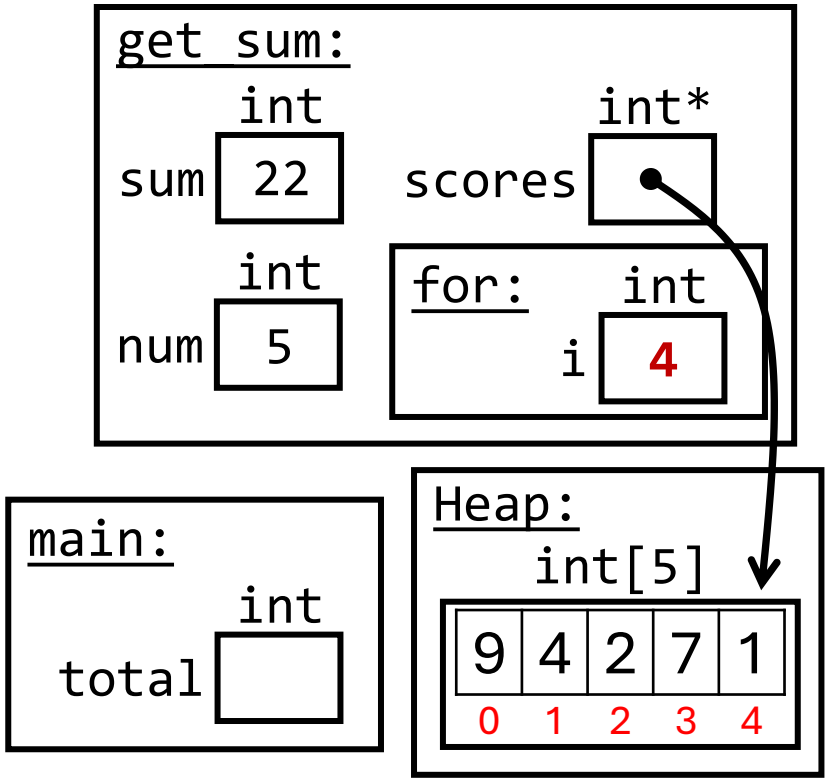
int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```



Dynamic Array Allocation Example

```
int get_sum() {
   int sum{0};
   int num{<read from user>};
   int* scores{new int[num]};
   <read values into array from file>
   for (int i{0}; i < num; i++) {
   ↳sum += scores[i];
   }
   delete[] scores;
   return sum;
}

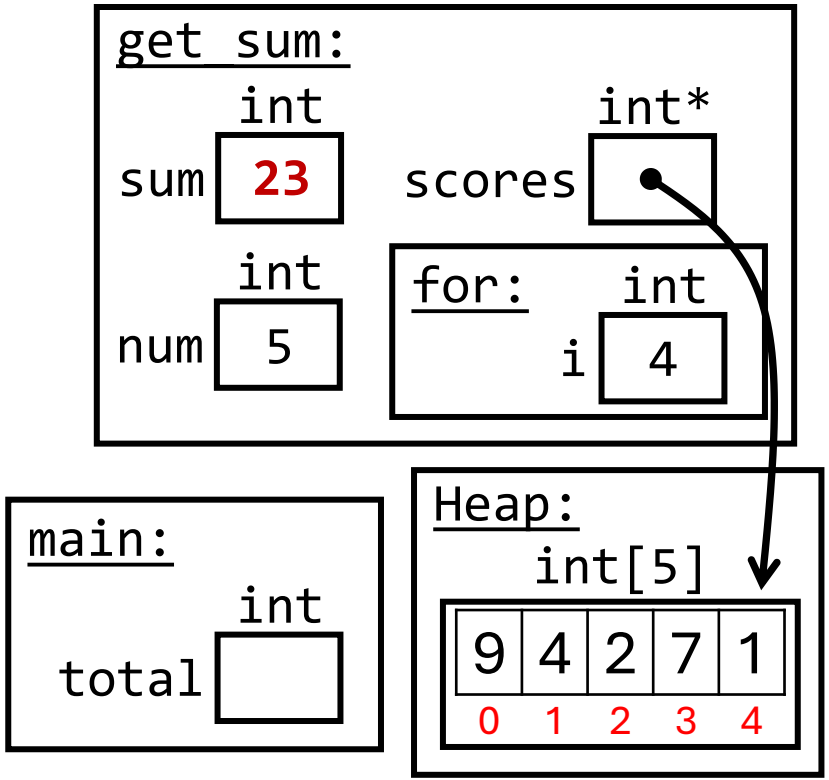
int main() {
   int total{get_sum()};
   std::cout << total << std::endl;
   return 0;
}
```



Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

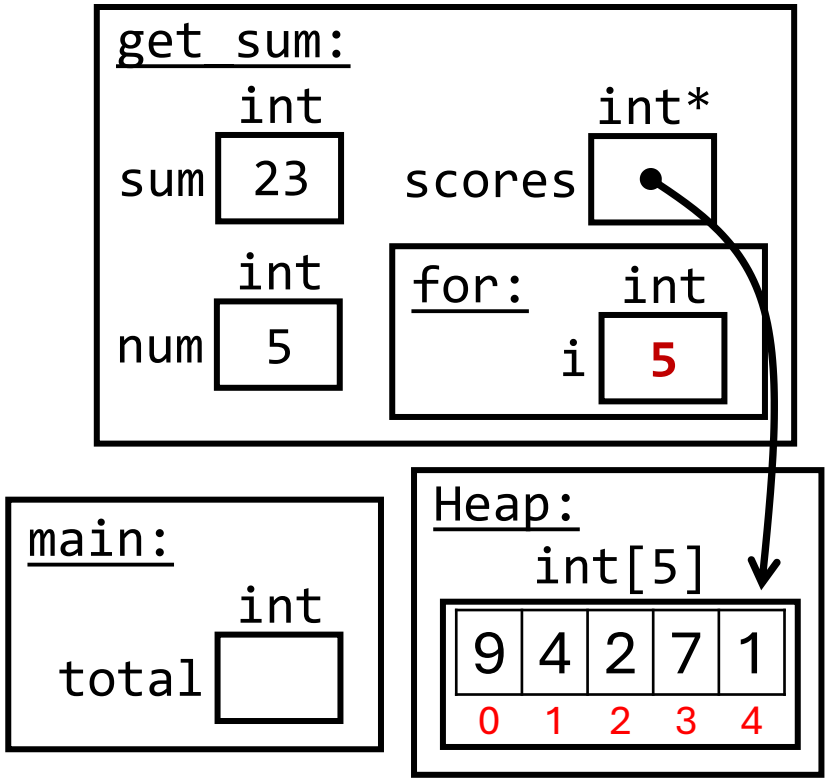
int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```



Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

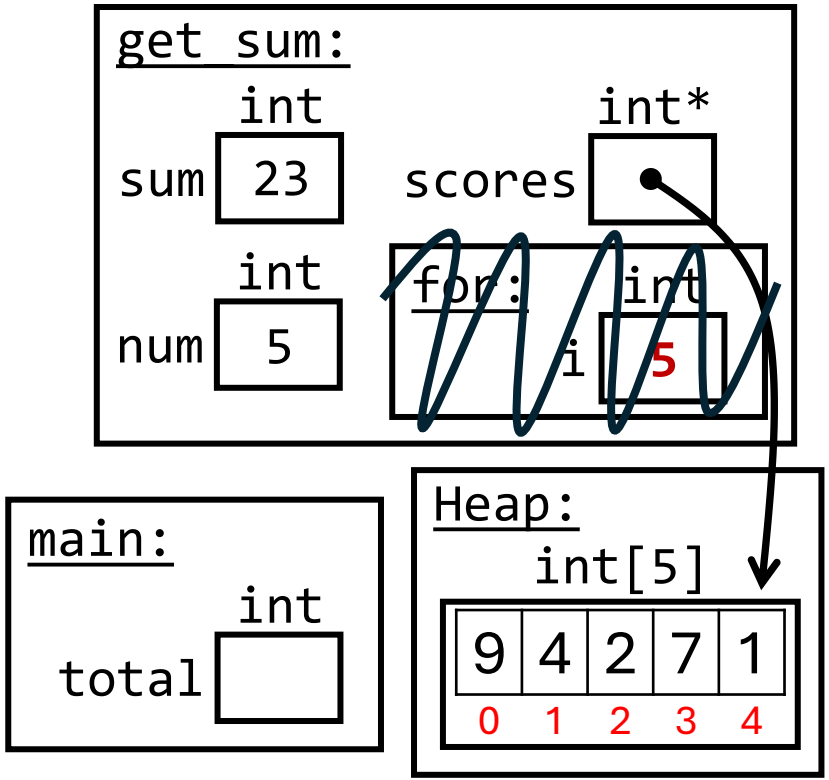
int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```



Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

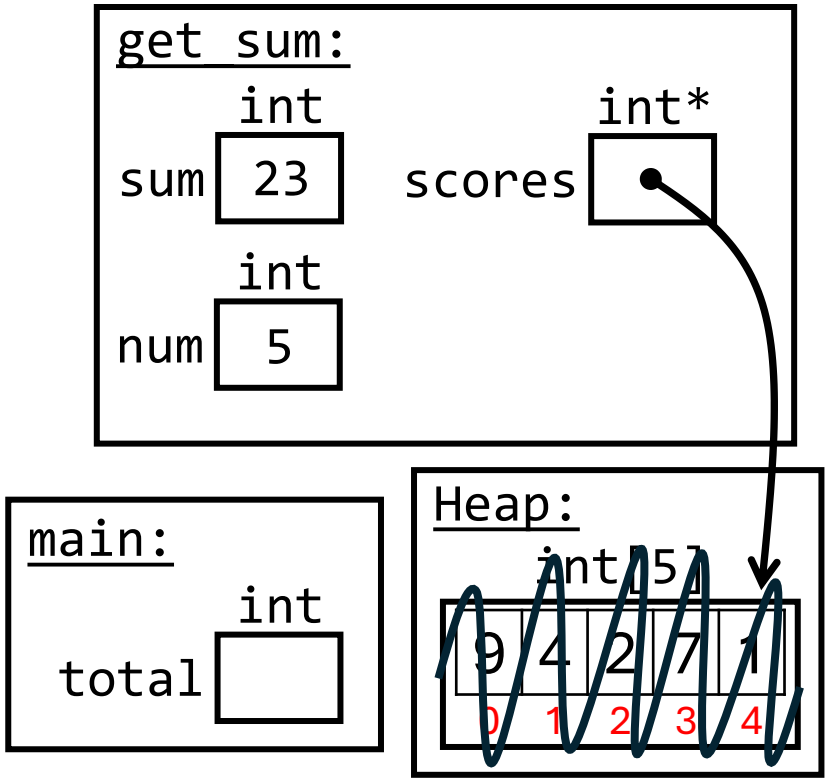
int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```



Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```

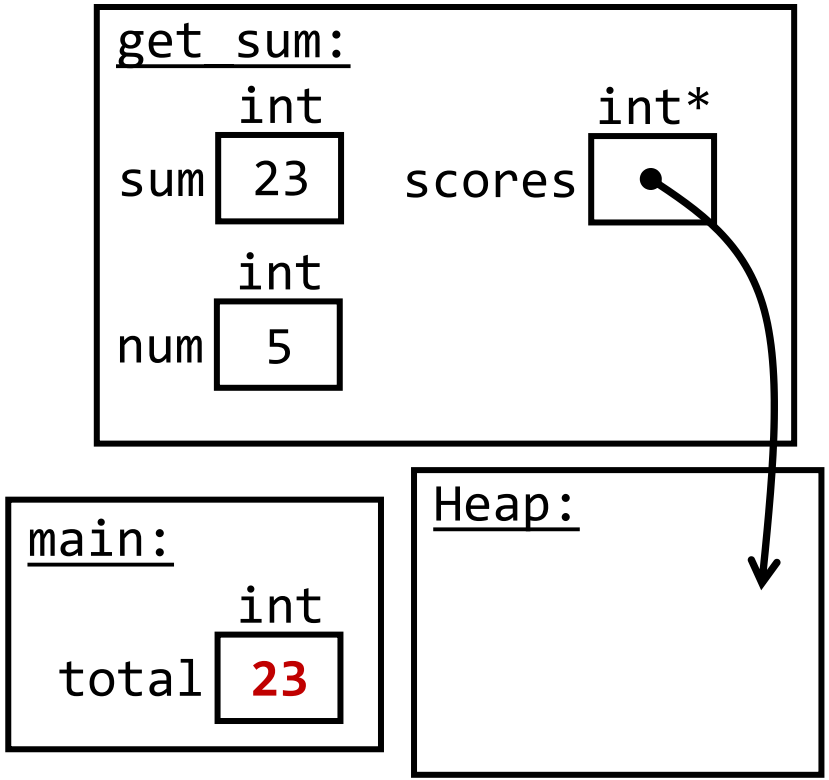


Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```

23

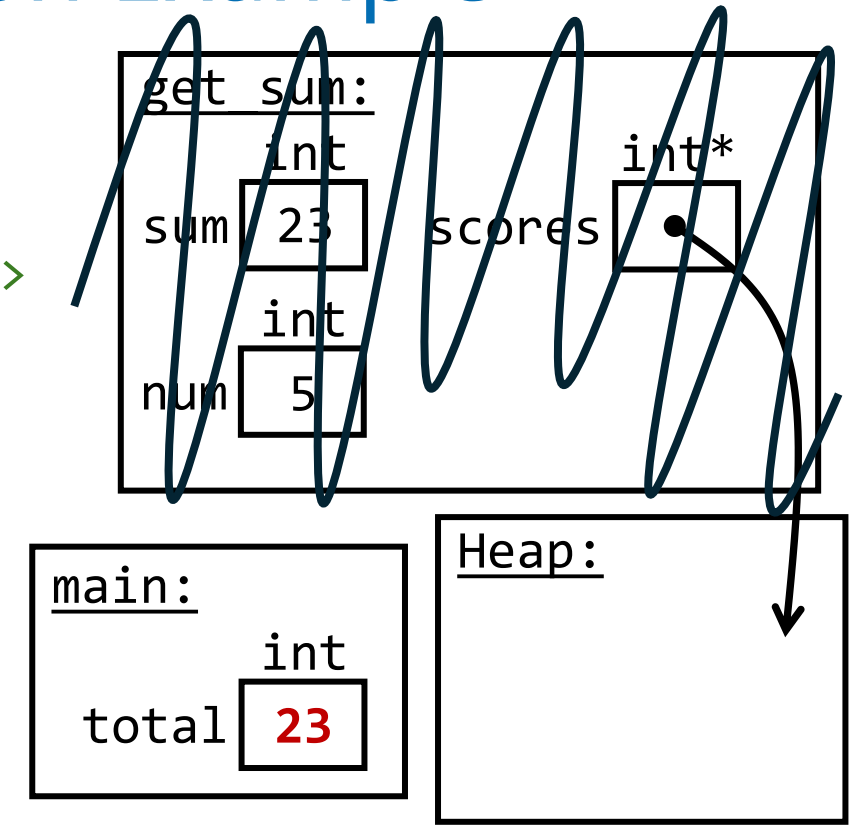


Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```

23

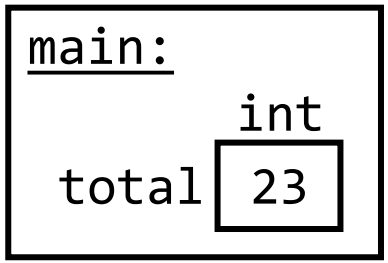


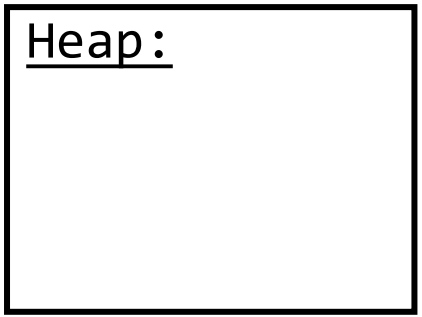
Dynamic Array Allocation Example

```
int get_sum() {
  int sum{0};
  int num{<read from user>};
  int* scores{new int[num]};
  <read values into array from file>
  for (int i{0}; i < num; i++) {
    sum += scores[i];
  }
  delete[] scores;
  return sum;
}

int main() {
  int total{get_sum()};
  std::cout << total << std::endl;
  return 0;
}
```

Output: 23





Dynamic Memory Summary

- Dynamic memory allocation allows us to do 2 new things:
  - It allows us to use memory allocated in a function after that function returns (we saw that heap memory is not removed with the function's stack frame)
  - It allows us to choose the size of the memory to be allocated when the program is running
- Remember, heap memory is not freed automatically:
  - Every **`new`** needs a matching **`delete`**
  - Array allocations need **`new[]`** and a matching **`delete[]`**
- Without the delete we get something called a **memory leak**

09: Memory Leaks

Anthony Estey
CSC 115: Fundamentals of Programming: II

Motivation

- We now know that every **`new`** needs a matching **`delete`**
  - Let’s take some time to explore what happens if we forget

- It isn't a compile error, and our program won’t just crash
  - The program will likely just keep running like nothing is wrong
  - This is exactly what makes this scenario dangerous!

Memory Leaks Example

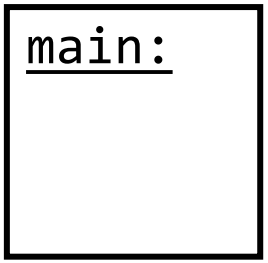
```
void leaky() {
    int* p{new int[5]};
}

int main() {
    leaky();
    int* p{new int[3]};
    p = new int[5];
    delete[] p;
    // more stuff here
    return 0;
}
```

Memory Leaks Example

```
void leaky() {
    int* p{new int[5]};
}

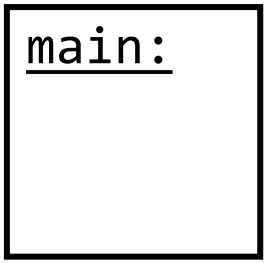
int main() {
    leaky();
    int* p{new int[3]};
    p = new int[5];
    delete[] p;
    // more stuff here
    return 0;
}
```



Memory Leaks Example

```
void leaky() {
    int* p{new int[5]};
}

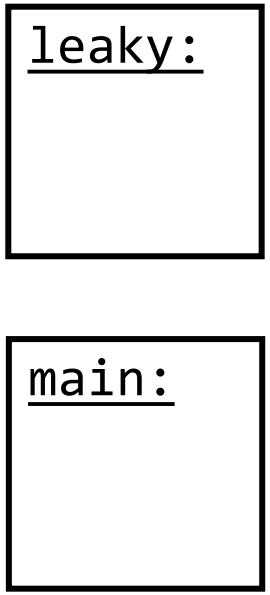
int main() {
    leaky();
    int* p{new int[3]};
    p = new int[5];
    delete[] p;
    // more stuff here
    return 0;
}
```



Memory Leaks Example

```
void leaky() {
    int* p{new int[5]};
}

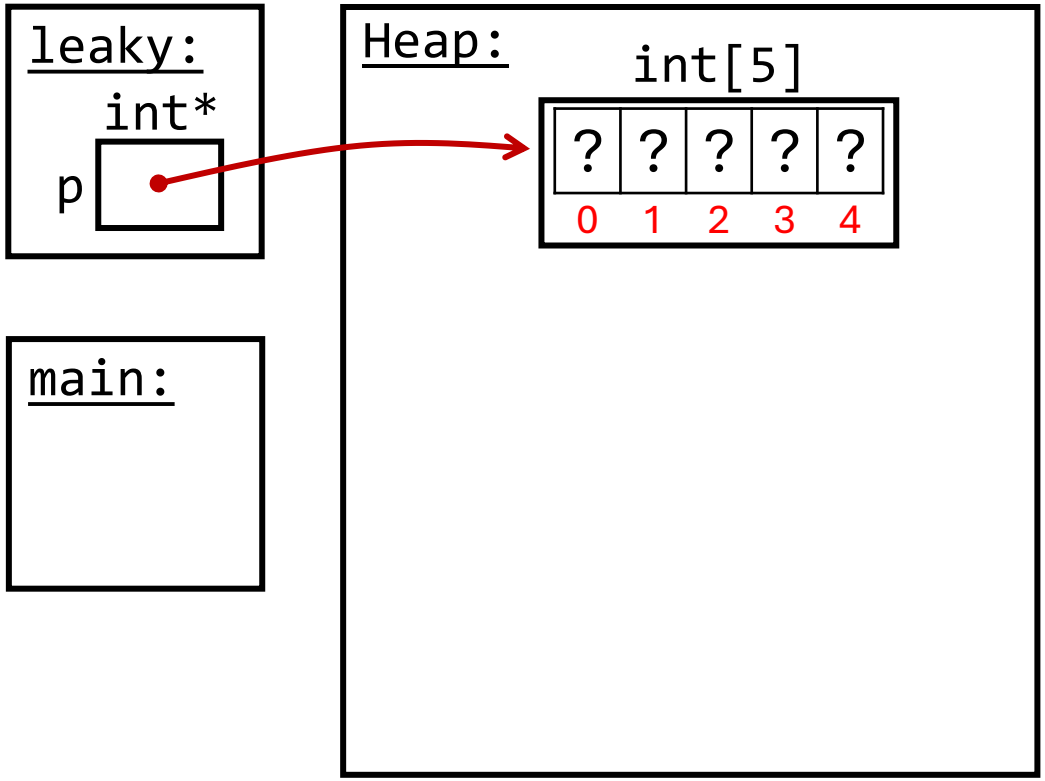
int main() {
    leaky();
    int* p{new int[3]};
    p = new int[5];
    delete[] p;
    // more stuff here
    return 0;
}
```



Memory Leaks Example

```
void leaky() {
    int* p{new int[5]};
}

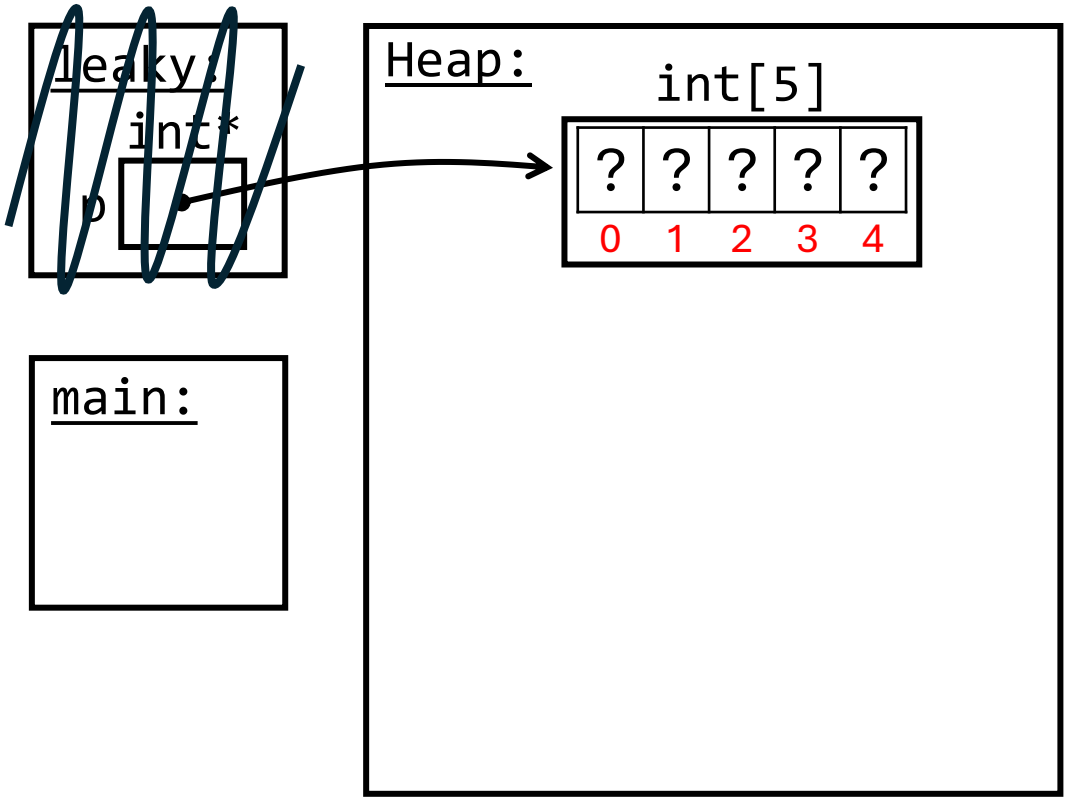
int main() {
    leaky();
    int* p{new int[3]};
    p = new int[5];
    delete[] p;
    // more stuff here
    return 0;
}
```



Memory Leaks Example

```
void leaky() {
    int* p{new int[5]};
}

int main() {
    leaky();
    int* p{new int[3]};
    p = new int[5];
    delete[] p;
    // more stuff here
    return 0;
}
```



Memory Leaks Example

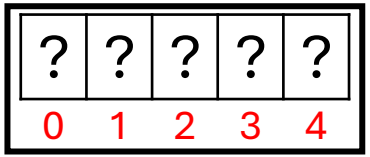
```
void leaky() {
    int* p{new int[5]};
}

int main() {
    leaky();
    int* p{new int[3]};
    p = new int[5];
    delete[] p;
    // more stuff here
    return 0;
}
```

main:

Heap:

int[5]

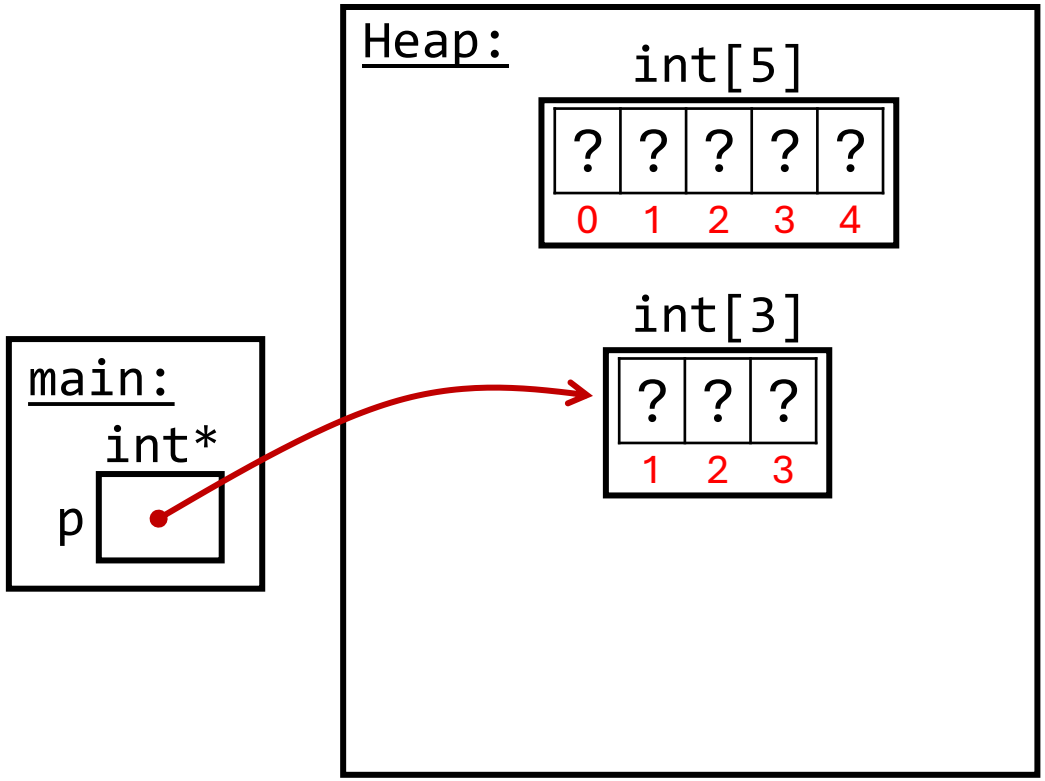


Big problem: Nothing is pointing to the array of 5 ints. We can't use it, and we also can't delete it! This memory is no longer reachable for the rest of the program's execution.

Memory Leaks Example

```
void leaky() {
    int* p{new int[5]};
}

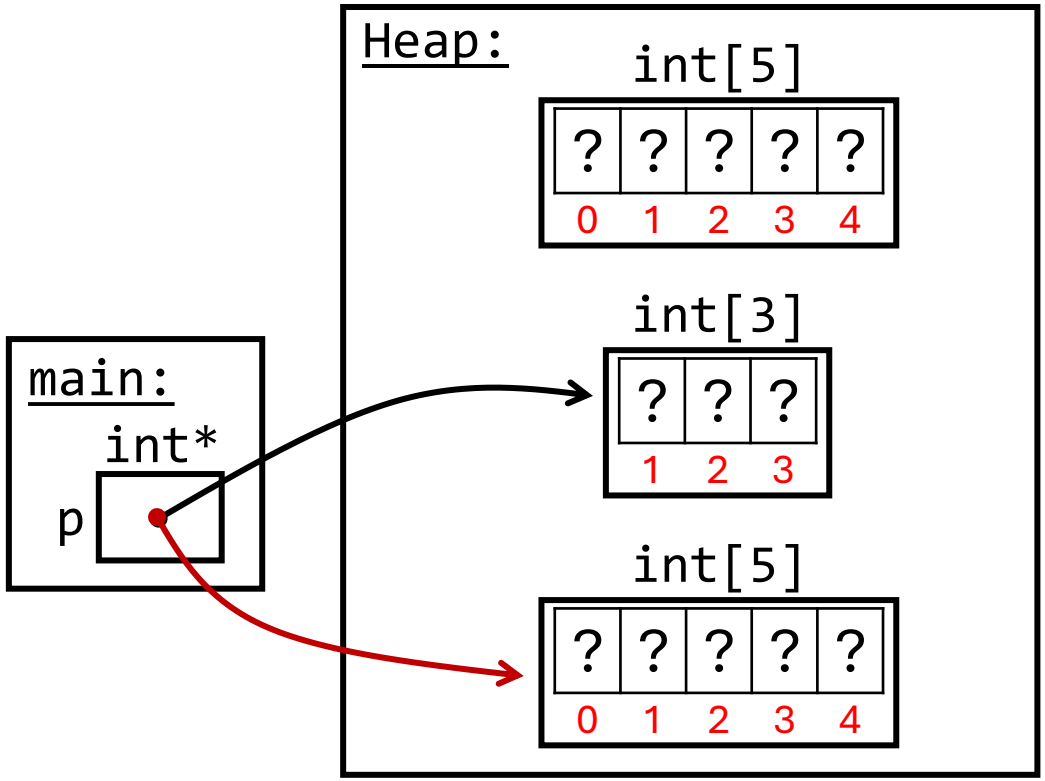
int main() {
    leaky();
    int* p{new int[3]};
    p = new int[5];
    delete[] p;
    // more stuff here
    return 0;
}
```



Memory Leaks Example

```
void leaky() {
    int* p{new int[5]};
}

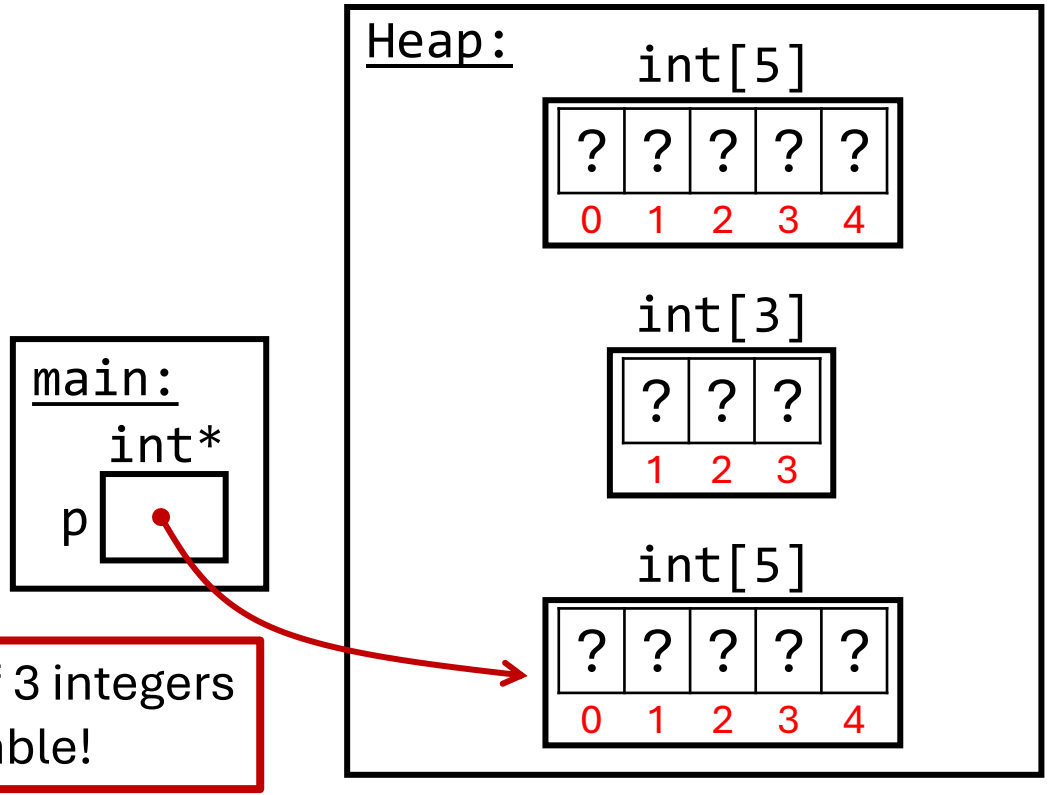
int main() {
    leaky();
    int* p{new int[3]};
    p = new int[5];
    delete[] p;
    // more stuff here
    return 0;
}
```



Memory Leaks Example

```
void leaky() {
    int* p{new int[5]};
}

int main() {
    leaky();
    int* p{new int[3]};
    p = new int[5];
    delete[] p;
    // more stuff here
    return 0;
}
```

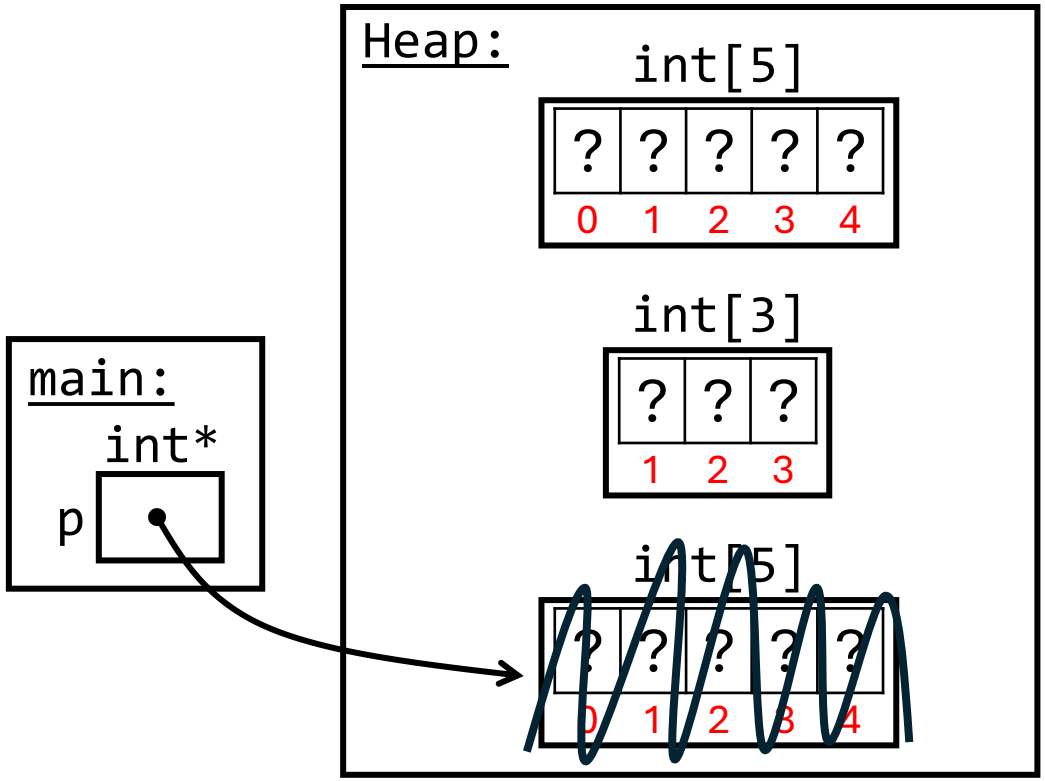


Now the array of 3 integers is also unreachable!

Memory Leaks Example

```
void leaky() {
    int* p{new int[5]};
}

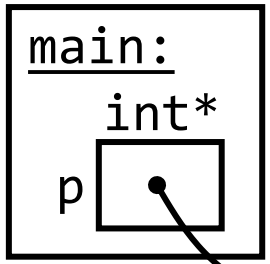
int main() {
    leaky();
    int* p{new int[3]};
    p = new int[5];
    delete[] p;
    // more stuff here
    return 0;
}
```

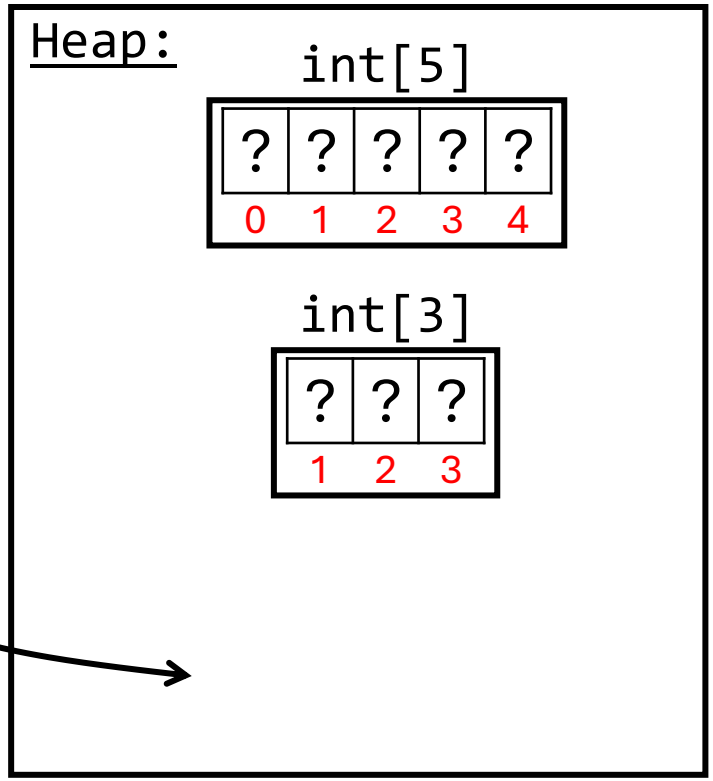


Memory Leaks Example

Note: p still holds the same address it held before the delete, but that memory has been freed, meaning the heap is free to reuse it for the next allocation, at any time.

```
int main() {
    leaky();
    int* p{new int[3]};
    p = new int[5];
    delete[] p;
    // more stuff here
    return 0;
}
```

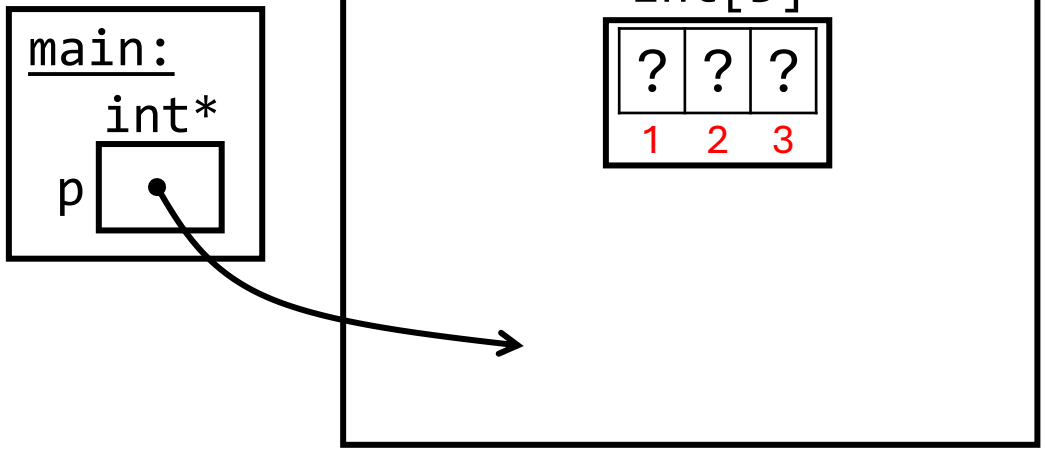




Memory Leaks Example

This is dangerous because p can still be used as if it were pointing to an array. The problem is that we have no idea what values may now be stored at that location!

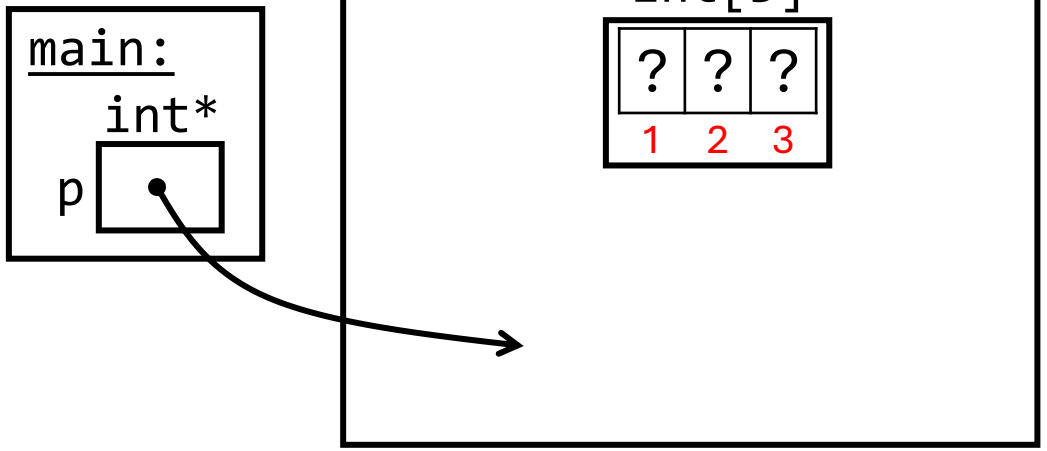
```
int main() {
    leaky();
    int* p{new int[3]};
    p = new int[5];
    delete[] p;
    // more stuff here
    return 0;
}
```



Memory Leaks Example

This is called a **dangling pointer:** something holding a real address that is unsafe to use. This is dangerous because C++ gives us no warning or compile error if we do try to use it.

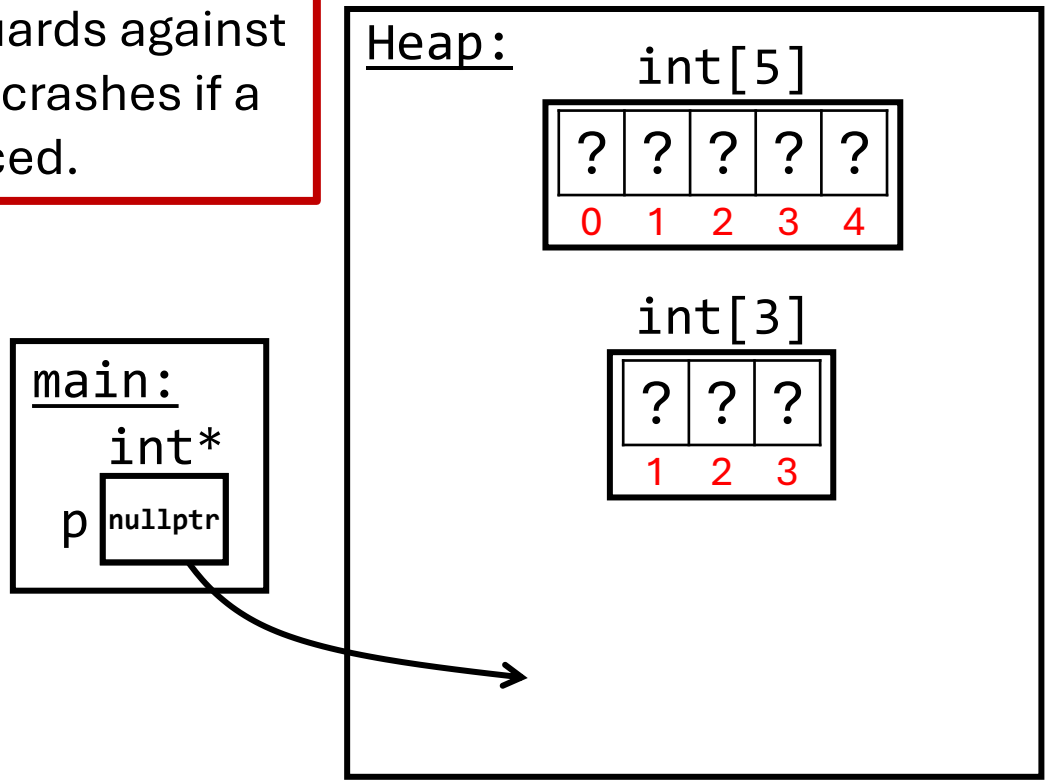
```
int main() {
    leaky();
    int* p{new int[3]};
    p = new int[5];
    delete[] p;
    // more stuff here
    return 0;
}
```



Memory Leaks Example

Some programmers would add p = nullptr; immediately after delete[]. This guards against accidental usage, as the program crashes if a pointer set to nullptr is dereferenced.

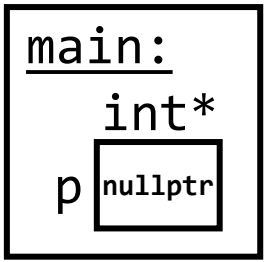
```
int main() {
    leaky();
    int* p{new int[3]};
    p = new int[5];
    delete[] p;
    // more stuff here
    return 0;
}
```

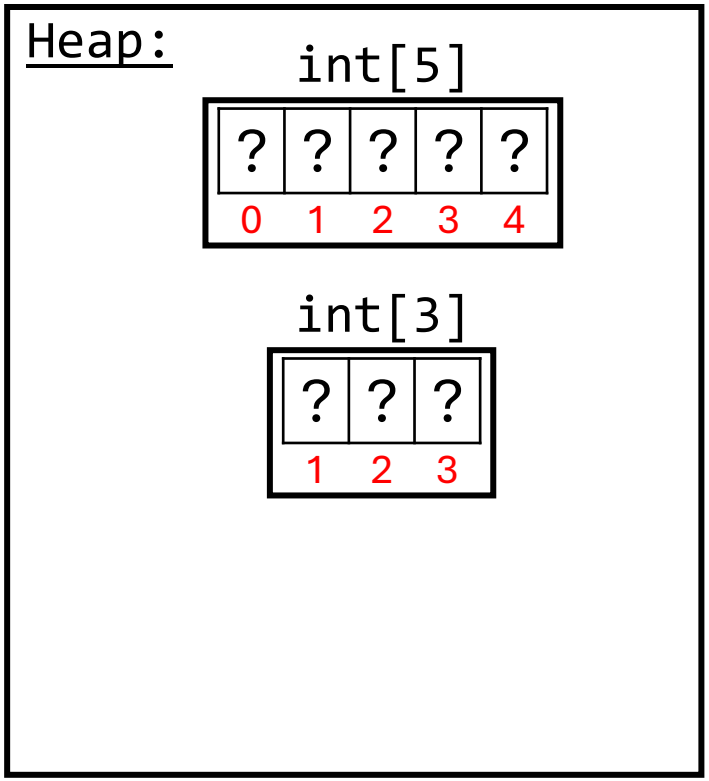


Memory Leaks Example

Back to memory leaks: Any unreachable heap memory is called a memory leak. There are already two in this program, and in a real, long-running program, this count will only grow.

```
int main() {
    leaky();
    int* p{new int[3]};
    p = new int[5];
    delete[] p;
    // more stuff here
    return 0;
}
```





Consequences of Memory Leaks

- A single leak in a short-running program often doesn't matter, as memory assigned to that program is reclaimed once it finishes its execution
- In any long-running program, the leaks will accumulate: memory usage climbs until performance suffers or the program crashes.
  - This is why we need to make sure to match every **`new`** with a corresponding **`delete`**

Other Considerations

- In general, it is considered best practice to free memory as soon as it's no longer needed:
  - One reason is just to be safe: make sure the memory is handled correctly and then we don't need to remember to do it later
- A consistent strategy also helps us avoid other problems:
  - When a pointer is passed between functions, it's not always obvious *whose job* it is to eventually delete it.
  - Both could delete it, but deleting the same pointer twice, called a 'double free' also results in undefined behaviour for the program
  - My suggestion: when we know we are done with it, delete it!
  - To be extra safe: some programmers set their pointers to `nullptr` after freeing the memory they are pointing to (avoids double free)

Going deeper

- Remember: a pointer holds an address and that's it
  - Pointers to arrays don't also hold the length of the array
  - Nor do they keep a record to whether the contents at that address have been freed or not (or who is responsible for freeing it!)
  - **They just store a single address**
- The compiler can't really help us manage Heap memory either:
  - Double-free isn't detected, because nothing keeps track of whether memory has already been freed
  - Ownership of the Heap data is ambiguous: C++ has no built-in idea of who should free the memory, that's up to us to track
  - Array size is tracked somewhere (just not on the pointer itself), which is why we use **delete[]** for arrays specifically, so C++ can free up the right amount of memory

Summary

- A memory leak happens when heap memory becomes unreachable before it's freed
- There is no compile error, no crash, just memory that's gone for good until the program ends.
- The heap's flexibility is powerful, but the cleanup responsibility is entirely ours.
  - Keeping track of all of our heap memory can be a lot to organize, and is easy to get wrong
  - One of the things we'll consider in the next unit is creating an object that can take care of its own cleanup automatically!

Memory Leaks Example

```
void leaky() {
    int* p{new int[5]};
}

int main() {
    leaky();
    int* p{new int[3]};
    p = new int[5];
    delete[] p;
    // more stuff here
    return 0;
}
```

